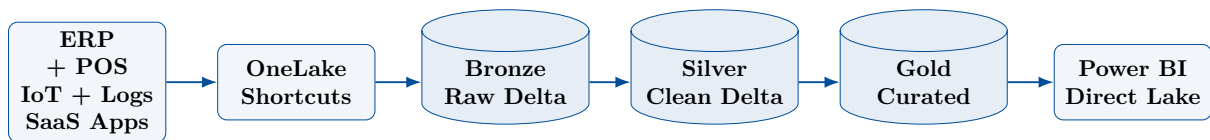


---

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

---

# Microsoft Fabric Master Data Engineer



*OneLake medallion pipeline without unnecessary copies*



*From OneLake foundations to governed, production-grade analytics products.*

---

**PhD Aned Esquerra Arguelles**

A 24-week Microsoft Fabric data engineering program

July 31, 2026

---

# Preface

Welcome to *Microsoft Fabric Master Data Engineer*. This book is a rigorous, hands-on academic guide for learning how modern data engineering changes when storage, compute, governance, orchestration, streaming, machine learning, and business intelligence are delivered through the unified Microsoft Fabric software-as-a-service platform.

The goal is not to memorize product screens. The goal is to develop the architecture judgment expected of a master data engineer: when to use OneLake shortcuts instead of copying data, when to model data as Delta tables versus warehouse tables, how to organize medallion layers, how to reason about capacity units, and how to publish governed analytics products that Power BI users can trust.

## Who This Book Is For

This book is written for data engineers, analytics engineers, BI developers, cloud architects, database professionals, and technical leaders who need to design and operate data platforms on Microsoft Fabric. It assumes basic SQL literacy and general familiarity with cloud storage, notebooks, and batch pipelines. Python and PySpark are introduced through practical listings, and the exercises gradually move from guided operations to architecture design.

## How This Book Is Organized

The program is organized as **6 Parts**, matching the six Fabric milestones of the training plan, and **24 chapters**, matching 24 instructional weeks plus appendices:

- **Part I: Core Data Infrastructure and OneLake** establishes workspaces, capacities, OneLake, lakehouses, Delta-Parquet, and medallion architecture.
- **Part II: Data Warehousing and Analytics** builds SQL warehouses, semantic models, Direct Lake datasets, and performance-aware reporting paths.
- **Part III: Data Engineering and ETL Orchestration** develops Spark notebooks, Dataflows Gen2, Data Factory pipelines, environments, deployment patterns, and production ETL.
- **Part IV: Real-Time Intelligence and Streaming** introduces Eventstreams, KQL databases, real-time dashboards, and operational telemetry scenarios.
- **Part V: Data Science and Machine Learning** applies Fabric notebooks, feature engineering, MLflow, experiments, and model operationalization to data products.
- **Part VI: Governance, Security, and Exam Prep** covers domains, Purview integration, sensitivity, lineage, access control, monitoring, cost management, and certification-style review.

## Reading Paths

- **New to Fabric:** read Part I in order, complete every hands-on lab, and draw each architecture before implementing it.
- **Experienced Azure data engineer:** focus on OneLake, shortcuts, Direct Lake, capacity management, and Fabric governance differences from separate Azure services.
- **BI-first professional:** read Chapters 1, 5–8, and 21–24 to connect semantic models and Power BI delivery to governed lakehouse design.
- **Platform architect:** read each Friday use case first, then work backward through the week’s concepts to validate design trade-offs.
- **Exam preparation:** use each chapter’s Key Takeaways, Interview Q&A, and Exercises as spaced-repetition checkpoints.

## Conventions

Code identifiers, workspace names, lakehouse paths, and commands appear in `monospace`. PySpark listings use the shared `pystyle` listing style. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations point to product documentation and data-management research in the bibliography. Hands-on steps are written for the Microsoft Fabric web experience and PowerShell-friendly Windows workflows unless a cloud service explicitly requires a browser action.

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>0</b> | <b>Getting Started: Your Microsoft Fabric Trial and Workspace</b> | <b>1</b>  |
| 0.1      | Why Chapter 0 Exists . . . . .                                    | 2         |
| 0.2      | Prerequisites: Identity Before Infrastructure . . . . .           | 3         |
| 0.2.1    | Free Ways to Obtain an Organizational Account . . . . .           | 3         |
| 0.2.2    | Minimum Access You Need . . . . .                                 | 4         |
| 0.3      | Starting the Free Fabric Trial . . . . .                          | 4         |
| 0.3.1    | What the Trial Includes . . . . .                                 | 4         |
| 0.3.2    | Trial Capacity Versus F SKUs . . . . .                            | 5         |
| 0.4      | Enabling Fabric at the Tenant . . . . .                           | 5         |
| 0.4.1    | Delegating to Capacity Administrators . . . . .                   | 5         |
| 0.5      | Creating Your Workspace . . . . .                                 | 6         |
| 0.5.1    | Workspace Roles . . . . .                                         | 6         |
| 0.5.2    | Creating Your First Lakehouse . . . . .                           | 6         |
| 0.6      | Tooling: Browser First, Optional Desktop Tools . . . . .          | 7         |
| 0.6.1    | A First PySpark Read and Delta Write . . . . .                    | 7         |
| 0.6.2    | REST and JSON Awareness . . . . .                                 | 8         |
| 0.7      | Cost and Quota Guardrails . . . . .                               | 8         |
| 0.7.1    | Trial Expiry . . . . .                                            | 8         |
| 0.7.2    | Bursting and Throttling . . . . .                                 | 9         |
| 0.7.3    | Paid Capacity Later . . . . .                                     | 9         |
| 0.8      | Verify the Setup . . . . .                                        | 9         |
| 0.8.1    | Step 1: Create the Workspace and Lakehouse . . . . .              | 9         |
| 0.8.2    | Step 2: Upload a Sample File to OneLake . . . . .                 | 9         |
| 0.8.3    | Step 3: Run a One-Cell Notebook . . . . .                         | 10        |
| 0.8.4    | Troubleshooting Checklist . . . . .                               | 11        |
| 0.9      | How This Setup Differs from Raw IaaS Clouds . . . . .             | 11        |
| 0.10     | Further Reading . . . . .                                         | 11        |
| 0.11     | Exercises . . . . .                                               | 12        |
| <b>I</b> | <b>Core Data Infrastructure and OneLake</b>                       | <b>13</b> |
| <b>1</b> | <b>OneLake and Microsoft Fabric Foundations</b>                   | <b>14</b> |
| 1.1      | Week 1 Roadmap . . . . .                                          | 15        |
| 1.2      | Monday: Introduction to Microsoft Fabric . . . . .                | 16        |
| 1.2.1    | The Workload Map . . . . .                                        | 16        |
| 1.2.2    | Capacities, F SKUs, and Capacity Units . . . . .                  | 17        |
| 1.2.3    | Workspaces and Domains . . . . .                                  | 18        |
| 1.2.4    | Monday Architecture Lens . . . . .                                | 18        |
| 1.3      | Tuesday: OneLake Concepts . . . . .                               | 18        |

|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| 1.3.1    | Workspaces, Items, Folders, and Paths . . . . .                                      | 19        |
| 1.3.2    | Delta-Parquet as the Native Open Format . . . . .                                    | 19        |
| 1.3.3    | Medallion Thinking in OneLake . . . . .                                              | 19        |
| 1.3.4    | Tuesday Engineering Lens . . . . .                                                   | 19        |
| 1.4      | Wednesday: OneLake Shortcuts . . . . .                                               | 20        |
| 1.4.1    | Internal and External Shortcuts . . . . .                                            | 20        |
| 1.4.2    | When Virtualization Beats Copying . . . . .                                          | 20        |
| 1.4.3    | Risks and Controls . . . . .                                                         | 21        |
| 1.4.4    | Shortcut Decision Matrix . . . . .                                                   | 21        |
| 1.5      | Thursday: Hands-on Project . . . . .                                                 | 21        |
| 1.5.1    | Goal and Architecture . . . . .                                                      | 21        |
| 1.5.2    | Step-by-Step Actions . . . . .                                                       | 22        |
| 1.5.3    | Representing the Shortcut as Configuration . . . . .                                 | 23        |
| 1.5.4    | Optional SQL Check . . . . .                                                         | 23        |
| 1.5.5    | Validation Checklist . . . . .                                                       | 23        |
| 1.6      | Friday: Use Case and Architecture . . . . .                                          | 24        |
| 1.6.1    | Architecture Strategy . . . . .                                                      | 24        |
| 1.6.2    | Design Decisions . . . . .                                                           | 24        |
| 1.7      | Operational Guidance for Week 1 . . . . .                                            | 25        |
| 1.7.1    | Governance Checklist . . . . .                                                       | 25        |
| 1.7.2    | Quality and Observability . . . . .                                                  | 25        |
| 1.7.3    | Interview Q&A . . . . .                                                              | 26        |
| 1.8      | Further Reading . . . . .                                                            | 26        |
| 1.9      | Exercises . . . . .                                                                  | 27        |
| 1.10     | Capstone Project: A Multi-Cloud Retail Landing Zone with OneLake Shortcuts . . . . . | 27        |
| <b>2</b> | <b>The Medallion Architecture</b>                                                    | <b>33</b> |
| 2.1      | Week 2 Roadmap . . . . .                                                             | 34        |
| 2.2      | Monday: Medallion Principles . . . . .                                               | 35        |
| 2.2.1    | Bronze: Raw Landing and Source Fidelity . . . . .                                    | 35        |
| 2.2.2    | Silver: Validation, Standardization, and Conformance . . . . .                       | 35        |
| 2.2.3    | Gold: Enriched, Curated, and Consumption Ready . . . . .                             | 36        |
| 2.2.4    | Layer Contracts . . . . .                                                            | 36        |
| 2.3      | Tuesday: Lakehouse vs Data Warehouse in Fabric . . . . .                             | 36        |
| 2.3.1    | When to Choose a Lakehouse . . . . .                                                 | 37        |
| 2.3.2    | When to Choose a Warehouse . . . . .                                                 | 37        |
| 2.3.3    | Decision Aid . . . . .                                                               | 37        |
| 2.4      | Wednesday: Organizing Workspaces and Lakehouses at Scale . . . . .                   | 38        |
| 2.4.1    | Single Lakehouse Pattern . . . . .                                                   | 38        |
| 2.4.2    | Three Lakehouses in One Workspace . . . . .                                          | 38        |
| 2.4.3    | Multi-Workspace Pattern . . . . .                                                    | 38        |
| 2.4.4    | Naming, Ownership, and Promotion . . . . .                                           | 38        |
| 2.5      | Thursday: Hands-on Project . . . . .                                                 | 39        |
| 2.5.1    | Goal and Architecture . . . . .                                                      | 39        |
| 2.5.2    | Step-by-Step Actions . . . . .                                                       | 39        |
| 2.5.3    | Optional Spark SQL Validation . . . . .                                              | 41        |
| 2.5.4    | Validation Checklist . . . . .                                                       | 41        |
| 2.6      | Friday: Use Case and Architecture . . . . .                                          | 41        |
| 2.6.1    | Architecture Strategy . . . . .                                                      | 42        |
| 2.6.2    | Design Decisions . . . . .                                                           | 42        |
| 2.7      | Operational Guidance for Week 2 . . . . .                                            | 43        |

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| 2.7.1    | Quality and Observability . . . . .                                           | 43        |
| 2.7.2    | Security Checklist . . . . .                                                  | 43        |
| 2.8      | Interview Q&A . . . . .                                                       | 44        |
| 2.9      | Further Reading . . . . .                                                     | 44        |
| 2.10     | Exercises . . . . .                                                           | 45        |
| 2.11     | Capstone Project: Financial Medallion with Workspace Isolation . . . . .      | 45        |
| <b>3</b> | <b>Apache Spark in Fabric</b>                                                 | <b>50</b> |
| 3.1      | Week 3 Roadmap . . . . .                                                      | 51        |
| 3.2      | Monday: Spark Compute in Fabric . . . . .                                     | 52        |
| 3.2.1    | Starter Pools . . . . .                                                       | 52        |
| 3.2.2    | Custom Pools . . . . .                                                        | 52        |
| 3.2.3    | Spark Environments . . . . .                                                  | 53        |
| 3.3      | Tuesday: Fabric Notebooks and PySpark Basics . . . . .                        | 53        |
| 3.3.1    | The PySpark DataFrame Model . . . . .                                         | 53        |
| 3.3.2    | Reading Files from OneLake . . . . .                                          | 54        |
| 3.3.3    | Writing Delta Tables . . . . .                                                | 54        |
| 3.4      | Wednesday: Spark SQL and Hybrid Notebooks . . . . .                           | 54        |
| 3.4.1    | Temporary Views . . . . .                                                     | 54        |
| 3.4.2    | Choosing DataFrame or SQL Syntax . . . . .                                    | 55        |
| 3.5      | Thursday: Hands-on Project . . . . .                                          | 55        |
| 3.5.1    | Goal and Architecture . . . . .                                               | 55        |
| 3.5.2    | Step-by-Step Actions . . . . .                                                | 55        |
| 3.5.3    | Validation Checklist . . . . .                                                | 57        |
| 3.6      | Friday: Use Case and Architecture . . . . .                                   | 57        |
| 3.6.1    | Architecture Strategy . . . . .                                               | 58        |
| 3.6.2    | Design Decisions . . . . .                                                    | 58        |
| 3.7      | Operational Guidance for Week 3 . . . . .                                     | 58        |
| 3.7.1    | Performance and Reliability . . . . .                                         | 58        |
| 3.7.2    | Security Checklist . . . . .                                                  | 59        |
| 3.8      | Interview Q&A . . . . .                                                       | 59        |
| 3.9      | Further Reading . . . . .                                                     | 59        |
| 3.10     | Exercises . . . . .                                                           | 60        |
| 3.11     | Capstone Project: Spark-Based Telemetry Ingestion at 100 GB per Day . . . . . | 61        |
| <b>4</b> | <b>Delta Lake Optimization</b>                                                | <b>66</b> |
| 4.1      | Week 4 Roadmap . . . . .                                                      | 67        |
| 4.2      | Monday: Delta Lake Internals . . . . .                                        | 68        |
| 4.2.1    | Parquet Data Files . . . . .                                                  | 68        |
| 4.2.2    | The JSON Transaction Log . . . . .                                            | 68        |
| 4.2.3    | Why Internals Affect Optimization . . . . .                                   | 69        |
| 4.3      | Tuesday: Optimizing Delta Tables . . . . .                                    | 69        |
| 4.3.1    | V-Order in Fabric . . . . .                                                   | 69        |
| 4.3.2    | OPTIMIZE for Compaction . . . . .                                             | 70        |
| 4.3.3    | Z-Ordering . . . . .                                                          | 70        |
| 4.4      | Wednesday: Managing Delta Tables . . . . .                                    | 70        |
| 4.4.1    | VACUUM and Retention . . . . .                                                | 70        |
| 4.4.2    | Time Travel . . . . .                                                         | 71        |
| 4.4.3    | Schema Evolution . . . . .                                                    | 71        |
| 4.5      | Thursday: Hands-on Project . . . . .                                          | 71        |
| 4.5.1    | Goal and Architecture . . . . .                                               | 71        |
| 4.5.2    | Step-by-Step Actions . . . . .                                                | 71        |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 4.5.3    | Validation Checklist . . . . .                                   | 73        |
| 4.6      | Friday: Use Case and Architecture . . . . .                      | 73        |
| 4.6.1    | Symptoms . . . . .                                               | 73        |
| 4.6.2    | Remediation Strategy . . . . .                                   | 73        |
| 4.6.3    | Operational Runbook . . . . .                                    | 74        |
| 4.7      | Operational Guidance for Week 4 . . . . .                        | 74        |
| 4.7.1    | Performance and Reliability . . . . .                            | 74        |
| 4.7.2    | Security and Governance Checklist . . . . .                      | 75        |
| 4.8      | Interview Q&A . . . . .                                          | 75        |
| 4.9      | Further Reading . . . . .                                        | 75        |
| 4.10     | Exercises . . . . .                                              | 76        |
| 4.11     | Milestone 1 Capstone: Spark-Driven Medallion Lakehouse . . . . . | 76        |
| <b>A</b> | <b>Cross-Cloud Service Equivalence</b>                           | <b>81</b> |
| A.1      | Object and Data-Lake Storage . . . . .                           | 81        |
| A.2      | Managed Relational Databases . . . . .                           | 82        |
| A.3      | Elastic Compute and Analytical Warehouses . . . . .              | 82        |
| A.4      | Data Organization and the Medallion Pattern . . . . .            | 82        |
| A.5      | Document Data Modeling . . . . .                                 | 83        |
| A.6      | Globally Distributed and NoSQL Databases . . . . .               | 83        |
| A.7      | Managed Apache Spark . . . . .                                   | 84        |
| A.8      | Query Acceleration and Materialized Views . . . . .              | 84        |
| A.9      | In-Memory Caching and Hybrid Ingestion . . . . .                 | 84        |
| A.10     | Wide-Column and Key-Value NoSQL . . . . .                        | 85        |
| A.11     | Data Protection: Cloning, Time Travel, and Migration . . . . .   | 85        |
|          | <b>Glossary</b>                                                  | <b>86</b> |

# List of Figures

|     |                                                                                            |    |
|-----|--------------------------------------------------------------------------------------------|----|
| 1   | Chapter 0 setup flow from organizational identity to a verified Fabric lakehouse.          | 3  |
| 1.1 | A OneLake-centered medallion pattern for Microsoft Fabric.                                 | 19 |
| 1.2 | Multi-cloud retail virtualization through OneLake shortcuts and medallion curation.        | 24 |
| 1.3 | Capstone architecture for a multi-cloud retail landing zone using OneLake shortcuts.       | 29 |
| 2.1 | Bronze-to-Silver-to-Gold flow for a Fabric medallion architecture.                         | 36 |
| 2.2 | Multi-workspace isolation pattern for financial medallion architecture.                    | 42 |
| 2.3 | Reference architecture for the financial medallion capstone.                               | 46 |
| 3.1 | Spark driver and executors on a Fabric pool reading OneLake and writing Delta.             | 53 |
| 3.2 | Spark notebook ingestion pipeline for 100 GB per day of telemetry JSON.                    | 58 |
| 3.3 | Reference architecture for the Spark telemetry ingestion capstone.                         | 62 |
| 4.1 | Delta table internals: Parquet data files plus a transaction log.                          | 69 |
| 4.2 | Small-file remediation through compaction and optimized layout.                            | 74 |
| 4.3 | Milestone 1 reference architecture from Bronze shortcuts to V-Order optimized Gold tables. | 77 |



# List of Tables

|     |                                                                                   |    |
|-----|-----------------------------------------------------------------------------------|----|
| 1   | Common Fabric workspace roles for a learning environment. . . . .                 | 6  |
| 2   | Common Chapter 0 setup symptoms and first checks. . . . .                         | 11 |
| 1.1 | Major Fabric workloads and their relationship to the shared analytics platform. . | 17 |
| 1.2 | Shortcut types and architecture questions. . . . .                                | 20 |
| 1.3 | Shortcut versus materialization decision matrix. . . . .                          | 21 |
| 1.4 | Retail architecture decisions for shortcut-first modernization. . . . .           | 25 |
| 2.1 | Layer contracts in a medallion design. . . . .                                    | 36 |
| 2.2 | Lakehouse-versus-Warehouse decision aid for Fabric medallion design. . . . .      | 37 |
| 2.3 | Workspace and lakehouse organization patterns for medallion scale. . . . .        | 39 |
| 2.4 | Financial institution decisions for strict medallion isolation. . . . .           | 42 |
| 3.1 | Compute choices for Spark development in Fabric. . . . .                          | 53 |
| 3.2 | Choosing PySpark or Spark SQL inside Fabric notebooks. . . . .                    | 55 |
| 3.3 | Design decisions for a 100 GB/day telemetry ingestion layer. . . . .              | 58 |
| 4.1 | Optimization choices for Delta tables in Fabric. . . . .                          | 70 |
| 4.2 | Diagnosing and remediating a degraded streaming Delta table. . . . .              | 74 |

# Getting Started: Your Microsoft Fabric Trial and Workspace

## Learning Objectives

- Explain why Microsoft Fabric requires an organizational Microsoft 365 or Power BI identity rather than a personal consumer account.
- Start a free Fabric trial capacity and distinguish the trial from paid Fabric F SKUs and capacity units.
- Verify the tenant-level Fabric switch that allows users to create Fabric items, and identify who can enable it.
- Create a workspace, assign it to the Trial capacity, and apply the appropriate workspace role for each collaborator.
- Create a first lakehouse in OneLake and understand how lakehouse **Files** and **Tables** become the foundation for later chapters.
- Use the browser-based Fabric experience, optional Windows tooling, and Fabric notebooks to read and write a Delta table.
- Establish cost, quota, and validation guardrails before beginning Chapter 1.

## Cross-Cloud Equivalence

The first practical difference between Microsoft Fabric and raw infrastructure clouds appears before you create any data: Fabric starts from an organization tenant and a SaaS analytics capacity, not from a general-purpose cloud subscription.

| Setup step      | AWS                                 | Azure                                      | GCP                                      |
|-----------------|-------------------------------------|--------------------------------------------|------------------------------------------|
| Free offer      | Free Tier (12 mo + always free)     | Free account (\$200/30 days + always free) | Free trial (\$300/90 days + Always Free) |
| Sign-up guard   | Card + root email                   | Card + Microsoft account                   | Card + Google account                    |
| Identity model  | IAM / IAM Identity Center           | Microsoft Entra ID + RBAC                  | Cloud IAM                                |
| Admin isolation | Never use root daily                | Avoid Global Admin daily                   | Avoid Owner; least privilege             |
| CLI             | AWS CLI v2 ( <b>aws configure</b> ) | Azure CLI ( <b>az login</b> )              | gcloud CLI ( <b>gcloud init</b> )        |
| Browser shell   | AWS CloudShell                      | Azure Cloud Shell                          | Cloud Shell                              |
| Cost guardrail  | AWS Budgets + alerts                | Cost Management budgets                    | Budgets & alerts                         |
| Console         | AWS Management Console              | Azure Portal                               | Google Cloud Console                     |

THIS book’s platform, Microsoft Fabric, uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card, org tenant) rather than an IaaS subscription; Snowflake uses a 30-day/\$400 trial (no card) guarded by Resource Monitors; MongoDB Atlas offers a forever-free M0 cluster (no card) secured by database users and an IP allowlist.

Key Terms

**Organizational account:** a Microsoft Entra ID-backed work or school account, normally written as `name@organization.com`. **Tenant:** the Microsoft 365 and Entra ID boundary that owns users, groups, admin settings, and Fabric’s OneLake namespace. **Fabric trial capacity:** a temporary Fabric capacity that lets an eligible user evaluate Fabric workloads. **Capacity unit:** a normalized measure of Fabric compute used across Spark, SQL, pipelines, real-time workloads, and Power BI. **Workspace:** the collaboration, permission, and capacity-assignment container for Fabric items. **Lakehouse:** a Fabric data engineering item that exposes OneLake files, managed Delta tables, Spark, and SQL-oriented access.

0.1 Why Chapter 0 Exists

Chapter 1 begins with OneLake architecture. Before that architecture is useful, you need a working Fabric tenant, an enabled Fabric experience, an active capacity, a workspace assigned to that capacity, and a lakehouse where notebooks can run. This chapter builds that foundation deliberately in a Windows and PowerShell learning environment, while keeping the important distinction clear: Microsoft Fabric is a SaaS analytics platform, not an IaaS cloud account where you provision virtual networks, storage accounts, virtual machines, and Spark clusters by hand.

Fabric unifies data engineering, data integration, data warehousing, real-time analytics, data science, and Power BI in one browser-based experience [21]. The platform hides many infrastructure chores, but it does not remove setup decisions. Identity, tenant settings, capacity assignment, workspace roles, cost controls, and verification still matter. A learner who skips these details often reaches Chapter 1 with a confusing symptom: the **New** button is missing, a lakehouse cannot be created, a notebook will not attach, or a workspace is quietly using the wrong capacity.

**Note**

This chapter assumes you are using Windows with PowerShell for local commands and a browser for Fabric. Fabric itself runs in the Microsoft cloud; the local machine is mostly a client for documentation, sample files, optional desktop tools, and book builds.

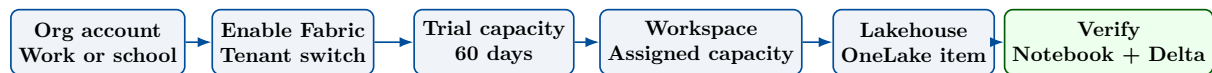


Figure 1: Chapter 0 setup flow from organizational identity to a verified Fabric lakehouse.

## 0.2 Prerequisites: Identity Before Infrastructure

The first prerequisite is not a credit card. It is an organizational identity. Fabric is attached to a Microsoft 365 and Power BI tenant backed by Microsoft Entra ID. The account must be a work or school account, such as `student@contoso.onmicrosoft.com` or `analyst@yourcompany.com`. That identity places the user inside a tenant where administrators can control Fabric settings, create or allow capacities, manage licenses, configure governance, and own OneLake.

A personal Microsoft account, such as a consumer Outlook.com account, is not enough to enable Fabric by itself. A Gmail address used as a Microsoft account has the same limitation. Those accounts can sign in to consumer Microsoft services, but they do not create the organizational tenant boundary that Fabric needs for tenant settings, workspace governance, Power BI administration, and OneLake’s tenant-wide namespace. In practical terms, a personal account has no Fabric tenant administrator who can turn on Fabric item creation, no organizational users or groups to assign workspace roles to, and no tenant-level OneLake home for the analytics estate.

**Definition**

An **organizational account** is an account managed by Microsoft Entra ID in a work, school, developer, or Power BI tenant. Fabric depends on that tenant because the platform’s security, administration, workspaces, capacities, and OneLake namespace are tenant-scoped.

### 0.2.1 Free Ways to Obtain an Organizational Account

For training, the most common path is a Microsoft 365 Developer Program tenant. If you qualify, this gives you a developer Microsoft 365 tenant with administrator control for learning and app development. The exact availability and renewal rules can change, so treat it as a learning tenant rather than a permanent production environment. After it exists, you sign in with an account such as `admin@yourdevtenant.onmicrosoft.com`, open Fabric, and evaluate whether the Fabric trial option is available.

A second path is a Power BI or Fabric free sign-up using a work email address. This path is appropriate when your organization allows self-service sign-up and the email domain can be associated with an organizational tenant. It is less suitable when a company blocks self-service trials or centrally manages Power BI. In that case, ask the Microsoft 365, Power BI, or Fabric administrator to enable the trial or provide a training workspace.

**Common Pitfall**

**Trying to start Fabric from a personal account.** If you sign in with a consumer Outlook.com, Hotmail, Live.com, or Gmail-backed Microsoft account, you may reach Microsoft pages but still be unable to create Fabric items. Switch to an organizational work or school account before troubleshooting capacities or workspaces.

**0.2.2 Minimum Access You Need**

For the hands-on work in this book, you need enough permission to do five things:

1. Sign in to <https://app.fabric.microsoft.com> with an organizational account.
2. Use or start a Fabric trial capacity, or have a workspace assigned to an existing Fabric capacity.
3. Create a workspace, or be added to a workspace as Admin, Member, or Contributor.
4. Create Fabric items in that workspace, including a lakehouse and notebook.
5. Upload or create a small sample file in the lakehouse and run a notebook attached to it.

If you are in a managed enterprise tenant, you may not personally control all five. That is normal. The setup is still valid if an administrator creates the capacity and workspace for you, then adds you as a workspace Member or Contributor.

**0.3 Starting the Free Fabric Trial**

Open a browser and go to <https://app.fabric.microsoft.com>. Sign in with your organizational account. If your tenant allows trials and your account is eligible, the Fabric interface provides an option to start a trial. Microsoft documents Fabric trial behavior separately from paid capacity licensing [16]. The trial is commonly described as a 60-day evaluation capacity that lets you create and run Fabric items without first purchasing a paid F SKU.

The trial does not mean Fabric has become a personal local tool. It creates or assigns a trial capacity in the tenant. Workspaces must use a Fabric capacity to run Fabric workloads such as lakehouses, notebooks, Data Factory pipelines, warehouses, and other Fabric items. The trial capacity is the temporary compute pool you will attach to your learning workspace.

**0.3.1 What the Trial Includes**

The trial capacity is meant for evaluation and hands-on learning. It enables Fabric experiences that otherwise require capacity. In this book, that means you can create a workspace, assign it to the Trial capacity, create a lakehouse, run notebooks, write Delta tables, and inspect OneLake storage behavior. The trial is shared through the tenant model: administrators and capacity settings can determine who can use it and which workspaces are assigned.

A trial is not a production capacity. It expires, can be disabled by tenant policy, and may have limits that are inappropriate for real workloads. Use it to learn concepts, validate book exercises, and understand how jobs consume capacity. Do not treat it as a durable home for business data.

**Definition**

A **Fabric capacity** is a pool of compute resources used by Fabric workloads. Paid capacities are sold as F SKUs with different amounts of compute. Capacity units, or CUs, are the

normalized way to reason about consumption across different Fabric engines [14].

### 0.3.2 Trial Capacity Versus F SKUs

Paid Fabric capacities are normally referred to by F SKU names, such as smaller and larger F capacities. A larger SKU provides more capacity units and can sustain more concurrent or heavier work. A trial capacity gives learners a temporary capacity so they can experience Fabric without committing to a paid SKU first. The engineering model is the same: your workspace is assigned to a capacity, workloads consume capacity, and overloaded capacity can affect responsiveness.

The practical takeaway is simple. You do not buy, size, or pause the trial the way you would a paid Azure Fabric capacity, but you should still think like an engineer. A poorly designed notebook, runaway loop, accidental full scan, or repeated refresh can consume capacity and make the trial feel slow. The habits you learn on the trial carry forward to paid F SKUs.

## 0.4 Enabling Fabric at the Tenant

Starting a trial and creating items depends on tenant settings. In the Fabric or Power BI Admin portal, administrators can control whether users can create Fabric items. The setting is often described as **Users can create Fabric items** under Fabric tenant settings [15]. If the switch is off for your whole organization or for your security group, the user interface may hide Fabric creation commands or show errors when you try to create a lakehouse, notebook, warehouse, or other Fabric item.

Who can enable it? In a managed tenant, a Power BI administrator, Fabric administrator, or Microsoft 365 Global Administrator usually controls the relevant Admin portal setting. A Global Administrator should not be used for daily analytics work, but that role may be needed to assign the first Fabric or Power BI admin roles. After the tenant setting is enabled, day-to-day management can be delegated through Fabric administrators, capacity administrators, workspace admins, and security groups.

### Common Pitfall

**The tenant switch is off.** If you can sign in but cannot create a lakehouse, do not immediately blame your browser, license, or notebook. Check whether the Admin portal setting **Users can create Fabric items** is enabled for your user or group. In many enterprise tenants, this is the first blocked setup step.

#### 0.4.1 Delegating to Capacity Administrators

Tenant administrators should not personally manage every workspace. A cleaner model is to enable Fabric for approved security groups, create or allow capacities, then delegate capacity administration to the platform or data engineering team. Capacity admins can manage capacity assignment and monitor usage without needing full tenant administrator privileges. Workspace admins then decide who can collaborate inside each workspace.

This separation matters for least privilege. A training instructor may be a capacity admin for a trial or paid learning capacity. Students may be workspace Contributors. A platform owner may be a Fabric admin. A business analyst may be a Viewer on a curated workspace. These roles are intentionally different.

## 0.5 Creating Your Workspace

After the tenant setting and capacity are available, create the workspace that will hold the book exercises. In the Fabric portal, select **Workspaces**, choose **New workspace**, and use a name that makes the environment clear. For this book, a useful personal training name is `fabric-book-dev`. In a classroom, include a short cohort or lab identifier, such as `fabric-de-july-lab01`. Avoid names that imply production unless the workspace is truly governed as production.

During or after workspace creation, assign the workspace to the Trial capacity. In the workspace settings, find the license or capacity section and choose the Trial capacity. If you do not see the trial, it may not have started, it may belong to another user, your tenant may restrict capacity assignment, or you may lack workspace admin permission. Microsoft describes workspaces as collaboration containers for Fabric items and permissions [23].

### 0.5.1 Workspace Roles

Workspace roles determine what collaborators can do. Use the smallest role that supports the learning task.

| Role        | Typical permissions                                              | Training use                                           |
|-------------|------------------------------------------------------------------|--------------------------------------------------------|
| Admin       | Manage workspace settings, access, and items                     | Instructor, lab owner, or learner in a personal tenant |
| Member      | Create and manage content, share many items, collaborate broadly | Senior learner or team lead in a shared workspace      |
| Contributor | Create and edit content but with less administrative control     | Default role for hands-on builders in a controlled lab |
| Viewer      | Read and interact with published content without editing it      | Reviewers, stakeholders, and learners observing demos  |

Table 1: Common Fabric workspace roles for a learning environment.

#### Tip

For book exercises, use one dedicated development workspace. Do not mix training lakehouses with enterprise reports, certified semantic models, or production pipelines. Separation makes cleanup and troubleshooting much safer.

### 0.5.2 Creating Your First Lakehouse

Inside the workspace, select **New item** and create a **Lakehouse**. Name it `lh_getting_started`. Fabric creates a lakehouse item with a file area and a table area. The file area is useful for raw uploads and folders. The table area contains managed Delta tables that Fabric can expose to Spark and SQL experiences.

This lakehouse lives in OneLake. OneLake is the tenant-wide logical store for Fabric [22]. You are not choosing a storage account, container, region-specific endpoint, or access key in the way you would with raw object storage. Fabric projects the lakehouse into OneLake under the tenant and workspace context. This is why the organizational account and tenant setup mattered at the beginning of the chapter.

**Note**

OneLake is tenant-wide, but access is not automatically tenant-wide. Users still need permission through workspaces, items, sharing, data access roles, semantic models, or other Fabric governance controls. The namespace is unified; authorization remains explicit.

## 0.6 Tooling: Browser First, Optional Desktop Tools

The primary tool for this chapter is the Fabric web experience. You do not need to install Spark, Hadoop, Python, Java, the Azure CLI, or a local database to create the first lakehouse. The browser provides the workspace, lakehouse explorer, notebook editor, Spark session attachment, and item management surface. This is one of the most important differences between Fabric and a self-managed data platform.

Optional tools can improve the experience on Windows:

- **Power BI Desktop** is useful later when you build semantic models and reports locally before publishing. It is not required for creating a lakehouse.
- **OneLake File Explorer for Windows** can show OneLake in File Explorer so authorized users can browse and copy files through a familiar interface. Use it carefully; it is convenient, but it does not replace data engineering discipline.
- **Fabric notebooks** run inside Fabric and support PySpark for reading files, transforming data, and writing Delta tables. This book uses notebooks for many data engineering labs.
- **Fabric REST APIs and command-line automation** are useful for repeatable administration and deployment where your organization supports them. For Chapter 0, the browser is enough; automation becomes more relevant after you understand the manual path.

### 0.6.1 A First PySpark Read and Delta Write

After creating `lh_getting_started`, create a new notebook in the same workspace and attach the lakehouse. The following cell creates a tiny in-memory DataFrame, writes it as a managed Delta table in the lakehouse, then reads it back. It intentionally avoids external files so that Spark itself is the only moving part.

```

1 from pyspark.sql import functions as F
2
3 rows = [
4     (1, "bronze", "raw file landing"),
5     (2, "silver", "clean conformed tables"),
6     (3, "gold", "business-ready metrics"),
7 ]
8
9 layers_df = spark.createDataFrame(rows, ["layer_id", "layer_name", "purpose"])
10
11 (
12     layers_df
13     .withColumn("loaded_at_utc", F.current_timestamp())
14     .write
15     .format("delta")
16     .mode("overwrite")
17     .saveAsTable("setup_layers")
18 )
19

```



```
20 spark.read.table("setup_layers").orderBy("layer_id").show(truncate=False)
```

Listing 1: PySpark one-cell smoke test for a Fabric lakehouse.

The expected result is a three-row table containing **bronze**, **silver**, and **gold**, plus a timestamp column. If this works, the notebook can start Spark, attach to the lakehouse, write Delta, register a table, and read the table back.

## 0.6.2 REST and JSON Awareness

You do not need REST automation for the first lab, but it is useful to recognize that Fabric items and administrative operations can be represented as structured requests. The exact endpoint and API version may change, so the next listing is a conceptual JSON body for documenting a workspace setup request rather than a command to paste blindly.

```
1 {
2   "workspaceName": "fabric-book-dev",
3   "capacity": {
4     "assignment": "Trial",
5     "purpose": "Microsoft Fabric book exercises"
6   },
7   "items": [
8     {
9       "type": "Lakehouse",
10      "displayName": "lh_getting_started"
11    },
12    {
13      "type": "Notebook",
14      "displayName": "nb_setup_smoke_test",
15      "defaultLakehouse": "lh_getting_started"
16    }
17  ],
18  "roles": [
19    {
20      "principal": "learner group",
21      "role": "Contributor"
22    }
23  ]
24 }
```

Listing 2: Conceptual REST/JSON setup record for a Fabric training workspace.

## 0.7 Cost and Quota Guardrails

Fabric's trial removes the initial purchase step, but it does not remove finite capacity. A capacity can become busy, throttled, or temporarily burst depending on workload shape and product behavior. Interactive report usage, Spark notebooks, warehouse queries, Data Factory activities, semantic model refreshes, and real-time workloads all consume from the same capacity model. The lesson for a data engineer is to measure consumption early, even when the direct bill is zero.

### 0.7.1 Trial Expiry

A trial is temporary. Before relying on a workspace for more than a lab, record when the trial started, who owns it, and what should happen when it expires. If the trial ends and the workspace is not reassigned to a paid capacity, Fabric item creation and execution may stop working as expected. Export notebooks or keep source-controlled copies of important code before the trial ends.

### 0.7.2 Bursting and Throttling

Fabric capacities can smooth bursts, but sustained overuse eventually produces delays or throttling. A single learner can trigger this with repeated full-table writes, an accidental Cartesian join, or too many notebook reruns. In shared classes, twenty learners running the same Spark cell at once can expose capacity limits quickly. That is not a failure of Fabric; it is an operating reality of shared compute.

### 0.7.3 Paid Capacity Later

If you later move to paid Fabric capacity, you can scale, pause, resume, or delete capacity depending on the purchasing model and administrative controls. Pausing a paid capacity is a cost guardrail, but it also stops workloads assigned to it. Plan environment separation so development capacities can be paused safely while production capacities remain governed and monitored.

#### Tip

**Measure it.** Install or ask for access to the Microsoft Fabric Capacity Metrics app when you move beyond a solo trial. Use it to identify which workspace, item, user, or workload is consuming capacity before increasing the SKU or blaming the platform [13].

## 0.8 Verify the Setup

A setup is complete only when it produces observable evidence. The verification path below proves four things: the workspace exists, the lakehouse exists, OneLake accepts a sample file, and Spark can read and write data.

### 0.8.1 Step 1: Create the Workspace and Lakehouse

Confirm that the workspace appears in the Fabric portal and is assigned to the Trial capacity. Confirm that `lh_getting_started` appears as a lakehouse item in that workspace. Open the lakehouse and verify that the explorer shows **Files** and **Tables**.

### 0.8.2 Step 2: Upload a Sample File to OneLake

Create a small CSV file on Windows using PowerShell, then upload it through the lakehouse explorer to **Files/setup**. The command below creates the local sample file in the current directory. It is local preparation only; the upload happens in the browser unless you choose an approved automation method later.

```
1 @'
2 order_id,customer_id,order_date,sales_amount
3 1001,C001,2026-07-01,42.50
4 1002,C002,2026-07-02,99.00
5 1003,C001,2026-07-03,18.25
6 '@ | Set-Content -Path .\fabric_setup_orders.csv -Encoding utf8
```

Listing 3: PowerShell command to create a small CSV sample file on Windows.

In the Fabric lakehouse, choose **Files**, create a folder named `setup`, and upload `fabric_setup_orders.csv`. The path should appear as `Files/setup/fabric_setup_orders.csv`. If you use OneLake File Explorer instead, copy the file only into the intended lakehouse folder and wait for the Fabric interface to refresh.

### 0.8.3 Step 3: Run a One-Cell Notebook

Create or open `nb_setup_smoke_test`, attach `lh_getting_started`, and run the following cell.

```

1 from pyspark.sql import functions as F
2
3 source_path = "Files/setup/fabric_setup_orders.csv"
4
5 orders_df = (
6     spark.read
7     .option("header", True)
8     .option("inferSchema", True)
9     .csv(source_path)
10 )
11
12 verified_df = (
13     orders_df
14     .withColumn("order_date", F.to_date("order_date"))
15     .withColumn("sales_amount", F.col("sales_amount").cast("decimal(18,2)"))
16 )
17
18 verified_df.write.format("delta").mode("overwrite").saveAsTable("setup_orders")
19
20 result_df = spark.sql("""
21 SELECT
22     COUNT(*) AS row_count,
23     COUNT(DISTINCT customer_id) AS customer_count,
24     CAST(SUM(sales_amount) AS DECIMAL(18,2)) AS total_sales_amount
25 FROM setup_orders
26 """)
27
28 result_df.show(truncate=False)

```

Listing 4: PySpark verification cell that reads an uploaded CSV and writes a Delta table.

Expected result:

```

1 +-----+-----+-----+
2 |row_count|customer_count|total_sales_amount|
3 +-----+-----+-----+
4 |3        |2            |159.75            |
5 +-----+-----+-----+

```

Listing 5: Expected verification output.

If the output matches, Chapter 0 is successful. Spark read a OneLake file, converted types, wrote a managed Delta table, registered it in the lakehouse, and queried it back.

### 0.8.4 Troubleshooting Checklist

| Symptom                   | Likely cause                                                       | Action                                                                                     |
|---------------------------|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| Cannot create Fabric item | Tenant setting is disabled or restricted                           | Ask Fabric or Power BI admin to enable <b>Users can create Fabric items</b> for your group |
| No Trial capacity option  | Trial not started, blocked, expired, or unavailable to user        | Start trial if eligible or ask admin for capacity assignment                               |
| Notebook will not attach  | Lakehouse missing, capacity unavailable, or workspace not assigned | Recheck workspace capacity and default lakehouse                                           |
| CSV path not found        | File uploaded to wrong folder or name differs                      | Confirm exact path under <b>Files/setup</b>                                                |
| Delta write fails         | Spark session not ready or table permission issue                  | Restart session and confirm workspace role is Contributor or higher                        |
| Slow or queued execution  | Capacity is busy or throttled                                      | Wait, stop unnecessary sessions, and inspect capacity metrics if available                 |

Table 2: Common Chapter 0 setup symptoms and first checks.

## 0.9 How This Setup Differs from Raw IaaS Clouds

In AWS, Azure, or GCP, a first data engineering lab often begins by creating a cloud subscription or project, selecting a region, enabling APIs, provisioning object storage, creating IAM roles, configuring command-line credentials, and deciding which compute service will run Spark. Fabric intentionally compresses many of those choices into a SaaS experience. You still make architecture decisions, but the first units are tenant, capacity, workspace, item, and OneLake path.

That difference is useful pedagogically. You can begin with data architecture before building a platform from primitives. However, it can also mislead learners into thinking there is no infrastructure. There is infrastructure, but Microsoft operates much of it behind the SaaS boundary. Your engineering responsibilities move upward: capacity usage, workspace design, item ownership, data contracts, governance, quality, and consumption paths.

#### Note

When this book compares Fabric with Azure services, remember that Fabric is not simply “Synapse plus Power BI.” It is a SaaS analytics platform with shared OneLake storage, capacities, workspaces, governance, and integrated workloads. Some Azure concepts reappear, but the operating model is different.

## 0.10 Further Reading

Start with Microsoft’s overview of Fabric [21], the Fabric trial documentation [16], Fabric concepts and licenses [14], tenant settings for Fabric item creation [15], workspaces [23], OneLake [22], lakehouses [20], and the Capacity Metrics app [13]. Return to these pages whenever a user-interface label changes; the concepts in this chapter are stable, but the portal wording and placement can evolve.

## Key Takeaways

- Microsoft Fabric requires an organizational work or school account because tenant settings, administration, workspaces, capacities, and OneLake are tenant-scoped.
- A personal Outlook.com, Hotmail, Live.com, or Gmail-backed Microsoft account cannot by itself enable a Fabric tenant for hands-on data engineering.
- The Fabric trial provides a temporary capacity for evaluation; it is not a production capacity and should be monitored like shared compute.
- The Admin portal setting **Users can create Fabric items** must be enabled for the relevant users or groups before lakehouses and notebooks can be created.
- Workspaces are collaboration and permission containers, and they must be assigned to the Trial or another Fabric capacity for Fabric workloads.
- A first lakehouse proves that OneLake is available and gives later chapters a place to store files, managed Delta tables, and notebook outputs.
- Verification is complete only after you upload a sample file, run a PySpark cell, write a Delta table, and confirm the expected result.

## 0.11 Exercises

1. **(Identity)** Write down the account you used for Fabric. Is it an organizational work or school account? Explain how you know without including private information.
2. **(Admin)** Find the tenant setting that controls whether users can create Fabric items. If you cannot access the Admin portal, document who in your organization owns that setting.
3. **(Capacity)** Start or identify the Fabric trial capacity. Record the trial start date, expected expiry date, and the workspace assigned to it.
4. **(Workspace)** Create `fabric-book-dev` or an equivalent training workspace. Assign it to the Trial capacity and add any collaborators with the least role they need.
5. **(Lakehouse)** Create `lh_getting_started`. Identify where Files and Tables appear in the lakehouse explorer.
6. **(Upload)** Create `fabric_setup_orders.csv` with PowerShell and upload it to Files/setup. Confirm the path in the Fabric portal.
7. **(Notebook)** Run the verification PySpark cell and confirm the expected result: three rows, two customers, and total sales of 159.75.
8. **(Cost)** If you have access, open the Capacity Metrics app or ask your administrator how capacity usage is monitored. Note where you would look if notebooks became slow.
9. **(Setup checklist before Chapter 1)** Confirm all of the following: organizational account works; Fabric item creation is enabled; trial or paid capacity is available; workspace is assigned to capacity; lakehouse exists; sample file is uploaded; notebook smoke test passes; cleanup owner is known.
10. **(Interview)** In five sentences, explain why Fabric setup begins with tenant identity and capacity assignment rather than with a storage account and virtual machine.

## Part I

# Core Data Infrastructure and OneLake

# OneLake and Microsoft Fabric Foundations

## Learning Objectives

- Explain Microsoft Fabric as a unified SaaS analytics platform and distinguish its major workloads.
- Reason about Fabric capacities, F SKUs, capacity units, workspaces, domains, and tenant-level organization.
- Define OneLake as a single logical data lake per tenant and describe how workspaces, folders, and lakehouses project into it.
- Describe Delta-Parquet as the native open storage pattern for reliable lakehouse tables.
- Create and evaluate OneLake shortcuts for internal and external data virtualization scenarios.
- Build a first lakehouse lab that reads shortcut data with PySpark without copying the underlying files.
- Architect a multi-cloud retail data virtualization pattern that unifies AWS S3 history and Dataverse CRM data through OneLake.

## Cross-Cloud Equivalence

Although this book teaches **Microsoft Fabric**, OneLake is built directly on the Azure column below, and the Week 1 storage foundations map almost one-to-one across the other clouds.

| Capability          | AWS                                     | Azure                                     | GCP                                       |
|---------------------|-----------------------------------------|-------------------------------------------|-------------------------------------------|
| Object storage      | Amazon S3                               | ADLS Gen2 / Blob                          | Cloud Storage                             |
| Hot tier            | S3 Standard                             | Hot                                       | Standard                                  |
| Infrequent access   | S3 Standard-IA                          | Cool / Cold                               | Nearline                                  |
| Archive (instant)   | Glacier Instant Retrieval               | Cold                                      | Coldline                                  |
| Deep archive        | Glacier Deep Archive                    | Archive                                   | Archive                                   |
| Lifecycle rules     | Lifecycle policies                      | Lifecycle management                      | Object Lifecycle Mgmt                     |
| Versioning          | S3 Versioning                           | Blob versioning                           | Object Versioning                         |
| Immutability (WORM) | Object Lock                             | Immutable blob storage                    | Bucket Lock                               |
| Cross-region copy   | CRR / SRR                               | Object replication (GRS)                  | Dual/multi-region + Transfer              |
| Online transfer     | DataSync                                | AzCopy                                    | Storage Transfer Service                  |
| Offline transfer    | Snow Family                             | Data Box                                  | Transfer Appliance                        |
| Design durability   | 11 nines                                | 11 nines (LRS) / 16 (GRS)                 | 11 nines                                  |
| Availability SLA    | 99.9% (Standard)                        | 99.9% (RA-GRS read 99.99%)                | 99.95% (multi-region)                     |
| Encryption at rest  | SSE-S3 / SSE-KMS / SSE-C                | SSE + CMK via Key Vault                   | Google-managed / CMEK / CSEK              |
| Access control      | IAM & bucket policies, ACLs             | Azure RBAC, SAS, POSIX ACLs               | IAM, ACLs, signed URLs                    |
| Pricing levers      | Storage + requests + egress + retrieval | Storage + operations + egress + retrieval | Storage + operations + egress + retrieval |
| Consistency         | Strong read-after-write                 | Strong read-after-write                   | Strong (global)                           |

The same storage layer reappears as a managed abstraction across the stack: **Fabric OneLake** is a tenant-wide lake over ADLS Gen2 (Delta-Parquet), **Snowflake** persists data as immutable micro-partitions on the provider's object store, and **MongoDB's** WiredTiger is a node-local engine rather than object storage—the contrast is itself instructive.

### Key Terms

**Fabric capacity:** a pool of compute resources, measured through capacity units, that powers Fabric workloads. **OneLake:** the tenant-wide logical data lake for Microsoft Fabric. **Lakehouse:** a Fabric item that combines data lake storage, Delta tables, SQL access, and Spark access. **Shortcut:** a OneLake pointer to data stored elsewhere, exposed as if it were local. **Delta-Parquet:** Parquet files plus Delta Lake transaction logs that support ACID table semantics.

## 1.1 Week 1 Roadmap

This chapter is the first week of the Microsoft Fabric Master Data Engineer program. Monday introduces the platform. Tuesday establishes OneLake and the lakehouse storage contract. Wednesday focuses on shortcuts and data virtualization. Thursday is a guided project in which you create a workspace, provision a lakehouse, create a shortcut to public blob data, and read the data from a notebook. Friday closes with a global retailer architecture pattern that uses OneLake as the unifying namespace for multi-cloud analytics.

The week deliberately begins with foundations rather than ingestion tools. In Fabric, storage architecture choices influence every later decision: Spark jobs write Delta tables, warehouses can query lakehouse data through SQL endpoints, Power BI can use Direct Lake over curated



models, domains organize data products, and governance metadata depends on workspace and item boundaries. Microsoft describes Fabric as an end-to-end analytics platform that brings data engineering, data integration, warehousing, real-time intelligence, data science, and business intelligence into a single SaaS experience [21]. OneLake is the storage foundation of that experience [22].

#### Note

This chapter intentionally uses the term *data product*. In Fabric, a useful data product is not only a table. It is a governed bundle of storage, transformations, ownership, quality expectations, lineage, security, and consumption paths.

## 1.2 Monday: Introduction to Microsoft Fabric

Microsoft Fabric is best understood as a unified SaaS analytics operating environment rather than a single data tool. It combines experiences that historically lived in separate products: Data Factory for integration, Synapse-style data engineering and warehousing, Real-Time Intelligence, Data Science, and Power BI. The unification matters because the platform gives these workloads shared identity, shared capacity, shared governance, shared workspace organization, and shared OneLake storage.

#### Definition

**Microsoft Fabric** is a unified SaaS analytics platform that integrates data movement, data engineering, data warehousing, real-time analytics, data science, governance, and Power BI over a common tenant, workspace, capacity, and OneLake foundation.

A traditional Azure analytics estate often contains independent services: storage accounts, Spark pools, SQL pools, Data Factory instances, Key Vaults, Power BI workspaces, monitoring resources, and security assignments. Fabric does not remove the need for architecture. It changes the default unit of architecture. Workspaces and Fabric items become the visible work surface; capacities become the shared compute boundary; OneLake becomes the default data lake; and Power BI semantic delivery is integrated instead of bolted on at the end.

### 1.2.1 The Workload Map

Fabric workloads are specialized experiences over a shared platform. A data engineer should know their purpose and their storage relationship.

| Workload               |              | Primary role                                              | Storage and serving pattern                                                          |
|------------------------|--------------|-----------------------------------------------------------|--------------------------------------------------------------------------------------|
| Data Engineering       |              | Build lakehouses, notebooks, Spark jobs, and Delta tables | Writes files and tables in OneLake, commonly in medallion layers                     |
| Data Factory           |              | Ingest, transform, and orchestrate data pipelines         | Moves or transforms data into lakehouses, warehouses, or external systems            |
| Data Warehouse         |              | Serve relational analytics with SQL                       | Uses warehouse storage and can integrate with lakehouse data through SQL experiences |
| Real-Time Intelligence | Intelligence | Process event streams, logs, and telemetry                | Eventstreams and KQL databases support streaming and operational analytics           |
| Data Science           |              | Explore, train, track, and operationalize models          | Uses notebooks, experiments, models, and lakehouse data                              |
| Power BI               |              | Model and visualize business data                         | Consumes lakehouse and warehouse data through import, DirectQuery, or Direct Lake    |

Table 1.1: Major Fabric workloads and their relationship to the shared analytics platform.

**Note**

Do not treat workloads as independent silos. The same customer table may be landed by Data Factory, cleaned by Spark, served by a warehouse, certified in a semantic model, secured by domain governance, and visualized in Power BI. Fabric’s value is the shared operating model across those steps.

**1.2.2 Capacities, F SKUs, and Capacity Units**

A Fabric capacity is the compute pool assigned to one or more workspaces. Microsoft licenses Fabric through capacity SKUs and concepts that determine how much work the platform can execute [14]. F SKUs represent Fabric capacities with different amounts of available compute. Capacity units, often abbreviated as CUs, are the practical abstraction used to reason about consumption across workloads.

**Definition**

A **capacity unit** is an abstract measure of Fabric compute throughput. It lets the platform meter diverse work—Spark sessions, warehouse queries, Data Factory activities, Power BI interactions, and real-time workloads—against a shared capacity pool.

Capacity planning is not only a procurement exercise. It is an engineering feedback loop. The same capacity can be affected by long Spark notebooks, expensive SQL queries, concurrent refreshes, pipeline bursts, and report traffic. A lakehouse design that creates many small files may slow downstream reads and burn capacity. A semantic model that unnecessarily imports data may duplicate storage and refresh work. A pipeline that copies external data that could have been virtualized through a shortcut may increase both latency and compute cost.

**Tip**

Start capacity design by classifying workloads by shape: interactive reports, scheduled batch jobs, ad hoc notebooks, streaming ingestion, and administrative operations. Then test concurrency, not just single-job performance.

### 1.2.3 Workspaces and Domains

Workspaces are collaboration and item containers. They hold lakehouses, warehouses, notebooks, pipelines, semantic models, reports, dataflows, eventstreams, KQL databases, and related artifacts. A workspace is also the practical boundary for assigning capacity and permissions. Domains provide a higher-level governance organization, allowing related workspaces and data products to be grouped by business area such as Finance, Retail Operations, Supply Chain, or Customer Experience [11].

A master data engineer treats workspace design as information architecture. A single shared workspace for every team becomes ungovernable. A workspace per developer fragments lineage and reuse. A useful middle ground aligns workspaces to data products, environments, or bounded domains. For example, `retail-sales-dev`, `retail-sales-test`, and `retail-sales-prod` can live under a Retail domain, while customer support analytics lives in a Customer domain.

#### Common Pitfall

**Using workspaces as folders.** Workspaces carry permissions, capacity assignment, lineage, deployment, and operational meaning. If you create workspaces only to make a navigation tree look tidy, you will eventually fight inconsistent access and duplicated data products.

### 1.2.4 Monday Architecture Lens

The platform view produces three questions that recur throughout the book. First, what is the compute boundary? In Fabric, the answer is normally a capacity assigned to workspaces. Second, what is the collaboration and security boundary? The answer is normally a workspace, refined by item permissions and downstream semantic model security. Third, what is the data boundary? The answer is OneLake plus the source systems linked through shortcuts or pipelines. Good Fabric architecture keeps those three boundaries aligned enough to operate, but not so tightly coupled that every change requires a tenant redesign.

## 1.3 Tuesday: OneLake Concepts

Microsoft presents OneLake as the “OneDrive for Data” for an organization [22]. The phrase is useful because it emphasizes a single logical namespace across the tenant. Users do not need to choose a storage account every time they create a lakehouse. Fabric items place data into OneLake paths managed by the platform, and authorized tools can access those files through Fabric experiences and supported APIs.

#### Definition

**OneLake** is the single logical data lake for a Fabric tenant. It stores Fabric data items and exposes a unified namespace so workloads can share data without every team provisioning separate storage accounts.

The single-lake idea does not mean every team sees every file. Authorization still matters. Workspaces, item permissions, data access roles, sharing, endorsement, and governance controls determine who can discover and use data. The value of OneLake is that storage becomes a shared substrate rather than a collection of disconnected lakes.

### 1.3.1 Workspaces, Items, Folders, and Paths

A Fabric lakehouse item has a storage layout that separates files and tables. The **Files** area is for arbitrary folders and files, including raw landing data. The **Tables** area is for managed Delta tables that Fabric can expose through Spark, SQL analytics endpoints, and Power BI-oriented experiences. This distinction becomes central when you design medallion layers.

#### Note

A folder of Parquet files in **Files** is not automatically the same thing as a managed lakehouse table. A Delta table has a transaction log and table metadata that enable reliable reads, writes, schema evolution, and query interoperability.

### 1.3.2 Delta-Parquet as the Native Open Format

Fabric lakehouses use open data formats. Parquet provides columnar file storage with efficient compression and predicate pushdown [1]. Delta Lake adds a transaction log over Parquet files to provide ACID table behavior on object storage [3, 6]. The lakehouse architecture pattern unifies warehouse-style analytics and data-science workloads over open storage [4]. Fabric adopts this direction by making Delta tables a first-class lakehouse construct [20].

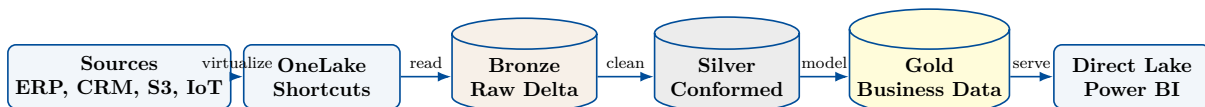
#### Definition

**Delta-Parquet** means table data is stored as Parquet files and table state is managed by a Delta transaction log. The combination supports atomic commits, time-aware metadata, schema enforcement, and consistent reads by multiple engines.

Delta matters because distributed engines do not write one neat database file. Spark jobs write many files, and concurrent jobs can overlap. Without a transaction log, readers may see partial results, duplicate files, or inconsistent schemas. With Delta, a table commit is a versioned metadata operation over a set of Parquet files.

### 1.3.3 Medallion Thinking in OneLake

The medallion architecture organizes data into Bronze, Silver, and Gold layers. Bronze stores raw or lightly standardized data. Silver stores cleaned, conformed, deduplicated, and enriched data. Gold stores business-ready aggregates and dimensional models for consumption [5]. OneLake does not force this layout, but Fabric lakehouses make it natural to model those layers through lakehouses, folders, schemas, or tables.



*Open Delta-Parquet tables in OneLake preserve interoperability across Spark, SQL, and BI workloads.*

Figure 1.1: A OneLake-centered medallion pattern for Microsoft Fabric.

### 1.3.4 Tuesday Engineering Lens

OneLake encourages teams to separate logical data access from physical storage administration. That separation is powerful, but it can hide operational reality. A table still has files. Files still have sizes, partitions, formats, schemas, and update patterns. A shortcut still depends

on a remote system. A report still depends on a semantic model and refresh behavior. The mature Fabric engineer uses OneLake to simplify access while continuing to reason about storage contracts.

## 1.4 Wednesday: OneLake Shortcuts

A shortcut is a pointer inside OneLake to data stored somewhere else. Instead of copying files from an external source into OneLake, Fabric can expose the remote data through the OneLake namespace. Microsoft documents shortcuts for sources such as ADLS Gen2, Amazon S3, Dataverse, and other Fabric locations [17]. For a data engineer, shortcuts are one of Fabric’s most important design primitives because they separate logical access from physical movement.

### Definition

A **OneLake shortcut** is a virtual folder reference that appears inside OneLake while the underlying files remain in their source location. The shortcut lets Fabric workloads read data through the OneLake namespace without first copying it.

### 1.4.1 Internal and External Shortcuts

Internal shortcuts point to data already in OneLake, often in another workspace or item. They support reuse without duplicating files. External shortcuts point to data outside OneLake, such as ADLS Gen2, Amazon S3, or Dataverse. Both patterns can simplify architecture, but they have different governance and operational implications.

| Shortcut type    | Typical use                                               | Engineering questions                                                                              |
|------------------|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Internal OneLake | Share curated data between workspaces without copying     | Who owns the source? What happens if the source table schema changes? How is lineage communicated? |
| ADLS Gen2        | Reuse Azure lake data during migration or coexistence     | Which identity accesses storage? Are firewall, region, and folder conventions compatible?          |
| Amazon S3        | Virtualize historical or partner data in AWS              | What are latency, egress, credentials, and consistency implications?                               |
| Dataverse        | Bring business application data into analytical pipelines | Which tables are needed? How are privacy, row-level access, and refresh expectations handled?      |

Table 1.2: Shortcut types and architecture questions.

### 1.4.2 When Virtualization Beats Copying

Shortcut virtualization is valuable when data is large, shared, governed by another platform, updated by another team, or too expensive to duplicate. It is also useful during migrations: a team can build Fabric transformation logic against external paths, then later decide whether selected data should be physically materialized into OneLake Delta tables.

### Tip

Use shortcuts for access and exploration; use managed Delta tables for reliable curated products. A shortcut to raw external data is not a substitute for Silver and Gold tables with clear ownership, quality checks, and contracts.

### 1.4.3 Risks and Controls

Shortcuts do not remove physics. Remote data can be slower than local data. External cloud access can involve network latency, availability dependencies, credential management, and possible data-transfer costs. External schemas can change without Fabric knowing in advance. The source system remains the system of record, so deletion or restructuring at the source may break downstream Fabric jobs.

#### Common Pitfall

**Treating virtual data as governed data.** A shortcut makes external data visible through OneLake, but it does not automatically certify quality, freeze schema, apply business definitions, or guarantee service-level agreements. Curate important shortcut data into managed Delta tables before exposing it broadly.

### 1.4.4 Shortcut Decision Matrix

Shortcut decisions should be documented explicitly. The following matrix is a practical review aid.

| Question                      | Shortcut-friendly answer                               | Materialize-first answer                                     |
|-------------------------------|--------------------------------------------------------|--------------------------------------------------------------|
| How large is the source?      | Too large or costly to copy before proving value       | Small enough to copy cheaply and repeatedly                  |
| Who owns source quality?      | Another team owns it and publishes a clear contract    | Your team must repair quality before any consumption         |
| How stable is schema?         | Stable or source publishes change notifications        | Volatile and likely to break downstream code                 |
| What is the consumption path? | Exploration, profiling, or controlled engineering jobs | Certified BI, finance reporting, or audited outputs          |
| What is the latency target?   | Batch or exploratory access tolerates remote reads     | Interactive dashboards require predictable local performance |

Table 1.3: Shortcut versus materialization decision matrix.

## 1.5 Thursday: Hands-on Project

The project for Week 1 creates the minimum useful Fabric data engineering environment: a workspace, a lakehouse, a shortcut, and a notebook that reads the shortcut. The scenario assumes you have access to a Fabric tenant, an available capacity or trial capacity, and permissions to create a workspace and lakehouse.

### 1.5.1 Goal and Architecture

You will create a workspace named `fabric-de-week01`, add a lakehouse named `lh_onelake_foundations`, create a shortcut named `public_samples`, and read files exposed by that shortcut with PySpark. The shortcut target can be any public Azure Blob container that your tenant can access. If your instructor provides a specific URL, use that URL. If not, use a public sample container approved for your training environment.

#### Note

Fabric user interface labels change over time. When a button label differs slightly, follow the action intent: create a workspace, create a lakehouse item, open the lakehouse explorer,

create a shortcut under **Files**, select the Azure Blob or ADLS-compatible source, and validate that files appear without uploading them.

### 1.5.2 Step-by-Step Actions

1. Open the Microsoft Fabric portal and confirm that the Fabric experience is enabled for your tenant.
2. Select **Workspaces**, then create a new workspace named `fabric-de-week01`. Assign it to a Fabric capacity if your organization requires explicit capacity selection.
3. Inside the workspace, choose **New item**, then create a **Lakehouse** named `lh_onelake_foundations`.
4. In the lakehouse explorer, expand **Files**. Choose the command to create a new shortcut.
5. Select the external source type for the public sample container. For an Azure Blob public container, choose the source option your tenant exposes for Blob or ADLS-compatible access.
6. Enter shortcut name `public_samples`. Enter the account, container, and folder information supplied by your instructor or the public training dataset.
7. Complete the connection step. If authentication is requested for a public source, choose the anonymous or organizational method that matches the published source and your tenant policy.
8. Confirm that `Files/public_samples` appears in the lakehouse and that its folders or files are visible. You have virtualized data; you have not copied it into the lakehouse.
9. Create a new notebook attached to the lakehouse. Name it `nb_week01_shortcut_read`.
10. Run the PySpark listing below, adjusting the relative path and file format to match the public dataset.

```

1 from pyspark.sql import functions as F
2
3 # In a Fabric notebook attached to lh_onelake_foundations, Files paths are
4 # reachable relative to the lakehouse mount. Adjust the child folder and file
5 # extension to match the public sample container used by your instructor.
6 shortcut_path = "Files/public_samples/sales/*.parquet"
7
8 sales_df = spark.read.parquet(shortcut_path)
9
10 profile_df = (
11     sales_df
12     .select(
13         F.count("*").alias("row_count"),
14         F.countDistinct("CustomerId").alias("customer_count"),
15         F.min("OrderDate").alias("first_order_date"),
16         F.max("OrderDate").alias("last_order_date"),
17         F.sum("SalesAmount").alias("total_sales_amount"),
18     )
19 )
20
21 profile_df.show(truncate=False)
22
23 # Persist a curated Delta table in the lakehouse. The source files remain in
24 # the external container; this table is a managed Fabric lakehouse asset.
25 clean_df = (
26     sales_df
27     .withColumn("OrderDate", F.to_date("OrderDate"))

```



```

28     .withColumn("SalesAmount", F.col("SalesAmount").cast("decimal(18,2)"))
29     .dropDuplicates(["OrderId"])
30 )
31
32 clean_df.write.format("delta").mode("overwrite").saveAsTable("
    silver_sales_orders")

```

Listing 1.1: PySpark notebook snippet for reading data exposed through a OneLake shortcut.

### 1.5.3 Representing the Shortcut as Configuration

In production, connection details should be managed through deployment processes, environment-specific parameters, and secure credentials. The following JSON is not a secret file. It is a conceptual deployment record that captures the intent of a shortcut so it can be reviewed consistently.

```

1 {
2   "workspace": "fabric-de-week01",
3   "lakehouse": "lh_onelake_foundations",
4   "shortcut": {
5     "name": "public_samples",
6     "location": "Files/public_samples",
7     "sourceType": "AzureBlobPublicContainer",
8     "sourceUri": "https://examplepublicdata.blob.core.windows.net/sales",
9     "copyData": false,
10    "intendedLayer": "Bronze exploration",
11    "owner": "data-engineering-training",
12    "qualityContract": "not certified; curate before BI consumption"
13  }
14 }

```

Listing 1.2: Conceptual JSON record for documenting a OneLake shortcut target.

### 1.5.4 Optional SQL Check

After the notebook writes a managed Delta table, use the SQL analytics endpoint or a SQL query cell to check record counts and date ranges. This listing is intentionally simple: Week 1 validates access and table creation, not warehouse performance tuning.

```

1 SELECT
2     COUNT(*) AS row_count,
3     COUNT(DISTINCT CustomerId) AS customer_count,
4     MIN(OrderDate) AS first_order_date,
5     MAX(OrderDate) AS last_order_date,
6     SUM(SalesAmount) AS total_sales_amount
7 FROM silver_sales_orders;

```

Listing 1.3: SQL validation after writing a managed Delta table from shortcut data.

#### Tip

After a successful shortcut read, immediately write down three facts: the owner of the external data, the expected refresh or update pattern, and the downstream table that will become the certified copy. These facts prevent shortcut sprawl.

### 1.5.5 Validation Checklist

A successful lab has observable evidence:

- The workspace exists and is assigned to the expected capacity.



- The lakehouse appears in the workspace and opens without errors.
- The shortcut folder appears under **Files** and exposes external files.
- The notebook reads the shortcut path without uploading files manually.
- A managed Delta table such as `silver_sales_orders` is created from the shortcut data.
- You can explain which data remains external and which data is now managed in the lakehouse.

## 1.6 Friday: Use Case and Architecture

A global retailer has ten years of historical sales data stored in Amazon S3 by region. Current customer and loyalty activity lives in Dataverse because front-office teams use Dynamics-based applications. Executives want a single Power BI view of customer lifetime value, store performance, and campaign effectiveness. The data engineering team wants to avoid a large one-time migration before proving value.

### 1.6.1 Architecture Strategy

The design uses OneLake shortcuts to virtualize both historical S3 data and current Dataverse data into a Fabric lakehouse. Bronze reads remain virtualized at first. Silver tables standardize customer, order, product, calendar, and region fields into managed Delta tables. Gold tables publish conformed metrics for Power BI through a semantic model using Direct Lake where appropriate [10].

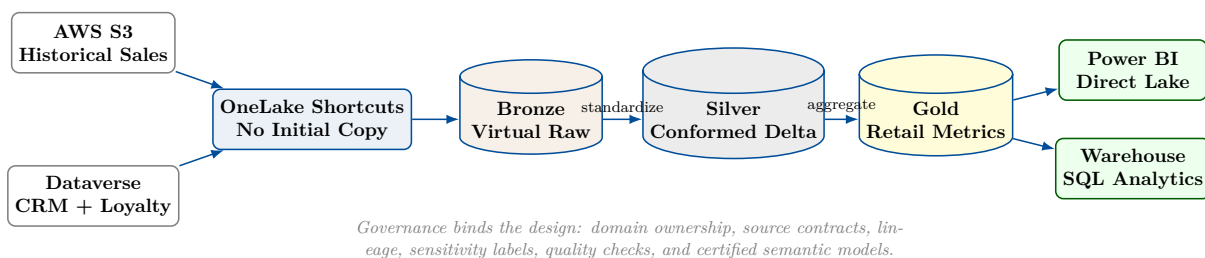


Figure 1.2: Multi-cloud retail virtualization through OneLake shortcuts and medallion curation.

### 1.6.2 Design Decisions

The first decision is to avoid copying all S3 history on day one. A shortcut makes the data visible to Fabric jobs, which lets engineers profile schema, quality, and partitioning before committing to a migration. The second decision is to materialize curated Silver and Gold tables in OneLake. Business users should not query raw external folders directly. The third decision is to represent Dataverse as a governed source with privacy-aware curation: customer identifiers, consent fields, and loyalty attributes require explicit access rules.

| Concern          | Shortcut-first decision                               | Curation decision                                                |
|------------------|-------------------------------------------------------|------------------------------------------------------------------|
| Historical sales | Use S3 shortcut for profiling and incremental value   | Convert validated partitions into Silver Delta tables            |
| Current CRM      | Use Dataverse shortcut for governed analytical access | Mask or restrict sensitive fields before Gold publication        |
| Cost and latency | Avoid bulk migration before value is proven           | Optimize high-value queries into local Delta and semantic models |
| Ownership        | Source teams retain operational systems               | Domain data owners certify curated tables and metrics            |
| BI consumption   | Do not point reports at raw virtual folders           | Publish Gold metrics through Direct Lake semantic models         |

Table 1.4: Retail architecture decisions for shortcut-first modernization.

### Common Pitfall

**Skipping the Silver contract.** Multi-cloud virtualization can make data look unified before it is semantically unified. Customer keys, time zones, currencies, returns, store hierarchies, and consent fields must be standardized before executives trust a global report.

## 1.7 Operational Guidance for Week 1

A Week 1 Fabric environment can already fail in production-like ways. A workspace can be created in the wrong capacity. A shortcut can point to the wrong folder. A notebook can read external files slowly because partitioning is poor. A user can assume a shortcut means the data has been certified. The data engineer's job is to make these assumptions visible.

### 1.7.1 Governance Checklist

- Assign every workspace to a domain or document why it is temporary.
- Name lakehouses, shortcuts, notebooks, and tables consistently.
- Document source ownership and update frequency for every shortcut.
- Separate raw shortcut exploration from certified Silver and Gold tables.
- Record sensitivity and privacy expectations before publishing semantic models.
- Test whether intended users can access the data and whether unintended users cannot.

### 1.7.2 Quality and Observability

Even in the first lab, build the habit of recording quality observations. Count rows, inspect null rates, check duplicate keys, record source file paths, and save the notebook output that proves the shortcut worked. In later chapters these checks become formal data quality gates. Week 1 is where the discipline begins.

```

1 from pyspark.sql import functions as F
2
3 required_columns = ["OrderId", "CustomerId", "OrderDate", "SalesAmount"]
4 missing_columns = sorted(set(required_columns) - set(sales_df.columns))
5 if missing_columns:
6     raise ValueError(f"Missing required columns: {missing_columns}")
7
8 quality_df = sales_df.select(
9     F.count("*").alias("rows"),

```

```

10 F.sum(F.col("OrderId").isNull().cast("int")).alias("null_order_ids"),
11 F.sum(F.col("CustomerId").isNull().cast("int")).alias("null_customer_ids"),
12 F.countDistinct("OrderId").alias("distinct_orders"),
13 )
14
15 quality_df.show(truncate=False)

```

Listing 1.4: Small data quality profile for a newly read shortcut data frame.

### 1.7.3 Interview Q&A

1. **Q: Why is OneLake described as OneDrive for Data?**

A: Because it provides a single logical tenant-wide namespace where Fabric data items can be discovered and accessed through shared platform controls, rather than forcing every team to manage separate storage accounts.

2. **Q: When would you use a shortcut instead of copying data?**

A: Use a shortcut when the source is large, externally owned, frequently reused, or still being evaluated. Copy or materialize when you need curated quality, predictable performance, stable schemas, and certified consumption.

3. **Q: What is the relationship between Parquet and Delta Lake?**

A: Parquet stores columnar files; Delta Lake adds the transaction log and metadata that make those files behave like reliable tables for concurrent engines.

4. **Q: Why do capacities matter to data engineers?**

A: Capacity is the shared compute pool. Inefficient Spark jobs, warehouse queries, refreshes, and report usage compete for it, so engineering choices affect performance and cost.

5. **Q: Why should a BI report not point directly to a raw external shortcut?**

A: Raw shortcut data usually lacks semantic standardization, quality checks, privacy treatment, and performance guarantees. A report should consume certified Gold tables or semantic models.

6. **Q: How do domains change the operating model?**

A: Domains group related workspaces and data products under business ownership. They make governance and discovery align with organizational accountability rather than with accidental workspace lists.

## 1.8 Further Reading

Begin with Microsoft's overview of Fabric [21], OneLake [22], OneLake shortcuts [17], lakehouses [20], capacities and licenses [14], domains [11], and Direct Lake [10]. For storage foundations, read the Delta Lake paper [3], the lakehouse architecture paper [4], Apache Parquet documentation [1], and Delta Lake documentation [6]. For medallion design vocabulary, compare Fabric implementations with the medallion architecture description [5].

## Key Takeaways

- Fabric is a unified SaaS analytics platform, not merely a collection of renamed Azure services.
- Capacities and capacity units turn compute into a shared engineering constraint across Spark, SQL, pipelines, real-time workloads, and Power BI.

- OneLake provides a single logical tenant-wide data lake, while workspaces and items provide practical collaboration and security boundaries.
- Fabric lakehouses use open Delta-Parquet patterns so Spark, SQL, and BI workloads can interoperate over reliable tables.
- Shortcuts virtualize data from internal OneLake locations and external systems such as ADLS Gen2, Amazon S3, and Dataverse.
- Virtualization avoids unnecessary copying, but curated Silver and Gold Delta tables are still required for trusted business consumption.
- A sound Week 1 architecture already includes ownership, naming, quality, privacy, capacity, and consumption decisions.

## 1.9 Exercises

1. **(Conceptual)** Explain the difference between a Fabric workspace, a domain, a capacity, and a lakehouse. Give one governance decision associated with each.
2. **(Conceptual)** Why is a shortcut not the same thing as a certified table? Describe at least three risks that remain after a shortcut is created.
3. **(Hands-on)** Create a development workspace and lakehouse. Add a shortcut under **Files**, then take a screenshot or write a short observation proving that the data was not uploaded manually.
4. **(Hands-on)** Modify the PySpark listing to read CSV or JSON data from your shortcut, infer or provide a schema, and write a managed Delta table named `bronze_source_profile`.
5. **(Design)** Design a workspace naming convention for development, test, and production Fabric environments in a retail organization. Include capacity assignment and domain placement.
6. **(Design)** For the global retailer, decide which data should remain virtualized and which should be materialized into OneLake. Justify your decision using cost, latency, ownership, and BI trust.
7. **(Interview)** In five sentences, defend Delta-Parquet as the native lakehouse format for Fabric data engineering.
8. **(Operational)** Create a Week 1 runbook entry that documents shortcut owner, source URI, authentication method, expected update frequency, downstream tables, and escalation contact.

## 1.10 Capstone Project: A Multi-Cloud Retail Landing Zone with OneLake Shortcuts

### Capstone Project

**Mission.** Design and implement a Week 1 retail landing zone that uses Microsoft Fabric as the SaaS control plane, a trial-backed workspace as the collaboration boundary, OneLake as the analytical namespace, and shortcuts as the virtualization layer. Your goal is to let analysts query historical sales in Amazon S3 and current CRM data from Dataverse without first performing a bulk migration. You will then write one curated Silver Delta table that

becomes the trusted starting point for downstream BI.

**Estimated time.** 3–5 hours, depending on tenant permissions, source availability, and whether your class uses real S3 and Dataverse sources or instructor-provided public Blob and simulated CRM files.

**What you'll practice.**

- Creating a Fabric workspace on trial capacity and assigning work to a Lakehouse item.
- Using OneLake shortcuts to expose external data in place instead of copying raw files.
- Organizing a Bronze zone that separates virtual raw folders from managed curated tables.
- Reading shortcut paths with PySpark and producing a unified Delta-Parquet Silver table.
- Explaining the difference between virtualized source access and certified analytical data products.

## Scenario

A global retailer operates stores and e-commerce channels in North America, Europe, and Asia-Pacific. Ten years of historical order, return, and store sales files are stored by region in AWS S3 because the original point-of-sale modernization program ran on AWS. Current customer profile, loyalty status, and campaign consent data lives in Dataverse because the customer service and marketing teams use Dynamics-based applications.

The executive request is simple: create a single Fabric landing zone that can answer customer lifetime value, regional sales, and loyalty conversion questions without waiting for a large data migration. The engineering constraint is equally important: historical S3 files and operational CRM exports must remain in their source systems during Week 1. Fabric should provide unified access, profiling, and curation while avoiding unnecessary duplication of raw data. This is the Friday architecture scenario made concrete as a hands-on capstone.

## Requirements

Build the smallest production-shaped solution that proves the architecture:

- A Fabric workspace named `fabric-de-week01-retail` assigned to a Fabric trial capacity or another instructor-approved capacity [16].
- A Lakehouse named `lh_retail_landing` in that workspace.
- A Bronze folder structure under `Files/bronze` with shortcut locations for historical sales and CRM data.
- OneLake shortcuts to an Amazon S3 source for historical sales, or to an instructor-approved public Blob source that simulates S3 if S3 credentials are unavailable.
- A OneLake shortcut to Dataverse CRM data, or to an instructor-approved simulated CRM folder if Dataverse permissions are unavailable.
- A PySpark notebook named `nb_capstone_retail_silver` attached to the Lakehouse.
- Notebook logic that reads both shortcut sources in place, standardizes schemas, joins or unions data as appropriate, and writes a managed Silver Delta table named `silver_retail_customer_sales`.
- No bulk copy of raw S3, Blob, or Dataverse data into OneLake. Only the curated Silver Delta output is materialized in the Lakehouse.

**Note**

If your tenant does not allow S3 or Dataverse shortcuts, use public Blob or instructor-provided files as stand-ins. Keep the design language honest: the substitute source represents the external system, while the shortcut still proves the no-copy access pattern described in the OneLake shortcut documentation [17].

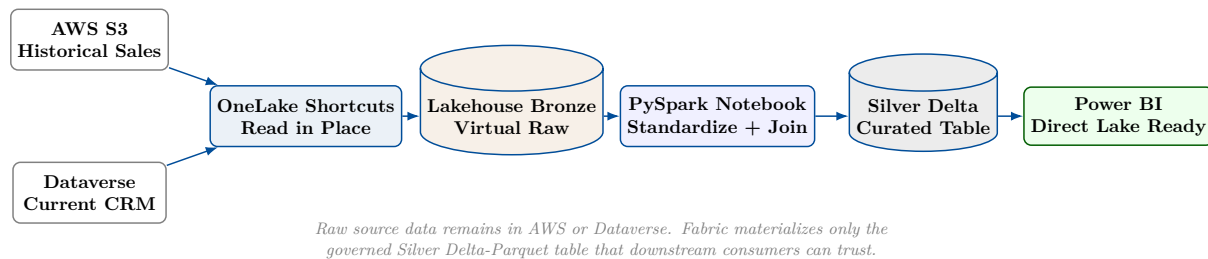
**Reference Architecture**

Figure 1.3: Capstone architecture for a multi-cloud retail landing zone using OneLake shortcuts.

**Implementation Steps**

1. **Create the workspace.** In Fabric, create `fabric-de-week01-retail`. Assign trial capacity or the instructor-approved capacity. Record the capacity name and whether the workspace is intended for development only.
2. **Create the Lakehouse.** Add `lh_retail_landing`. Confirm that the Lakehouse exposes **Files** and **Tables**; the Silver output will become a managed Delta table, not a loose folder of files [20].
3. **Create the Bronze folders.** Under **Files**, use `bronze/s3_sales` for historical sales and `bronze/dataverse_crm` for current CRM. These names describe source origin and medallion intent.
4. **Create the sales shortcut.** Add a OneLake shortcut at `Files/bronze/s3_sales`. Use Amazon S3 when credentials are available. Otherwise, point the shortcut to a public Blob folder that simulates historical sales. Do not upload the sales files manually.
5. **Create the CRM shortcut.** Add a shortcut at `Files/bronze/dataverse_crm`. Use Dataverse when permitted. Otherwise, point to simulated CRM profile and loyalty files.
6. **Validate virtualization.** Open each shortcut folder and confirm that files are visible through OneLake. Document the remote source, authentication method, owner, and expected refresh pattern.
7. **Create the notebook.** Add `nb_capstone_retail_silver`, attach it to `lh_retail_landing`, and run a small row-count profile before transformation.
8. **Write the Silver table.** Use the notebook pattern in listing 1.5 and adjust column names to match your source files.

```

1 from pyspark.sql import functions as F
2
3 sales_path = "Files/bronze/s3_sales/**/*.parquet"
4 crm_path = "Files/bronze/dataverse_crm/**/*.parquet"
5

```

```

6 sales_raw_df = spark.read.parquet(sales_path)
7 crm_raw_df = spark.read.parquet(crm_path)
8
9 sales_df = (
10     sales_raw_df
11     .select(
12         F.col("OrderId").cast("string").alias("order_id"),
13         F.col("CustomerId").cast("string").alias("customer_id"),
14         F.to_date("OrderDate").alias("order_date"),
15         F.col("Region").cast("string").alias("region"),
16         F.col("SalesAmount").cast("decimal(18,2)").alias("sales_amount"),
17         F.col("CurrencyCode").cast("string").alias("currency_code"),
18     )
19     .where(F.col("order_id").isNotNull())
20     .dropDuplicates(["order_id"])
21 )
22
23 crm_df = (
24     crm_raw_df
25     .select(
26         F.col("CustomerId").cast("string").alias("customer_id"),
27         F.col("LoyaltyTier").cast("string").alias("loyalty_tier"),
28         F.col("EmailOptIn").cast("boolean").alias("email_opt_in"),
29         F.to_timestamp("ModifiedOn").alias("crm_modified_on"),
30     )
31     .where(F.col("customer_id").isNotNull())
32     .dropDuplicates(["customer_id"])
33 )
34
35 silver_df = (
36     sales_df
37     .join(crm_df, on="customer_id", how="left")
38     .withColumn("sales_year", F.year("order_date"))
39     .withColumn("silver_loaded_at", F.current_timestamp())
40 )
41
42 silver_df.write.format("delta").mode("overwrite").saveAsTable(
43     "silver_retail_customer_sales"
44 )

```

Listing 1.5: PySpark notebook pattern for reading shortcut data in place and writing a Silver Delta table.

The second listing is a documentation artifact you can store with your runbook or deployment notes. It records the intended shortcut layout without placing secrets in source control. Real deployments should use secure connection objects, tenant-approved credentials, and environment-specific automation.

```

1 {
2     "workspace": "fabric-de-week01-retail",
3     "lakehouse": "lh_retail_landing",
4     "bronzeShortcuts": [
5         {
6             "name": "s3_sales",
7             "oneLakePath": "Files/bronze/s3_sales",
8             "sourceType": "AmazonS3",
9             "sourceUri": "s3://retail-history-example/sales/",
10            "copyData": false,
11            "owner": "retail-platform-history"
12        },
13        {
14            "name": "dataverse_crm",
15            "oneLakePath": "Files/bronze/dataverse_crm",

```

```

16     "sourceType": "Dataverse",
17     "sourceUri": "environment://crm/customers",
18     "copyData": false,
19     "owner": "customer-experience-platform"
20   }
21 ],
22 "silverOutput": {
23   "table": "silver_retail_customer_sales",
24   "format": "Delta",
25   "materializedInOneLake": true
26 }
27 }

```

Listing 1.6: Shortcut intent record for the capstone landing zone.

If your class prefers SQL validation, run a SQL cell or SQL analytics endpoint query after the write:

```

1 SELECT
2   COUNT(*) AS row_count,
3   COUNT(DISTINCT customer_id) AS customer_count,
4   MIN(order_date) AS first_order_date,
5   MAX(order_date) AS last_order_date,
6   SUM(sales_amount) AS total_sales_amount
7 FROM silver_retail_customer_sales;

```

Listing 1.7: SQL validation for the Silver customer sales table.

## Deliverables

Submit a concise package that proves the architecture and implementation:

- Workspace name, capacity assignment, Lakehouse name, and notebook name.
- Screenshot or exported evidence showing both shortcut folders under **Files/bronze**.
- A short runbook entry for each shortcut: source owner, source URI pattern, authentication method, update frequency, and escalation contact.
- Notebook output showing row counts for historical sales, CRM profiles, and the final Silver table.
- The schema of `silver_retail_customer_sales`, including keys, dates, measures, and CRM enrichment fields.
- A paragraph explaining which data was virtualized, which data was materialized, and why that split is appropriate for Week 1.

## Acceptance Criteria

Use this rubric before declaring the capstone complete:

- ☐ The workspace exists on trial or approved Fabric capacity and contains the required Lakehouse.
- ☐ Both external sources are represented through OneLake shortcuts under the Bronze zone.
- ☐ Historical sales data is read in place and not bulk copied into OneLake.
- ☐ CRM or simulated CRM data is read in place and not bulk copied into OneLake.
- ☐ The PySpark notebook reads both shortcut paths successfully and includes basic row-count or null-count validation.



- ☐ The notebook writes `silver_retail_customer_sales` as a managed Delta table.
- ☐ The Silver table has standardized customer keys, dates, currency or amount fields, and CRM enrichment fields.
- ☐ The design explicitly distinguishes virtual Bronze access from materialized Silver curation.
- ☐ The final explanation states that raw source data remains in the external systems and that only curated output is stored as Delta-Parquet in the Lakehouse.
- ☐ No secrets, connection strings, access keys, or personal data are pasted into notebook markdown, listings, screenshots, or source-controlled files.

## Stretch Goals

If you finish early, extend the capstone without changing the core no-copy promise:

- Add a Gold aggregate table such as `gold_regional_loyalty_sales` grouped by region, sales year, and loyalty tier.
- Build a small Power BI semantic model over the Silver or Gold table and test whether Direct Lake is appropriate for the table shape.
- Add an additional shortcut to ADLS Gen2 to represent a supplier, inventory, or product catalog feed.
- Convert the notebook into a repeatable medallion job with parameters for source paths, target table name, and write mode.
- Add data quality assertions for duplicate order identifiers, invalid dates, negative sales amounts, missing customer keys, and unsupported currency codes.
- Document a promotion plan from development to test to production workspaces, including who owns shortcut credentials and who certifies the Gold semantic model.

# The Medallion Architecture

## Learning Objectives

- Explain the Bronze, Silver, and Gold responsibilities in a medallion architecture.
- Distinguish raw landing, validated conformance, and enriched analytical curation in Microsoft Fabric.
- Decide when a Fabric Lakehouse is a better serving target than a Fabric Data Warehouse, and when the warehouse is the safer choice.
- Organize workspaces and lakehouses so medallion layers scale across teams, environments, domains, and compliance boundaries.
- Deploy Bronze, Silver, and Gold lakehouses and prepare folder conventions for future transformations.
- Use PySpark to write Delta tables across medallion layers and inspect OneLake paths.
- Design a multi-workspace financial architecture that strictly isolates raw ingested logs from curated financial reports.

## Cross-Cloud Equivalence

The medallion pattern is not a Microsoft-only vocabulary. It is a durable data-organization idea: separate raw facts from validated facts, then publish curated business products. The implementation changes by platform, but the engineering intent stays consistent.

| Concept               | Fabric                                                                                                           | AWS/Azure/GCP lake                                                                                                    | data | Snowflake                                                                  |
|-----------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|------|----------------------------------------------------------------------------|
| Bronze raw zone       | OneLake such as <code>lh_bronze</code> ; raw files under <code>Files/bronze</code> and optional raw Delta tables | Object prefixes such as <code>s3://lake/raw/</code> , <code>abfss://lake/raw/</code> , or <code>gs://lake/raw/</code> |      | RAW database or schema, often loaded by Snowpipe, COPY, or external tables |
| Silver validated zone | Lakehouse or schema holding Delta tables with types, keys, deduplication, and quality checks                     | Prefixes such as <code>/validated/</code> or <code>/staging/</code> with Delta, Iceberg, or Parquet contracts         |      | STAGING, CLEAN, or CONFORMED schemas with transformed tables               |
| Gold curated zone     | Lakehouse tables, warehouse tables, semantic models, and certified Power BI datasets                             | Prefixes such as <code>/curated/</code> or <code>/analytics/</code> optimized for BI and data products                |      | ANALYTICS, MART, or presentation schemas consumed by BI tools              |
| Layer boundary        | Workspace, lakehouse, folder, table, and item permissions in OneLake                                             | Bucket, account, container, prefix, IAM, ACL, and catalog boundaries                                                  |      | Database, schema, role, warehouse, and masking-policy boundaries           |
| Promotion path        | Dev/test/prod Fabric workspaces and deployment pipelines                                                         | Environment-specific buckets or prefixes with CI/CD and catalogs                                                      |      | Environment databases plus controlled role grants and cloning              |

MongoDB can express a similar progression with collections or databases such as `raw_events`, `validated_events`, and `analytics_summaries`; however, document databases usually emphasize application access patterns rather than object-lake folder zones.

### Key Terms

**Bronze:** the raw or minimally standardized layer that preserves source fidelity. **Silver:** the validated layer that enforces schema, removes duplicates, standardizes keys, and records quality. **Gold:** the enriched and curated layer that serves business metrics, dimensional models, semantic models, and reports. **Lakehouse:** the Fabric item for Spark-first open Delta-Parquet engineering. **Warehouse:** the Fabric item for SQL-first relational modeling and serving.

## 2.1 Week 2 Roadmap

Week 2 turns the storage foundation from Chapter 1 into an operating architecture. Monday defines the medallion layers. Tuesday compares Fabric Lakehouses and Fabric Data Warehouses. Wednesday explains how to organize workspaces and lakehouses at scale. Thursday is a guided hands-on project that creates separate Bronze, Silver, and Gold lakehouses and prepares OneLake paths. Friday applies the design to a financial institution that must isolate raw ingested logs from curated financial reports.

The medallion architecture is valuable because it gives data teams a shared language for trust. Bronze answers, “What did the source send?” Silver answers, “What has engineering validated and conformed?” Gold answers, “What should the business consume?” Databricks popularized the Bronze, Silver, Gold vocabulary for lakehouse pipelines [5], and Fabric makes the pattern practical with OneLake, lakehouses, Delta tables, Spark notebooks, warehouses, and Power BI [20, 22].

**Note**

Medallion is not a law of nature. It is a contract. A small team may implement all three layers in one lakehouse; a regulated enterprise may use separate workspaces, lakehouses, capacities, domains, and security groups for each layer.

## 2.2 Monday: Medallion Principles

The simplest medallion definition is a progression from raw to trusted to useful. Each layer exists because downstream consumers need a different balance between fidelity, quality, and usability. A Bronze layer should preserve evidence. A Silver layer should enforce engineering truth. A Gold layer should publish business truth.

**Definition**

A **medallion architecture** is a layered data design that incrementally improves raw source data into validated, conformed, and curated analytical products. In Fabric, medallion layers are commonly implemented with OneLake Lakehouses, Delta tables, notebooks, pipelines, warehouses, and semantic models.

### 2.2.1 Bronze: Raw Landing and Source Fidelity

Bronze is the first analytical landing zone. It should answer audit and replay questions: which file arrived, when did it arrive, what source produced it, what schema did it have, and can the pipeline reprocess it if transformation logic changes? Bronze may contain CSV, JSON, Avro, Parquet, or raw Delta tables. The key design principle is that Bronze should be close to the source contract and should avoid premature business interpretation.

A useful Bronze table often adds ingestion metadata without changing the business payload. Examples include source file path, ingestion timestamp, batch identifier, hash values, and source system name. These fields make later debugging possible. They also preserve lineage when a Silver validation fails.

**Tip**

For regulated data, treat Bronze as evidence. Avoid overwriting raw data unless the retention policy explicitly requires it. Prefer append-only ingestion, immutable landing folders, and metadata that proves when and how records arrived.

### 2.2.2 Silver: Validation, Standardization, and Conformance

Silver is where engineering earns trust. Raw strings become typed dates, decimals, booleans, and identifiers. Duplicates are removed according to deterministic rules. Nullability expectations are checked. Source-specific codes are mapped to enterprise reference values. Personally identifiable or sensitive fields are masked, tokenized, or excluded when required by policy. Delta Lake's transaction log helps make those transformations reliable over object storage [3, 6].

Silver tables should be boring in the best sense. They are not usually the final executive dashboard tables, but they should be reusable across multiple Gold products. Customer, account, product, branch, employee, calendar, and transaction tables often become Silver building blocks.

**Common Pitfall**

**Skipping Silver to save time.** A direct Bronze-to-report path can look fast during a demo, but it pushes schema drift, duplicate handling, invalid dates, privacy rules, and business definitions into every downstream report.

**2.2.3 Gold: Enriched, Curated, and Consumption Ready**

Gold is optimized for use. It may contain star schemas, aggregate tables, metric marts, feature sets, executive reporting tables, or semantic models. A Gold product should have an owner, definition, refresh expectation, access model, and quality signal. In Fabric, Gold may be represented by Delta tables in a lakehouse, relational tables in a warehouse, or a certified Power BI semantic model that uses Direct Lake where appropriate [10].

Gold is where business language matters most. The table name `gold_daily_loan_risk` means little unless the organization agrees on loan status, delinquency bucket, exposure amount, currency conversion, calendar, and risk score timing. The architecture is technical, but the value is semantic.

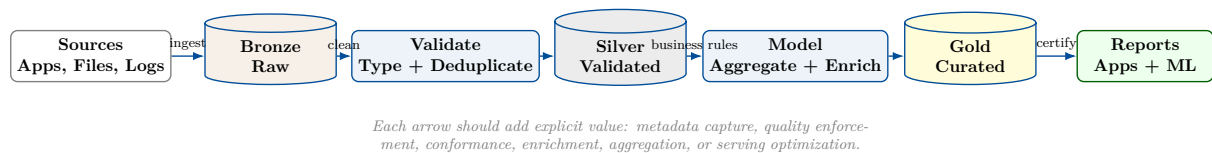


Figure 2.1: Bronze-to-Silver-to-Gold flow for a Fabric medallion architecture.

**2.2.4 Layer Contracts**

A layer contract states what consumers may assume. Bronze consumers may assume source fidelity and ingestion metadata, but not business correctness. Silver consumers may assume typed, validated, deduplicated, and conformed records. Gold consumers may assume business definitions, curated dimensions, certified metrics, and an intended serving path.

| Layer  | Primary promise                                | Typical checks                                                                 | Common consumers                                  |
|--------|------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------|
| Bronze | Preserve source payload and ingestion evidence | File count, row count, schema snapshot, corrupt-record handling                | Data engineers, auditors, reprocessing jobs       |
| Silver | Provide reliable conformed records             | Required columns, types, duplicate keys, referential checks, privacy treatment | Engineering teams, analysts, Gold pipelines       |
| Gold   | Publish business-ready products                | Metric reconciliation, dimensional completeness, performance, endorsement      | Power BI, finance teams, applications, executives |

Table 2.1: Layer contracts in a medallion design.

**2.3 Tuesday: Lakehouse vs Data Warehouse in Fabric**

Fabric includes both Lakehouse and Data Warehouse items because analytical work has different shapes. The Lakehouse is Spark-first and file-aware. The Warehouse is SQL-first and relational. Both can participate in a medallion architecture, and neither is automatically superior.

**Definition**

A **Fabric Lakehouse** is best for open Delta-Parquet engineering, notebooks, Spark transformations, files, data science, and mixed structured or semi-structured ingestion. A **Fabric Data Warehouse** is best for SQL-first relational modeling, familiar warehouse development, controlled schemas, and serving teams that primarily use T-SQL.

**2.3.1 When to Choose a Lakehouse**

Choose a Lakehouse when the work begins with files, semi-structured data, data engineering notebooks, or Delta table transformations. Bronze and Silver are frequently lakehouse-centered because engineers need to inspect files, manage folders, process large datasets with Spark, and preserve open formats. Lakehouses are also natural when data scientists need feature exploration from the same Delta tables used by data engineers.

A lakehouse does not mean ungoverned data. It simply means the primary abstraction is open files and Delta tables. Fabric's lakehouse item still participates in workspaces, OneLake, SQL analytics endpoints, semantic models, and governance metadata [20].

**2.3.2 When to Choose a Warehouse**

Choose a Warehouse when the work is primarily relational, SQL-authored, and consumption oriented. Finance reporting, regulatory aggregates, dimensional marts, and teams with strong T-SQL practices may prefer warehouse development. A warehouse can be a strong Gold layer target when consumers expect SQL schemas, constraints of a relational serving model, and controlled access patterns.

The warehouse choice can also reduce cognitive load for teams that do not need Spark. If the source is already well-modeled relational data and the transformation logic is SQL-centric, moving everything through notebooks may be unnecessary.

**2.3.3 Decision Aid**

| Decision question       | Prefer Lakehouse                                                              | Prefer Warehouse                                            |
|-------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------|
| Primary authoring style | PySpark, Spark SQL, notebooks, file operations                                | T-SQL, stored procedures, relational modeling               |
| Data shape              | Raw files, JSON, CSV, Parquet, Delta, semi-structured feeds                   | Structured tables, facts, dimensions, SQL serving           |
| Medallion layer         | Bronze and Silver by default; Gold when Direct Lake and open Delta are enough | Gold marts, finance reporting, SQL-serving products         |
| Consumer expectation    | Data engineers, data scientists, mixed engines, Power BI Direct Lake          | SQL analysts, BI engineers, governed relational users       |
| Change pattern          | Schema discovery and iterative engineering                                    | Stable schemas and curated consumption contracts            |
| Optimization focus      | File layout, partitions, Delta transactions, Spark jobs                       | SQL query patterns, dimensional models, relational security |

Table 2.2: Lakehouse-versus-Warehouse decision aid for Fabric medallion design.

### Tip

A common enterprise pattern is Lakehouse for Bronze and Silver, then either Lakehouse Gold for Direct Lake semantic models or Warehouse Gold for SQL-first finance and regulatory marts. Let consumption requirements decide the Gold serving technology.

### Common Pitfall

**Choosing by team preference only.** A Spark team may overuse notebooks for relational marts, while a SQL team may force raw file engineering into warehouse-shaped workflows. Choose the item type that best fits data shape, transformation style, and consumer contract.

## 2.4 Wednesday: Organizing Workspaces and Lakehouses at Scale

Medallion layers can be implemented inside one lakehouse, across multiple lakehouses in one workspace, or across multiple workspaces. The right boundary depends on security, team ownership, environment promotion, capacity management, and operational blast radius.

### 2.4.1 Single Lakehouse Pattern

A single lakehouse can contain `Files/bronze`, `Tables/silver_*`, and `Tables/gold_*`. This pattern is useful for training, prototypes, and small teams. It minimizes setup and makes lineage easy to see. Its weakness is that layer permissions and operational ownership are harder to separate.

### 2.4.2 Three Lakehouses in One Workspace

A workspace can contain `lh_bronze`, `lh_silver`, and `lh_gold`. This pattern makes layer intent explicit while keeping collaboration simple. It is a good default for a domain team that shares one security boundary but wants clean medallion separation.

### 2.4.3 Multi-Workspace Pattern

A regulated organization may use separate workspaces such as `fin-raw-prod`, `fin-curated-prod`, and `fin-reporting-prod`. Raw ingestion teams receive access to the raw workspace. Transformation services receive controlled access to read Bronze and write Silver. Report builders receive access only to curated Gold or semantic models. This pattern has more operational overhead, but it creates stronger isolation and auditability [23, 11].

### Note

Workspace boundaries are not just visual organization. They affect permissions, item sharing, deployment pipelines, capacity assignment, lineage review, and incident blast radius. Use them deliberately.

### 2.4.4 Naming, Ownership, and Promotion

Names should reveal layer, domain, environment, and purpose. A practical convention is `<domain>-<layer>-<env>` for workspaces and `lh_<domain>-<layer>` for lakehouses. Examples include `finance-bronze-dev`, `finance-silver-test`, and `finance-gold-prod`. Tables can use prefixes such as `bronze_`, `silver_`, and `gold_` when multiple layers share a catalog.

Ownership should be equally explicit. Bronze often belongs to ingestion or platform teams, Silver to data engineering domain teams, and Gold to business data product owners. Promotion should move definitions from development to test to production without granting developers broad write access to production raw zones.

| Scale pattern    | Best fit                                  | Strength                                  | Trade-off                           |
|------------------|-------------------------------------------|-------------------------------------------|-------------------------------------|
| One lakehouse    | Training, proof of concept, small team    | Fast setup and simple navigation          | Weak layer isolation                |
| Three lakehouses | Domain team with shared security boundary | Clear medallion ownership                 | Requires cross-lakehouse references |
| Multi-workspace  | Regulated or multi-team enterprise        | Strong isolation and blast-radius control | More governance and deployment work |

Table 2.3: Workspace and lakehouse organization patterns for medallion scale.

## 2.5 Thursday: Hands-on Project

The Week 2 lab creates separate Bronze, Silver, and Gold lakehouses. You will upload a raw CSV to Bronze, prepare folder conventions, and run PySpark patterns that write Delta data to each layer. The lab is intentionally small: the goal is to practice medallion shape before adding orchestration and production data quality gates in later weeks.

### 2.5.1 Goal and Architecture

Create a workspace named `fabric-de-week02`. Add three lakehouses: `lh_bronze`, `lh_silver`, and `lh_gold`. Upload a raw CSV file to `lh_bronze` under `Files/bronze/raw_csv/finance_transactions/`. Then use a notebook attached to the lakehouses to write Bronze, Silver, and Gold Delta outputs.

#### Note

Fabric notebook path syntax can vary depending on attached lakehouses and workspace context. In a training tenant, first use the lakehouse explorer and notebook file browser to confirm paths before parameterizing production jobs.

### 2.5.2 Step-by-Step Actions

1. Open Microsoft Fabric and create a workspace named `fabric-de-week02`. Assign the expected training capacity.
2. Create lakehouses named `lh_bronze`, `lh_silver`, and `lh_gold`.
3. In `lh_bronze`, create folders `Files/bronze/raw_csv/finance_transactions/`, `Files/bronze/quarant` and `Files/bronze/metadata/`.
4. Upload a small raw CSV with columns such as `transaction_id`, `account_id`, `transaction_date`, `amount`, `currency`, and `source_system`.
5. In `lh_silver`, reserve table names such as `silver_finance_transactions` and `silver_account_daily`.
6. In `lh_gold`, reserve table names such as `gold_daily_cash_movement` and `gold_currency_exposure`.
7. Create a notebook named `nb_week02_medallion_setup`. Attach the three lakehouses if your Fabric experience supports multiple attachments; otherwise run one layer at a time with the appropriate default lakehouse.
8. Run the following PySpark patterns, adjusting paths for your tenant and sample file.

```

1 from pyspark.sql import functions as F
2
3 raw_path = "Files/bronze/raw_csv/finance_transactions/*.csv"
4

```



```

5 raw_df = (
6     spark.read
7     .option("header", "true")
8     .option("inferSchema", "false")
9     .csv(raw_path)
10 )
11
12 bronze_df = (
13     raw_df
14     .withColumn("ingested_at", F.current_timestamp())
15     .withColumn("source_file", F.input_file_name())
16 )
17
18 bronze_df.write.format("delta").mode("overwrite").saveAsTable(
19     "bronze_finance_transactions_raw"
20 )
21
22 silver_df = (
23     bronze_df
24     .select(
25         F.col("transaction_id").cast("string").alias("transaction_id"),
26         F.col("account_id").cast("string").alias("account_id"),
27         F.to_date("transaction_date").alias("transaction_date"),
28         F.col("amount").cast("decimal(18,2)").alias("amount"),
29         F.upper(F.col("currency")).alias("currency"),
30         F.col("source_system").cast("string").alias("source_system"),
31         "ingested_at",
32         "source_file",
33     )
34     .where(F.col("transaction_id").isNotNull())
35     .where(F.col("account_id").isNotNull())
36     .where(F.col("transaction_date").isNotNull())
37     .dropDuplicates(["transaction_id"])
38 )
39
40 silver_df.write.format("delta").mode("overwrite").saveAsTable(
41     "silver_finance_transactions"
42 )
43
44 gold_df = (
45     silver_df
46     .groupBy("transaction_date", "currency")
47     .agg(
48         F.count("*").alias("transaction_count"),
49         F.sum("amount").alias("net_amount"),
50         F.sum(F.when(F.col("amount") >= 0, F.col("amount")).otherwise(F.lit(0)))
51         .alias("credits"),
52         F.sum(F.when(F.col("amount") < 0, F.col("amount")).otherwise(F.lit(0)))
53         .alias("debits"),
54     )
55     .withColumn("gold_loaded_at", F.current_timestamp())
56 )
57
58 gold_df.write.format("delta").mode("overwrite").saveAsTable(
59     "gold_daily_cash_movement"
60 )

```

Listing 2.1: PySpark pattern for reading raw CSV and writing Delta outputs to Bronze, Silver, and Gold.

The next listing records folder and OneLake path conventions. Use it as a notebook cell, deployment note, or runbook fragment. It keeps names explicit before automation is introduced.

```

1 workspace = "fabric-de-week02"
2 lakehouses = {
3     "bronze": "lh_bronze",
4     "silver": "lh_silver",
5     "gold": "lh_gold",
6 }
7
8 path_conventions = [
9     "Files/bronze/raw_csv/finance_transactions/",
10    "Files/bronze/quarantine/",
11    "Files/bronze/metadata/",
12    "Tables/bronze_finance_transactions_raw",
13    "Tables/silver_finance_transactions",
14    "Tables/gold_daily_cash_movement",
15 ]
16
17 for layer, lakehouse in lakehouses.items():
18     print(f"{layer}: {workspace}/{lakehouse}")
19
20 for path in path_conventions:
21     print(path)

```

Listing 2.2: Folder and OneLake path convention helper for Week 2.

### 2.5.3 Optional Spark SQL Validation

After the writes complete, validate row counts and simple totals. These checks do not prove financial correctness, but they confirm that each layer exists and is queryable.

```

1 SELECT 'bronze' AS layer, COUNT(*) AS row_count
2 FROM bronze_finance_transactions_raw
3 UNION ALL
4 SELECT 'silver' AS layer, COUNT(*) AS row_count
5 FROM silver_finance_transactions
6 UNION ALL
7 SELECT 'gold' AS layer, COUNT(*) AS row_count
8 FROM gold_daily_cash_movement;

```

Listing 2.3: Spark SQL validation for Week 2 medallion tables.

### 2.5.4 Validation Checklist

- The workspace exists and contains `lh_bronze`, `lh_silver`, and `lh_gold`.
- A raw CSV is uploaded only to the Bronze raw folder.
- Bronze output preserves source fields plus ingestion metadata.
- Silver output casts types, filters invalid keys, removes duplicate transaction identifiers, and writes Delta.
- Gold output summarizes daily cash movement by date and currency.
- Folder conventions are documented without secrets or connection strings.

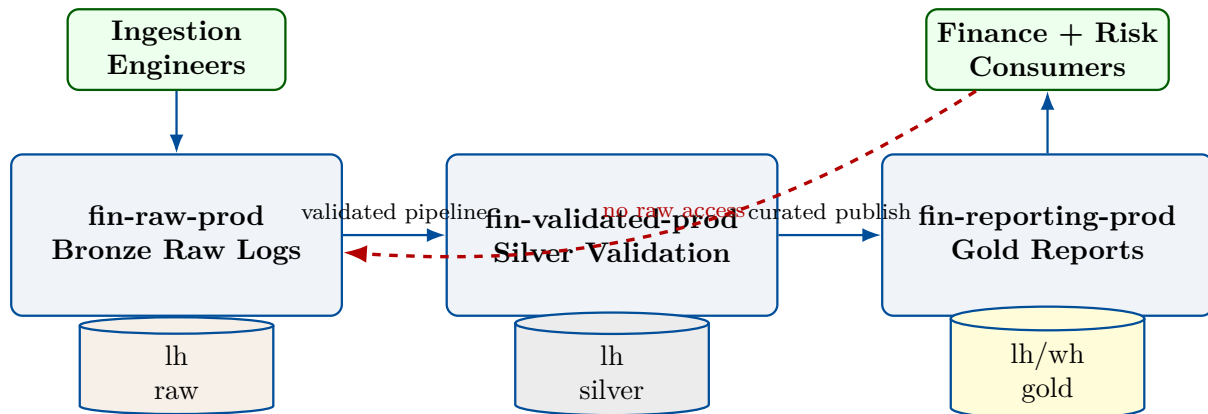
## 2.6 Friday: Use Case and Architecture

A financial institution ingests raw application logs, payment events, customer interaction events, and operational telemetry. The raw logs may contain sensitive identifiers, operational secrets, IP addresses, device identifiers, and unfiltered message text. Finance and risk teams need curated

reports about transaction volume, exception rates, settlement timeliness, and control failures. They must not receive broad access to raw logs.

### 2.6.1 Architecture Strategy

The design separates raw ingestion from curated reporting through workspaces. **fin-raw-prod** contains Bronze lakehouses and ingestion notebooks. **fin-validated-prod** contains Silver validation tables and quality outcomes. **fin-reporting-prod** contains Gold reporting tables, a warehouse or semantic model, and certified reports. Access flows forward through controlled jobs and item sharing; business users never receive direct access to the raw workspace.



*Workspace isolation prevents report users from browsing raw logs while still allowing governed, forward-only medallion publication.*

Figure 2.2: Multi-workspace isolation pattern for financial medallion architecture.

### 2.6.2 Design Decisions

Raw logs remain in the Bronze workspace with restricted permissions because they may contain sensitive or irrelevant operational detail. Silver jobs parse, classify, mask, deduplicate, and validate events. Gold tables expose only approved metrics, dimensions, and report-ready aggregates. The organization can certify Gold semantic models without granting report builders access to raw messages.

| Concern              | Isolation decision                                                                    | Medallion impact                                           |
|----------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------|
| Raw log sensitivity  | Restrict <b>fin-raw-prod</b> to ingestion operators and controlled service principals | Bronze stores evidence but is not broadly browsable        |
| Validation ownership | Data engineering owns <b>fin-validated-prod</b> and publishes quality results         | Silver becomes the trusted reuse layer                     |
| Reporting access     | Finance and risk users access <b>fin-reporting-prod</b> only                          | Gold contains approved metrics and semantic models         |
| Incident response    | Raw workspace access can be audited separately                                        | Investigations do not require report workspace permissions |

Table 2.4: Financial institution decisions for strict medallion isolation.

## 2.7 Operational Guidance for Week 2

A medallion design becomes real only when ownership and controls match the diagrams. Every layer should have an owner, retention expectation, quality expectation, and consumer policy. A missing policy will eventually become a production incident: either raw data leaks too widely, Silver quality is assumed without evidence, or Gold reports drift from definitions.

### 2.7.1 Quality and Observability

Quality checks should become more strict as data moves forward. Bronze checks prove ingestion completeness. Silver checks prove validity and conformance. Gold checks prove business reconciliation. Fabric capacity monitoring also matters because inefficient transformations can consume shared capacity units [13].

```

1 from pyspark.sql import functions as F
2
3 required_columns = [
4     "transaction_id",
5     "account_id",
6     "transaction_date",
7     "amount",
8     "currency",
9 ]
10
11 missing_columns = sorted(set(required_columns) - set(silver_df.columns))
12 if missing_columns:
13     raise ValueError(f"Missing required columns: {missing_columns}")
14
15 quality_df = silver_df.select(
16     F.count("*").alias("rows"),
17     F.sum(F.col("transaction_id").isNull().cast("int")).alias("
18 null_transaction_ids"),
19     F.countDistinct("transaction_id").alias("distinct_transactions"),
20     F.sum(F.col("amount").isNull().cast("int")).alias("null_amounts"),
21 )
22 quality_df.show(truncate=False)

```

Listing 2.4: Small quality gate for Silver transaction data.

### 2.7.2 Security Checklist

- Limit Bronze workspace membership to ingestion operators and approved service principals.
- Store curated outputs in a separate workspace when raw data contains sensitive or unfiltered fields.
- Use item sharing, semantic model permissions, and workspace roles intentionally rather than relying on naming conventions.
- Remove raw message text, secrets, tokens, and unnecessary identifiers before Gold publication.
- Record lineage from Gold reports back to Silver validation tables and Bronze ingestion batches.
- Audit who can create shortcuts or copy raw folders into less restricted workspaces.

## 2.8 Interview Q&A

1. **Q: What problem does the medallion architecture solve?**

A: It separates source fidelity, engineering validation, and business consumption so teams know which data is raw, which data is trusted, and which data is certified for decisions.

2. **Q: What belongs in Bronze?**

A: Raw or minimally standardized source data, ingestion metadata, source file paths, batch identifiers, and enough evidence to replay or audit ingestion.

3. **Q: What makes Silver different from Bronze?**

A: Silver applies schema enforcement, type casting, deduplication, key standardization, privacy treatment, and quality checks.

4. **Q: What makes Gold different from Silver?**

A: Gold adds business definitions, aggregations, dimensional models, semantic models, performance tuning, and certification for consumption.

5. **Q: When would you choose a Fabric Warehouse for Gold?**

A: Choose it when the Gold product is SQL-first, relational, finance-oriented, or consumed by teams that expect controlled warehouse schemas.

6. **Q: Why might a financial institution split medallion layers across workspaces?**

A: Separate workspaces enforce stronger access boundaries, limit raw data exposure, reduce blast radius, and support audited promotion from raw logs to curated reports.

## 2.9 Further Reading

Review Microsoft’s Fabric overview [21], OneLake overview [22], lakehouse documentation [20], workspace guidance [23], domains [11], Direct Lake [10], and capacity metrics [13]. For lakehouse theory and storage reliability, read the medallion architecture overview [5], the Delta Lake paper [3], Delta Lake documentation [6], and the lakehouse architecture paper [4].

## Key Takeaways

- Bronze preserves source fidelity and ingestion evidence; it should not be treated as business-ready data.
- Silver validates, conforms, deduplicates, types, and secures data so it can be reused safely.
- Gold publishes curated business products through lakehouse tables, warehouses, semantic models, and reports.
- Lakehouses are best for Spark-first, file-aware, open Delta-Parquet engineering; warehouses are best for SQL-first relational serving.
- Workspace and lakehouse boundaries should reflect ownership, security, environment promotion, and operational blast radius.
- Regulated scenarios often require multi-workspace isolation so raw logs and curated reports have different access models.
- Every medallion layer needs an owner, contract, quality expectation, and documented consumer policy.

## 2.10 Exercises

1. **(Conceptual)** Define Bronze, Silver, and Gold in your own words. Include one example table for each layer.
2. **(Conceptual)** Explain why preserving source fidelity in Bronze can be more important than cleaning data immediately.
3. **(Design)** Choose Lakehouse or Warehouse for a Gold monthly finance mart. Justify the decision using authoring style, consumers, and security.
4. **(Design)** Draft a workspace naming convention for development, test, and production medallion workspaces in a bank.
5. **(Hands-on)** Create the three Week 2 lakehouses and upload a sample CSV to the Bronze raw folder.
6. **(Hands-on)** Modify listing 2.1 to quarantine rows with missing `transaction_id` instead of dropping them silently.
7. **(Operational)** Write a runbook checklist for promoting a Silver table to Gold, including quality, owner approval, and access review.
8. **(Interview)** In four sentences, defend why raw financial logs should not be exposed directly to report authors.

## 2.11 Capstone Project: Financial Medallion with Workspace Isolation

### Capstone Project

**Mission.** Build an end-to-end medallion architecture for a financial institution. The architecture must isolate raw ingested logs from curated financial reports by using separate Fabric workspaces and medallion lakehouses. Your implementation should prove that raw events can be landed, validated, curated, and reported without giving report consumers direct access to raw log folders.

**Estimated time.** 4–6 hours, depending on tenant permissions, available sample data, and whether your class uses separate development and production workspaces.

**What you'll practice.**

- Creating medallion workspaces for raw, validated, and reporting layers.
- Landing raw financial logs in a restricted Bronze lakehouse.
- Validating and masking data into a Silver Delta table.
- Publishing a Gold aggregate for finance and risk reporting.
- Documenting access boundaries, lineage, and acceptance criteria for a regulated data product.

### Scenario

A financial institution operates digital banking, card processing, loan servicing, and fraud-monitoring platforms. Each platform emits operational logs and transaction events. Raw files may include account identifiers, user agents, IP addresses, exception text, internal service names, and operational metadata. Finance and risk teams need curated metrics about transaction volume, rejected payments, settlement timeliness, high-risk exception categories,

and daily exposure. They must not browse raw logs because raw events may contain sensitive or irrelevant operational details.

Your job is to design and implement the smallest production-shaped medallion that proves separation of duties. The ingestion team owns raw landing. The data engineering team owns validation. Finance owns curated reporting definitions. Access must flow forward through governed transformations, not through broad workspace membership.

## Requirements

Build a working Fabric design with these minimum elements:

- A restricted workspace named `fin-raw-dev` or `fin-raw-prod` with a Bronze lakehouse named `lh_fin_bronze`.
- A validation workspace named `fin-validated-dev` or `fin-validated-prod` with a Silver lakehouse named `lh_fin_silver`.
- A reporting workspace named `fin-reporting-dev` or `fin-reporting-prod` with a Gold lakehouse or warehouse named `lh_fin_gold` or `wh_fin_gold`.
- A raw folder path `Files/bronze/raw_logs/payment_events/` that contains sample CSV or JSON events.
- A Silver Delta table named `silver_payment_events` with typed fields, duplicate removal, and sensitive-field treatment.
- A Gold table named `gold_daily_payment_controls` with daily metrics suitable for finance and risk reports.
- Documentation proving that report consumers need access only to the reporting workspace or semantic model.
- No secrets, keys, tokens, connection strings, or real personal data in notebooks, screenshots, or exported files.

## Reference Architecture

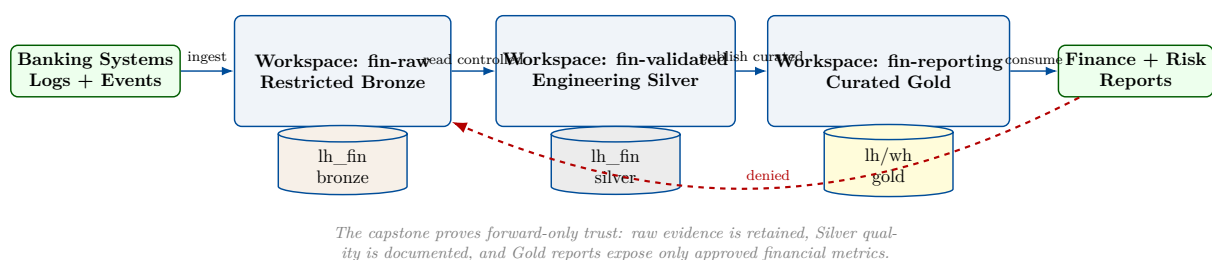


Figure 2.3: Reference architecture for the financial medallion capstone.

## Implementation Steps

1. **Create workspaces.** Create `fin-raw-dev`, `fin-validated-dev`, and `fin-reporting-dev`. If your tenant permits only one workspace, simulate the boundary with three lakehouses and document the production workspace split.
2. **Create lakehouses.** Add `lh_fin_bronze`, `lh_fin_silver`, and `lh_fin_gold`. Use a warehouse for Gold if your team chooses SQL-first reporting.

3. **Create raw folders.** In Bronze, create `Files/bronze/raw_logs/payment_events/`, `Files/bronze/quarantine/`, and `Files/bronze/metadata/`.
4. **Upload sample events.** Use instructor-provided synthetic logs or create a small CSV with payment event identifiers, timestamps, account identifiers, amount, currency, status, exception category, and channel.
5. **Write Bronze and Silver.** Run the first notebook listing to preserve raw ingestion metadata and create `silver_payment_events`.
6. **Write Gold.** Run the second listing to create `gold_daily_payment_controls` for reporting.
7. **Document access.** Record which personas can access each workspace and explain why report consumers do not need raw workspace membership.
8. **Validate lineage.** Capture row counts for raw, Silver, and Gold outputs and explain any rejected or quarantined records.

```

1 from pyspark.sql import functions as F
2
3 raw_path = "Files/bronze/raw_logs/payment_events/*.csv"
4
5 raw_events_df = (
6     spark.read
7     .option("header", "true")
8     .option("inferSchema", "false")
9     .csv(raw_path)
10 )
11
12 bronze_events_df = (
13     raw_events_df
14     .withColumn("ingested_at", F.current_timestamp())
15     .withColumn("source_file", F.input_file_name())
16 )
17
18 bronze_events_df.write.format("delta").mode("overwrite").saveAsTable(
19     "bronze_payment_events_raw"
20 )
21
22 silver_payment_events_df = (
23     bronze_events_df
24     .select(
25         F.col("event_id").cast("string").alias("event_id"),
26         F.col("payment_id").cast("string").alias("payment_id"),
27         F.sha2(F.col("account_id").cast("string"), 256).alias("account_hash"),
28         F.to_timestamp("event_timestamp").alias("event_timestamp"),
29         F.to_date("event_timestamp").alias("event_date"),
30         F.col("amount").cast("decimal(18,2)").alias("amount"),
31         F.upper(F.col("currency")).alias("currency"),
32         F.upper(F.col("status")).alias("status"),
33         F.col("exception_category").cast("string").alias("exception_category"),
34         F.col("channel").cast("string").alias("channel"),
35         "ingested_at",
36         "source_file",
37     )
38     .where(F.col("event_id").isNotNull())
39     .where(F.col("payment_id").isNotNull())
40     .where(F.col("event_timestamp").isNotNull())
41     .dropDuplicates(["event_id"])
42 )
43
44 silver_payment_events_df.write.format("delta").mode("overwrite").saveAsTable(

```



```

45     "silver_payment_events"
46 )

```

Listing 2.5: Capstone Bronze-to-Silver PySpark transformation with sensitive-field treatment.

```

1  from pyspark.sql import functions as F
2
3  gold_daily_controls_df = (
4      silver_payment_events_df
5      .groupBy("event_date", "currency", "channel")
6      .agg(
7          F.count("*").alias("event_count"),
8          F.countDistinct("payment_id").alias("payment_count"),
9          F.sum("amount").alias("gross_amount"),
10         F.sum(F.when(F.col("status") == "REJECTED", 1).otherwise(0)).alias("
rejected_count"),
11         F.sum(F.when(F.col("exception_category").isNotNull(), 1).otherwise(0)).
alias("exception_count"),
12     )
13     .withColumn(
14         "rejection_rate",
15         F.col("rejected_count") / F.when(F.col("event_count") == 0, None).
otherwise(F.col("event_count")),
16     )
17     .withColumn("gold_loaded_at", F.current_timestamp())
18 )
19
20 gold_daily_controls_df.write.format("delta").mode("overwrite").saveAsTable(
21     "gold_daily_payment_controls"
22 )
23
24 gold_daily_controls_df.orderBy("event_date", "currency", "channel").show(50,
truncate=False)

```

Listing 2.6: Capstone Silver-to-Gold PySpark aggregate for financial controls.

If Gold is implemented as a warehouse table, use a SQL validation query after loading or shortcutting the curated data into the SQL-serving layer.

```

1  SELECT
2      event_date,
3      currency,
4      SUM(event_count) AS event_count,
5      SUM(rejected_count) AS rejected_count,
6      SUM(exception_count) AS exception_count
7  FROM gold_daily_payment_controls
8  GROUP BY event_date, currency
9  ORDER BY event_date, currency;

```

Listing 2.7: Capstone SQL validation for curated financial control metrics.

## Deliverables

Submit evidence that the architecture and implementation are complete:

- Workspace names, lakehouse or warehouse names, and capacity assignment notes.
- Screenshot or exported evidence of the Bronze raw folder and the Silver and Gold tables.
- Notebook output showing row counts for Bronze, Silver, and Gold.
- Schema for `silver_payment_events`, including which sensitive fields were hashed, removed, or retained.

- Schema for `gold_daily_payment_controls`, including metric definitions.
- Access matrix listing ingestion engineers, data engineers, finance report authors, risk consumers, and administrators.
- A short explanation of why report consumers do not need access to `fin-raw-dev` or `fin-raw-prod`.

## Acceptance Criteria

Use this rubric before declaring the capstone complete:

- ☐ Raw files are stored only in the Bronze raw path.
- ☐ Bronze data includes ingestion metadata such as load timestamp and source file.
- ☐ Silver data casts types, removes duplicate event identifiers, and handles sensitive account identifiers.
- ☐ Gold data contains only curated metrics required for finance and risk reporting.
- ☐ Report consumers can use Gold outputs without direct raw workspace access.
- ☐ The access matrix shows different personas for raw ingestion, validation, and reporting.
- ☐ Row counts are reconciled or rejected records are explained.
- ☐ No secrets, credentials, tokens, real account identifiers, or unnecessary personal data appear in submitted artifacts.
- ☐ The final architecture explains how data moves forward from Bronze to Silver to Gold.
- ☐ The design states whether Gold is served from a Lakehouse, Warehouse, semantic model, or a combination.

## Stretch Goals

If you finish early, extend the capstone while preserving strict isolation:

- Add a quarantine table for invalid payment events and report rejection reasons by source file.
- Add a reference table for currency validation and reject unsupported currency codes.
- Build a small Power BI semantic model over `gold_daily_payment_controls` and document whether Direct Lake is appropriate.
- Split development and production into separate workspace sets and describe a deployment-pipeline promotion path.
- Add sensitivity labels or item-level notes that identify raw, validated, and curated assets.
- Add a reconciliation check that compares total Bronze payment identifiers with Silver accepted plus quarantined identifiers.

# Apache Spark in Fabric

## Learning Objectives

- Explain how Microsoft Fabric provides managed Apache Spark compute through Starter pools and Custom pools.
- Configure Spark environments so notebooks use consistent libraries, runtime versions, and session settings.
- Use Fabric Notebooks to write PySpark that reads and writes files in OneLake.
- Combine PySpark DataFrame logic and Spark SQL in the same notebook for practical lakehouse engineering.
- Build a Bronze-to-Silver transformation that reads raw CSV, removes duplicates, drops invalid null records, and writes Delta tables.
- Design a scalable Spark notebook ingestion layer for telemetry systems that process 100 GB of JSON logs daily.
- Implement a capstone telemetry pipeline from Bronze JSON to cleaned Silver Delta with operational checks.

## Cross-Cloud Equivalence

Managed Spark exists on every major analytical cloud, but the operating model differs. Fabric hides much of the cluster administration behind pools, lakehouses, environments, and notebooks; other platforms expose more choices around clusters, jobs, and object storage.

| Concept                                                                                                                                                                                                                                                                                   | Fabric Spark                                                                                                           | AWS                                                                                                                  | Azure                                                                                        | GCP                                                                         |                                   |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------|
| Cluster or pool model                                                                                                                                                                                                                                                                     | Starter pools for fast managed sessions; Custom pools for controlled node family, size, and scale settings             | AWS Glue jobs, EMR clusters, and EMR Serverless applications                                                         | Synapse Spark pools and Azure Databricks clusters or serverless compute                      | Dataprocs, Databricks batches                                               | clusters and Serverless           |
| Autoscaling                                                                                                                                                                                                                                                                               | Pool-level autoscale and dynamic Spark allocation are managed through Fabric capacity and pool settings                | Glue workers scale by job configuration; EMR supports managed scaling; EMR Serverless scales per application         | Synapse pools and Databricks support autoscale settings by pool or cluster                   | Dataprocs, Databricks policies and serverless automatic resource allocation | autoscaling                       |
| Notebook experience                                                                                                                                                                                                                                                                       | Fabric Notebooks attach to lakehouses, use Spark sessions, support PySpark, Spark SQL, and collaborative authoring     | Glue Studio notebooks, EMR notebooks, SageMaker Studio, or managed Jupyter patterns                                  | Synapse notebooks and Databricks notebooks with rich job integration                         | Vertex AI, Dataprocs, and notebook batches                                  | Workbench, Jupyter, and connected |
| Delta and lakehouse integration                                                                                                                                                                                                                                                           | Native Fabric Lakehouse tables in OneLake use Delta-Parquet and integrate with SQL analytics endpoints and Direct Lake | Delta Lake on S3 through EMR, Glue, or Databricks; catalog integration depends on Glue Data Catalog or Unity Catalog | Delta Lake through Synapse or Databricks on ADLS; strong lakehouse integration in Databricks | Delta Lake on Cloud Storage through Dataprocs or Databricks on GCP          |                                   |
| Serverless option                                                                                                                                                                                                                                                                         | Starter pools feel serverless to notebook authors; Fabric capacity still governs consumption                           | AWS Glue and EMR Serverless                                                                                          | Synapse serverless patterns for SQL, and Databricks serverless where available               | Dataprocs for Spark batches                                                 | Serverless                        |
| Snowflake approaches distributed data engineering through Snowpark rather than a user-managed Spark pool. MongoDB commonly participates through the MongoDB Spark Connector, where Spark reads operational collections and writes curated analytical outputs to a lakehouse or warehouse. |                                                                                                                        |                                                                                                                      |                                                                                              |                                                                             |                                   |

### Key Terms

**Driver:** the Spark process that plans work and coordinates executors. **Executor:** a worker process that runs tasks and stores shuffle or cached data. **Starter pool:** Fabric-managed Spark compute intended for quick notebook startup and low-friction development. **Custom pool:** a Fabric Spark pool with explicit sizing and scale settings. **Spark environment:** a reusable Fabric item that pins runtime, libraries, and Spark properties.

## 3.1 Week 3 Roadmap

Week 3 moves from storage architecture into distributed processing. Monday introduces Spark compute in Fabric, including Starter pools, Custom pools, and Spark environments. Tuesday focuses on Fabric Notebooks, PySpark basics, and reading or writing files in OneLake. Wednesday adds Spark SQL and shows how to combine SQL cells with PySpark DataFrames in the same notebook. Thursday is a hands-on transformation from raw Bronze CSV to cleaned Silver Delta tables. Friday applies the pattern to a telemetry platform that must process 100 GB of JSON logs every day.

Apache Spark is the distributed execution engine behind many modern lakehouse workloads. It splits large file scans and transformations into tasks, runs them across executors, and writes outputs in formats such as Delta and Parquet. Fabric packages this capability as a managed

notebook and lakehouse experience so data engineers can focus on ingestion, validation, and table contracts rather than cluster plumbing [2, 7].

#### Note

Spark is not required for every Fabric workload. Use it when the data shape, volume, file format, transformation logic, or semi-structured parsing requirement benefits from distributed processing. For small relational transformations, Warehouse SQL or Dataflows Gen2 may be simpler.

## 3.2 Monday: Spark Compute in Fabric

Fabric Spark compute is organized around managed sessions. A notebook requests a Spark session from a pool, the session starts with a runtime and environment, and Spark creates a driver plus executors to process work. The user experience is intentionally simpler than administering open-source clusters, but the engineering decisions still matter: startup latency, concurrency, scale limits, library versions, and capacity consumption.

#### Definition

**Apache Spark in Fabric** is a managed Spark service for notebooks, lakehouse transformations, machine learning preparation, and file-scale data engineering. Spark sessions run on Fabric capacity, read and write OneLake, and commonly store trusted outputs as Delta tables.

### 3.2.1 Starter Pools

Starter pools are the fastest way to begin Spark development in Fabric. They are managed for convenience and optimized for quick session startup. A training notebook, exploratory data profiling task, or small Bronze-to-Silver prototype is a good fit when the default runtime and general sizing are enough.

Starter pools reduce friction. New engineers can open a notebook, attach a lakehouse, run a PySpark cell, and inspect results without waiting for a separately administered cluster. The trade-off is control: production workloads may need more explicit sizing, library pinning, and concurrency planning than a starter configuration provides.

#### Tip

Use Starter pools while discovering schemas, writing the first notebook cells, and teaching the programming model. Move repeatable production jobs to a controlled pool and environment once workload size, dependencies, and service-level expectations are known.

### 3.2.2 Custom Pools

Custom pools give teams more control over Spark resources. A Custom pool can specify node families, node sizes, autoscale boundaries, and other settings that affect throughput and cost. The goal is not to create the largest possible cluster; the goal is to provide enough parallelism for the workload while keeping predictable capacity consumption.

Custom pools are useful when notebooks become scheduled jobs, when multiple pipelines compete for the same capacity, or when a large transformation repeatedly reads many files. For the telemetry scenario later in this chapter, a Custom pool is easier to reason about because daily data volume, expected duration, and concurrency can be estimated.

**Common Pitfall**

**Treating pool size as the only performance lever.** Larger pools may hide inefficient code for a while, but file layout, partition sizing, schema inference, skew, shuffle volume, and Delta write patterns often matter more than raw executor count.

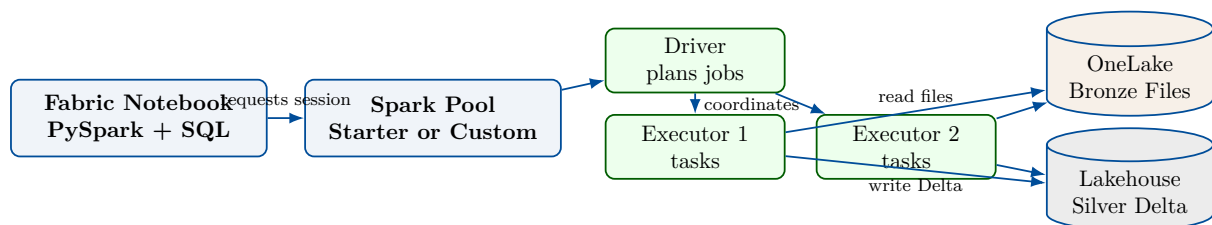
### 3.2.3 Spark Environments

Spark environments package runtime choices, libraries, and Spark properties so notebooks start consistently [8]. Without an environment, two notebooks can drift: one may install a library interactively, another may use a different runtime, and a scheduled job may fail because a dependency was not available. Environments make notebook behavior repeatable.

A practical environment should specify only what the workload needs. Common examples include JSON parsing helpers, validation packages approved by the organization, or Spark properties for shuffle partitions. Avoid turning the environment into a dumping ground for every library used by every team.

| Choice            | Best fit                                                        | Engineering concern                                 |
|-------------------|-----------------------------------------------------------------|-----------------------------------------------------|
| Starter pool      | Training, exploration, short notebooks, early prototypes        | Limited control and less workload-specific sizing   |
| Custom pool       | Scheduled transformations, larger files, predictable operations | Requires sizing, autoscale, and capacity planning   |
| Spark environment | Repeatable notebooks and dependency governance                  | Must be versioned and kept small enough to maintain |

Table 3.1: Compute choices for Spark development in Fabric.



*Fabric manages session startup and pool resources, while Spark still uses the standard driver-and-executor execution model over OneLake files and Delta tables.*

Figure 3.1: Spark driver and executors on a Fabric pool reading OneLake and writing Delta.

## 3.3 Tuesday: Fabric Notebooks and PySpark Basics

Fabric Notebooks are interactive development items for Spark workloads. A notebook can attach to a lakehouse, run PySpark cells, use Spark SQL, display DataFrames, and be scheduled or orchestrated later. The notebook is both a development surface and a repeatable job artifact, so it should be written with production habits even during learning [12].

### 3.3.1 The PySpark DataFrame Model

PySpark code usually describes transformations before Spark executes them. Reading a file creates a DataFrame. Selecting columns, casting types, filtering rows, and joining tables create

a logical plan. Spark executes the plan when an action such as `count()`, `show()`, `write`, or `collect()` is called.

This lazy execution model is powerful because Spark can optimize the plan. It can also surprise beginners. A cell that defines ten transformations may appear to succeed even when the source path is wrong; the error appears only when an action triggers a scan.

#### Definition

A **DataFrame** is a distributed table-like abstraction with named columns and a schema. In PySpark, DataFrames are transformed with methods such as `select`, `where`, `groupBy`, and `join`; Spark translates those operations into distributed execution.

### 3.3.2 Reading Files from OneLake

A Fabric notebook attached to a lakehouse can read relative paths under `Files/`. For example, raw CSV might live under `Files/bronze/raw_csv/orders/`, and raw JSON telemetry might live under `Files/bronze/raw_json/telemetry/`. In production, use explicit path conventions, avoid hardcoded personal workspace names, and validate that files exist before starting expensive transformations.

#### Tip

For CSV, specify whether headers exist and decide whether schema inference is acceptable. For large or scheduled jobs, prefer explicit schemas so a single malformed file cannot silently change column types.

### 3.3.3 Writing Delta Tables

Fabric lakehouse tables are commonly written as Delta tables. A DataFrame write using `format("delta")` and `saveAsTable(...)` creates a table that can be read by Spark and exposed through the lakehouse experience. Delta transactions provide a safer table abstraction than unmanaged folders because readers see consistent versions and writers record metadata in the transaction log [3, 6].

#### Common Pitfall

**Using `overwrite` casually.** `mode("overwrite")` is convenient in labs, but production jobs should specify overwrite scope carefully, preserve history when required, and avoid replacing an entire table because one daily batch arrived late.

## 3.4 Wednesday: Spark SQL and Hybrid Notebooks

Spark SQL lets engineers express transformations in SQL while Spark executes them on the same distributed engine. In Fabric notebooks, a team can use PySpark for file parsing and schema logic, then use SQL for validation queries, aggregates, or analyst-friendly transformations. This hybrid style is practical because data engineering teams often include both Python-first and SQL-first contributors.

### 3.4.1 Temporary Views

A PySpark DataFrame can be registered as a temporary view with `createOrReplaceTempView`. After that, Spark SQL can query it in the same session. Temporary views are not durable lakehouse tables; they are session-scoped names that help organize notebook logic.

**Note**

Use temporary views to bridge cells, not as production storage. Durable outputs should be written as Delta tables with clear names, owners, and refresh expectations.

### 3.4.2 Choosing DataFrame or SQL Syntax

DataFrame syntax works well when logic is programmatic: dynamic column lists, loops over quality rules, reusable functions, or complex parsing. SQL syntax works well for joins, aggregates, window functions, and validation queries that analysts can read. Both compile to Spark plans, so the choice is often about maintainability.

| Need                   | Prefer PySpark                                 | Prefer Spark SQL                                              |
|------------------------|------------------------------------------------|---------------------------------------------------------------|
| File parsing           | Explicit schemas, file metadata, dynamic paths | Simple external table queries after files are tabled          |
| Data quality           | Programmatic checks across many columns        | Clear count and exception queries                             |
| Aggregations           | Reusable functions and conditional expressions | Grouping, joins, and window calculations readable by analysts |
| Notebook collaboration | Python-first engineering team                  | SQL-first analytics team                                      |

Table 3.2: Choosing PySpark or Spark SQL inside Fabric notebooks.

## 3.5 Thursday: Hands-on Project

The Week 3 lab implements a small but production-shaped Silver transformation. A raw CSV arrives in the Bronze Lakehouse. The notebook reads the file, captures ingestion metadata, removes duplicate business keys, drops rows missing required fields, and writes the accepted records to the Silver Lakehouse as a Delta table. The same notebook then mixes PySpark with Spark SQL for validation and summary checks.

### 3.5.1 Goal and Architecture

Create or reuse a workspace named `fabric-de-week03`. Add lakehouses named `lh_bronze` and `lh_silver`. Upload raw CSV files to `lh_bronze` under `Files/bronze/raw_csv/telemetry_devices/`. Create a notebook named `nb_week03_spark_csv_to_silver` and attach the required lakehouses.

**Note**

If your tenant permits only one default lakehouse per notebook, first run the lab in one lakehouse with explicit layer prefixes. Then repeat in a multi-lakehouse setup when attachments and shortcuts are available.

### 3.5.2 Step-by-Step Actions

1. Create `fabric-de-week03`, `lh_bronze`, and `lh_silver` if they do not already exist.
2. In Bronze, create `Files/bronze/raw_csv/telemetry_devices/` and upload a CSV with columns such as `event_id`, `device_id`, `event_timestamp`, `temperature_c`, `status`, and `source_system`.



3. Create a notebook attached to the Bronze and Silver lakehouses, or attach one lakehouse and adjust paths for your training tenant.
4. Run the PySpark transformation in listing 3.1.
5. Run the hybrid Spark SQL validation in listing 3.2.
6. Confirm that `silver_telemetry_device_events` appears as a Delta table and that duplicate and null records are excluded.

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.types import StructType, StructField, StringType
3
4 raw_schema = StructType([
5     StructField("event_id", StringType(), True),
6     StructField("device_id", StringType(), True),
7     StructField("event_timestamp", StringType(), True),
8     StructField("temperature_c", StringType(), True),
9     StructField("status", StringType(), True),
10    StructField("source_system", StringType(), True),
11 ])
12
13 raw_path = "Files/bronze/raw_csv/telemetry_devices/*.csv"
14
15 raw_df = (
16     spark.read
17     .option("header", "true")
18     .schema(raw_schema)
19     .csv(raw_path)
20 )
21
22 bronze_with_metadata_df = (
23     raw_df
24     .withColumn("ingested_at", F.current_timestamp())
25     .withColumn("source_file", F.input_file_name())
26 )
27
28 required_columns = ["event_id", "device_id", "event_timestamp"]
29
30 silver_df = (
31     bronze_with_metadata_df
32     .dropDuplicates(["event_id"])
33     .dropna(subset=required_columns)
34     .select(
35         F.col("event_id").cast("string").alias("event_id"),
36         F.col("device_id").cast("string").alias("device_id"),
37         F.to_timestamp("event_timestamp").alias("event_timestamp"),
38         F.to_date("event_timestamp").alias("event_date"),
39         F.col("temperature_c").cast("double").alias("temperature_c"),
40         F.upper(F.col("status")).alias("status"),
41         F.col("source_system").cast("string").alias("source_system"),
42         "ingested_at",
43         "source_file",
44     )
45     .where(F.col("event_timestamp").isNotNull())
46 )
47
48 silver_df.write.format("delta").mode("overwrite").saveAsTable(
49     "silver_telemetry_device_events"
50 )

```

Listing 3.1: PySpark Bronze CSV to cleaned Silver Delta table.

The second listing registers the Silver DataFrame as a temporary view and uses Spark SQL

from Python. This is useful when a notebook cell needs to calculate validation metrics or feed a SQL-first aggregate without leaving the PySpark session.

```

1 silver_df.createOrReplaceTempView("v_silver_telemetry_device_events")
2
3 quality_summary_df = spark.sql("""
4 SELECT
5     COUNT(*) AS accepted_rows,
6     COUNT(DISTINCT event_id) AS distinct_events,
7     SUM(CASE WHEN temperature_c IS NULL THEN 1 ELSE 0 END) AS
8     missing_temperature_rows,
9     MIN(event_timestamp) AS first_event_timestamp,
10    MAX(event_timestamp) AS last_event_timestamp
11 FROM v_silver_telemetry_device_events
12 """)
13 quality_summary_df.show(truncate=False)
14
15 daily_status_df = spark.sql("""
16 SELECT
17     event_date,
18     status,
19     COUNT(*) AS event_count,
20     AVG(temperature_c) AS average_temperature_c
21 FROM v_silver_telemetry_device_events
22 GROUP BY event_date, status
23 ORDER BY event_date, status
24 """)
25
26 daily_status_df.write.format("delta").mode("overwrite").saveAsTable(
27     "silver_telemetry_daily_status_summary"
28 )

```

Listing 3.2: Mixing Spark SQL with PySpark in the same notebook.

### 3.5.3 Validation Checklist

- The notebook uses an explicit schema instead of relying on CSV inference.
- Duplicate `event_id` values are removed before the Silver table is written.
- Records with null `event_id`, `device_id`, or `event_timestamp` are excluded from Silver.
- The Silver table is written in Delta format and appears in the lakehouse table list.
- A Spark SQL query validates row counts, distinct keys, and basic time range.
- No secret values, personal workspace identifiers, or real device identifiers are hardcoded in the notebook.

## 3.6 Friday: Use Case and Architecture

A telemetry platform receives approximately 100 GB of JSON logs per day from applications, devices, gateways, and operational services. The raw stream is valuable for debugging and audit, but analytics teams need reliable Silver Delta tables with typed timestamps, normalized severities, required identifiers, and duplicate handling. Spark Notebooks are a strong fit because JSON parsing, nested fields, and large daily files benefit from distributed processing.

### 3.6.1 Architecture Strategy

The design lands raw JSON in a Bronze Lakehouse under date-partitioned folders, then runs a scheduled Spark Notebook that reads one processing date, applies an explicit schema, drops malformed or incomplete records, deduplicates event identifiers, and writes Silver Delta partitioned by event date. The pool should be sized from observed throughput, not guessed permanently. Start with a Custom pool for predictable daily processing, measure duration, then tune file sizes, shuffle partitions, and schedule windows.

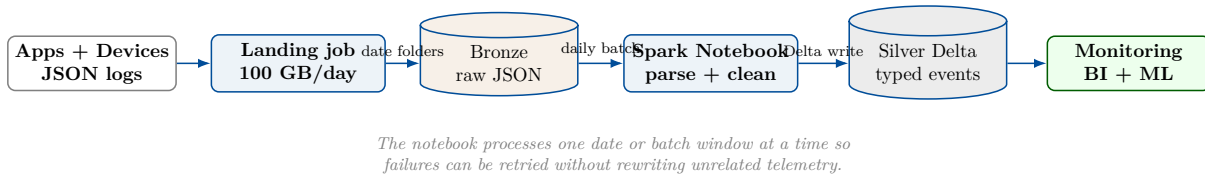


Figure 3.2: Spark notebook ingestion pipeline for 100 GB per day of telemetry JSON.

### 3.6.2 Design Decisions

Date folders make incremental processing observable. Explicit JSON schemas reduce expensive inference and prevent schema drift from surprising downstream tables. Silver Delta tables provide durable, queryable outputs for BI, alerting, and machine learning features. Quality metrics should be written after every run so operators know how many raw records were accepted, rejected, duplicated, or malformed.

| Concern      | Decision                                           | Reason                                                   |
|--------------|----------------------------------------------------|----------------------------------------------------------|
| Daily volume | Process by date or batch window                    | Limits retry scope and supports predictable scheduling   |
| JSON parsing | Use explicit schema with a corrupt-record strategy | Avoids inference cost and captures malformed data        |
| Compute      | Use a Custom pool after prototype validation       | Provides repeatable sizing and autoscale boundaries      |
| Storage      | Write Silver as Delta partitioned by event date    | Improves query pruning and table reliability             |
| Operations   | Record quality metrics per run                     | Makes acceptance, rejection, and duplicate rates visible |

Table 3.3: Design decisions for a 100 GB/day telemetry ingestion layer.

## 3.7 Operational Guidance for Week 3

Spark operations should focus on repeatability and evidence. Every scheduled notebook should identify its processing window, input path, output table, pool, environment, and quality metrics. A failed daily load should be retryable without guessing which files were read or which table partitions were overwritten.

### 3.7.1 Performance and Reliability

Avoid using `inferSchema` for large CSV or JSON jobs. Avoid tiny files when possible because thousands of small files can create scheduling overhead. Prefer explicit schemas, partition-aware reads, and deterministic output paths. Monitor duration, executor utilization, data skew, and failed tasks before increasing pool size.

**Tip**

Tune with measurements. Capture input bytes, input file count, accepted rows, rejected rows, duplicate count, write duration, and pool used. These numbers make capacity conversations factual.

### 3.7.2 Security Checklist

- Store raw telemetry in Bronze with least-privilege workspace access.
- Remove tokens, secrets, request headers, and unnecessary payload text before Silver publication.
- Do not print full raw JSON records in notebook output.
- Pin required libraries in a Spark environment rather than installing packages interactively in production cells.
- Use parameterized processing dates and avoid hardcoded personal paths.
- Keep Silver Delta tables documented with owner, refresh cadence, schema, and quality expectations.

## 3.8 Interview Q&A

1. **Q: What is the difference between a Starter pool and a Custom pool in Fabric Spark?**

A: A Starter pool is optimized for quick, managed notebook startup with minimal configuration. A Custom pool gives teams explicit control over sizing and scale settings for repeatable workloads.

2. **Q: Why are Spark environments important?**

A: They make notebooks repeatable by pinning runtime choices, libraries, and Spark properties instead of relying on interactive installs or hidden session state.

3. **Q: When should you avoid schema inference?**

A: Avoid it for large or scheduled jobs because it adds scan cost and can let unexpected files change inferred column types.

4. **Q: How can PySpark and Spark SQL work together?**

A: PySpark can create a DataFrame and register it as a temporary view; Spark SQL can then query that view in the same session.

5. **Q: Why write Silver outputs as Delta tables?**

A: Delta tables provide transactional metadata, consistent reads, schema management options, and strong integration with Fabric lakehouses.

6. **Q: What is the safest retry unit for a daily telemetry pipeline?**

A: A specific processing date or batch window, because the job can reread the same Bronze files and replace only the corresponding Silver output scope.

## 3.9 Further Reading

Review Microsoft documentation for Fabric Spark compute [7], Fabric Notebooks [12], Spark environments [8], Fabric Lakehouses [20], and OneLake [22]. For distributed execution and table

reliability, read the Apache Spark overview [2], Delta Lake documentation [6], and the Delta Lake paper [3].

## Key Takeaways

- Fabric Spark provides managed distributed processing through notebook sessions that run on Starter or Custom pools.
- Starter pools are convenient for learning and exploration; Custom pools are better for repeatable production workloads.
- Spark environments keep notebooks consistent by controlling runtime, dependencies, and Spark properties.
- PySpark DataFrames are lazily evaluated, so errors often appear when an action reads, displays, or writes data.
- Fabric notebooks can combine PySpark and Spark SQL through temporary views and shared Spark sessions.
- Bronze-to-Silver Spark jobs should use explicit schemas, deterministic duplicate rules, null handling, and Delta writes.
- A 100 GB/day telemetry ingestion layer should process bounded windows, record quality metrics, and tune from observed performance.

## 3.10 Exercises

1. **(Conceptual)** Explain the driver and executor roles in a Spark application running on a Fabric pool.
2. **(Conceptual)** Compare Starter pools and Custom pools for a training notebook, a scheduled daily job, and an urgent debugging notebook.
3. **(Design)** Define a Spark environment for a telemetry ingestion team. Include runtime, approved libraries, and one Spark property you would discuss before changing.
4. **(Hands-on)** Create a Fabric notebook that reads a CSV from `Files/bronze/raw_csv/` and displays the schema without writing a table.
5. **(Hands-on)** Modify listing 3.1 to write rejected rows with missing required fields to a quarantine Delta table.
6. **(SQL)** Register a DataFrame as a temporary view and write a Spark SQL query that counts records by processing date and status.
7. **(Operational)** Draft a runbook for rerunning a failed daily telemetry batch without corrupting other dates.
8. **(Interview)** In five sentences, explain why explicit schemas matter for 100 GB/day JSON processing.

### 3.11 Capstone Project: Spark-Based Telemetry Ingestion at 100 GB per Day

#### Capstone Project

**Mission.** Build a Spark-based telemetry ingestion pipeline in Microsoft Fabric. The pipeline must read raw JSON logs from a Bronze Lakehouse, clean and deduplicate the records, drop invalid null-key rows, and write typed Silver Delta tables that can support a 100 GB/day operating target.

**Estimated time.** 4–6 hours, depending on tenant permissions, available sample data, and whether the class simulates 100 GB with smaller generated files.

**What you'll practice.**

- Designing a bounded daily Spark processing window.
- Reading nested JSON from OneLake with an explicit schema.
- Applying duplicate and null handling before Silver publication.
- Writing partitioned Delta tables for queryable telemetry.
- Recording run metrics that prove operational readiness.

#### Scenario

A software company operates web applications, device gateways, and background services. Each component emits JSON telemetry containing event identifiers, device identifiers, timestamps, severity, service name, region, latency, and optional diagnostic payloads. The platform lands about 100 GB per day in Bronze. Engineering, reliability, BI, and machine learning teams need a cleaned Silver table for trend analysis, incident review, anomaly detection, and service-level reporting.

Your job is to build the first production-shaped notebook. It does not need to ingest a real 100 GB during training, but the design must show how it would process one daily folder, validate records, remove duplicates, and write partitioned Delta output without exposing raw payloads unnecessarily.

#### Requirements

Build a working Fabric design with these minimum elements:

- A workspace named `telemetry-de-dev` with a Bronze lakehouse `lh_telemetry_bronze` and a Silver lakehouse `lh_telemetry_silver`, or one training lakehouse with clearly separated layer paths.
- A raw JSON path such as `Files/bronze/raw_json/telemetry/event_date=2026-07-31/`.
- A notebook named `nb_telemetry_bronze_to_silver` that accepts a processing date variable.
- An explicit JSON schema with required fields `event_id`, `device_id`, and `event_timestamp`.
- Duplicate removal by `event_id` and null dropping for required keys.
- A Silver Delta table named `silver_telemetry_events` partitioned by `event_date`.
- A run metrics table named `silver_telemetry_ingestion_metrics`.
- No secrets, tokens, full request headers, or real customer identifiers in notebook output or deliverables.

## Reference Architecture

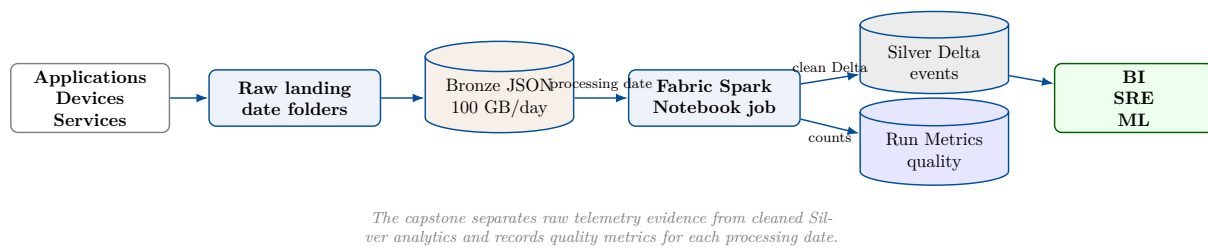


Figure 3.3: Reference architecture for the Spark telemetry ingestion capstone.

## Implementation Steps

1. **Create lakehouses.** Create `lh_telemetry_bronze` and `lh_telemetry_silver`. If your training tenant supports one lakehouse only, create paths and table prefixes that preserve the same boundary.
2. **Prepare raw JSON.** Upload instructor-provided synthetic telemetry JSON to `Files/bronze/raw_json`. Use smaller files for training if needed.
3. **Configure compute.** Start with a Starter pool for the first run. Document the Custom pool settings you would use for scheduled 100 GB/day processing.
4. **Create an environment.** Pin approved runtime and library choices in a Spark environment when your tenant allows it.
5. **Run Bronze to Silver.** Execute listing 3.3 for one processing date.
6. **Record metrics.** Execute listing 3.4 to write run-level quality metrics.
7. **Validate output.** Query the Silver table by `event_date`, `severity`, and `service_name`.
8. **Document operations.** Describe retry behavior, expected schedule, pool choice, and how rejected records would be investigated.

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.types import (
3     StructType, StructField, StringType, DoubleType, LongType
4 )
5
6 processing_date = "2026-07-31"
7 raw_path = f"Files/bronze/raw_json/telemetry/event_date={processing_date}/*.
8     json"
9
10 telemetry_schema = StructType([
11     StructField("event_id", StringType(), True),
12     StructField("device_id", StringType(), True),
13     StructField("event_timestamp", StringType(), True),
14     StructField("severity", StringType(), True),
15     StructField("service_name", StringType(), True),
16     StructField("region", StringType(), True),
17     StructField("latency_ms", DoubleType(), True),
18     StructField("bytes_in", LongType(), True),
19     StructField("bytes_out", LongType(), True),
20     StructField("payload", StringType(), True),
21 ])
22 raw_json_df = spark.read.schema(telemetry_schema).json(raw_path)

```

```

23 raw_count = raw_json_df.count()
24
25 accepted_df = (
26     raw_json_df
27     .withColumn("ingested_at", F.current_timestamp())
28     .withColumn("source_file", F.input_file_name())
29     .dropDuplicates(["event_id"])
30     .dropna(subset=["event_id", "device_id", "event_timestamp"])
31     .select(
32         F.col("event_id").cast("string").alias("event_id"),
33         F.sha2(F.col("device_id").cast("string"), 256).alias("device_hash"),
34         F.to_timestamp("event_timestamp").alias("event_timestamp"),
35         F.to_date("event_timestamp").alias("event_date"),
36         F.upper(F.col("severity")).alias("severity"),
37         F.col("service_name").cast("string").alias("service_name"),
38         F.col("region").cast("string").alias("region"),
39         F.col("latency_ms").cast("double").alias("latency_ms"),
40         F.col("bytes_in").cast("long").alias("bytes_in"),
41         F.col("bytes_out").cast("long").alias("bytes_out"),
42         "ingested_at",
43         "source_file",
44     )
45     .where(F.col("event_timestamp").isNotNull())
46 )
47
48 accepted_df.write.format("delta").mode("overwrite").partitionBy("event_date").
49     saveAsTable(
50     "silver_telemetry_events"
51 )

```

Listing 3.3: Capstone Bronze JSON to cleaned Silver Delta transformation.

```

1 accepted_df.createOrReplaceTempView("v_accepted_telemetry")
2 accepted_count = accepted_df.count()
3
4 duplicate_count = raw_count - raw_json_df.dropDuplicates(["event_id"]).count()
5 required_non_null_count = (
6     raw_json_df
7     .dropDuplicates(["event_id"])
8     .dropna(subset=["event_id", "device_id", "event_timestamp"])
9     .count()
10 )
11 rejected_count = raw_count - accepted_count
12
13 metrics_rows = [(
14     processing_date,
15     raw_count,
16     accepted_count,
17     rejected_count,
18     duplicate_count,
19     required_non_null_count,
20 )]
21 metrics_columns = [
22     "processing_date",
23     "raw_rows",
24     "accepted_rows",
25     "rejected_rows",
26     "duplicate_rows",
27     "required_non_null_rows",
28 ]
29
30 metrics_df = spark.createDataFrame(metrics_rows, metrics_columns).withColumn(
31     "metrics_recorded_at", F.current_timestamp()
32 )

```



```

33
34 metrics_df.write.format("delta").mode("append").saveAsTable(
35     "silver_telemetry_ingestion_metrics"
36 )
37
38 spark.sql("""
39 SELECT
40     event_date,
41     severity,
42     service_name,
43     COUNT(*) AS event_count,
44     AVG(latency_ms) AS average_latency_ms,
45     SUM(bytes_in + bytes_out) AS total_bytes
46 FROM v_accepted_telemetry
47 GROUP BY event_date, severity, service_name
48 ORDER BY event_date, severity, service_name
49 """).show(100, truncate=False)

```

Listing 3.4: Capstone run metrics and Spark SQL validation.

## Deliverables

Submit evidence that the architecture and implementation are complete:

- Workspace, lakehouse, notebook, pool, and environment names.
- Screenshot or exported evidence of the Bronze JSON folder and Silver Delta tables.
- Notebook output showing raw, accepted, rejected, duplicate, and required non-null counts.
- Schema for `silver_telemetry_events`, including hashed device identifiers and removed raw payload fields.
- Description of partitioning strategy and retry unit for daily processing.
- A short operational note explaining how the design scales from training files to 100 GB/day.
- Confirmation that no secrets, request headers, tokens, or real customer identifiers were printed or submitted.

## Acceptance Criteria

Use this rubric before declaring the capstone complete:

- ☐ Raw JSON files are stored in a Bronze path by processing date.
- ☐ The notebook reads only the requested processing date or batch window.
- ☐ The JSON reader uses an explicit schema.
- ☐ Silver records remove duplicate `event_id` values.
- ☐ Records missing `event_id`, `device_id`, or `event_timestamp` are excluded or quarantined.
- ☐ The Silver Delta table is partitioned by `event_date`.
- ☐ Run metrics capture raw, accepted, rejected, and duplicate counts.
- ☐ Raw payloads, tokens, and full diagnostic text are not exposed in Silver outputs.
- ☐ The design states when a Custom pool would replace a Starter pool.
- ☐ Retry behavior is bounded to one processing date or batch window.

## Stretch Goals

If you finish early, extend the capstone while preserving the Bronze-to-Silver contract:

- Add a quarantine Delta table that stores rejected records and rejection reasons.
- Add schema evolution handling for a new optional field such as `app_version`.
- Add a small Gold aggregate for daily service health by region and severity.
- Compare Starter pool and Custom pool execution time on the same sample data.
- Add a validation that fails the notebook when rejected records exceed an agreed threshold.
- Document a deployment path from development to production workspaces.

# Delta Lake Optimization

## Learning Objectives

- Explain how a Delta table combines Parquet data files with a JSON transaction log.
- Describe how Fabric optimization features such as V-Order, OPTIMIZE, and Z-Ordering improve analytical reads.
- Manage Delta table history, retention, and schema changes with Time Travel, VACUUM, and schema evolution.
- Run OPTIMIZE with V-Order on a large Silver table and use Time Travel after an accidental deletion.
- Diagnose the small-file problem and design a remediation plan for streaming tables degraded over months.
- Integrate Weeks 1–4 by building a Spark-driven Bronze, Silver, and Gold lakehouse with OneLake shortcuts and optimized Delta tables.

## Cross-Cloud Equivalence

Open table formats solve a shared problem: many cloud object stores are excellent for durable files, but analytical tables also need transactions, metadata, compaction, clustering, retention, and historical access. Fabric centers this experience on Delta tables in OneLake, while other platforms expose similar ideas through Delta, Iceberg, Hudi, managed clustering, or compaction services.

| Optimization need                  | Fabric Delta                                                                                                                     | AWS                                                                                                      | Azure/Databricks                                                                                                   | Snowflake                                                                                                                  |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Compaction and small-file handling | <code>OPTIMIZE</code> rewrites many small Delta files into fewer larger files; V-Order can be enabled for Fabric read efficiency | Delta, Iceberg, or Hudi tables on S3 use compaction through EMR, Glue, Athena, or table-maintenance jobs | Delta <code>OPTIMIZE</code> on Databricks and Synapse/Fabric-style lakehouse patterns compact object-storage files | Micro-partitions are managed by the service; clustering maintenance is automatic or configured through clustering features |
| Data skipping and clustering       | Z-Ordering colocates related values; Delta statistics and V-Order improve selective scans                                        | Iceberg partition specs, Hudi clustering, Delta Z-Order, and Glue catalog metadata help prune S3 reads   | Databricks <code>OPTIMIZE ZORDER BY</code> and data-skipping statistics are common tuning levers                   | Automatic clustering and pruning reduce scanned micro-partitions                                                           |
| Time travel                        | Delta history supports <code>VERSION AS OF</code> and timestamp-based reads before retention cleanup                             | Delta, Iceberg, and Hudi support snapshot or commit-based reads when retained                            | Delta Time Travel is a core Databricks and Delta Lake capability                                                   | Built-in Time Travel exposes previous table states within the configured retention window                                  |
| Vacuum and retention               | <code>VACUUM</code> removes old unreferenced files after the safety retention period                                             | Table formats require expire-snapshot, clean, or compaction jobs to remove obsolete files from S3        | Delta <code>VACUUM</code> and retention settings control cleanup after history is no longer needed                 | Fail-safe and Time Travel retention are managed service features rather than user file deletion                            |

GCP commonly expresses the same pattern through BigLake, BigQuery external tables, Dataproc, and Iceberg or Delta tables on Cloud Storage. MongoDB is not an open-table-format lakehouse engine; it typically feeds optimized Delta, Iceberg, or warehouse tables through connectors, change streams, or batch exports.

### Key Terms

**Delta table:** a table abstraction that stores data as Parquet files and records table actions in a transaction log. **Transaction log:** ordered metadata files that describe adds, removes, schema changes, and commits. **V-Order:** Fabric’s write-time layout optimization for faster analytical reads. **Z-Ordering:** a clustering technique that colocates related values to improve data skipping. **Small-file problem:** degraded performance caused by many tiny files that create excessive listing, scheduling, and metadata overhead.

## 4.1 Week 4 Roadmap

Week 4 closes Part I by turning lakehouse storage into an operational discipline. Monday explains Delta internals: Parquet data files plus the JSON transaction log. Tuesday covers table optimization with V-Order, `OPTIMIZE`, and Z-Ordering. Wednesday covers table management with `VACUUM`, Time Travel, and schema evolution. Thursday is a hands-on recovery and optimization lab. Friday applies the concepts to a streaming table that accumulated millions of small files over six months.

Delta Lake matters because a lakehouse is only useful when tables remain reliable and fast as data changes. Fabric Lakehouses store managed tables in Delta format, which means Spark, SQL analytics endpoints, Direct Lake semantic models, and other engines can reason about consistent table versions rather than loose folders of files [20, 6]. Optimization does not replace good modeling, but it keeps Silver and Gold tables usable when ingestion patterns, streaming frequency, and analytical filters evolve.

**Note**

This chapter focuses on data-engineering operations, not one-time performance tricks. A healthy Delta table has a clear owner, expected write pattern, retention policy, optimization cadence, recovery plan, and evidence that reads improved after maintenance.

## 4.2 Monday: Delta Lake Internals

A Delta table is not a single file. It is a directory containing Parquet data files plus a special `_delta_log` folder. The Parquet files store columnar data. The transaction log stores ordered JSON actions that describe which files belong to each table version, which files were removed, what schema is active, and what operation produced the commit [3, 1].

**Definition**

**Delta Lake** is an open table format that adds ACID transactions, scalable metadata, schema enforcement, schema evolution, and Time Travel over Parquet files in object storage or lakehouse storage.

### 4.2.1 Parquet Data Files

Parquet is a columnar file format. Values for the same column are stored together, which improves compression and lets analytical engines read only the columns required by a query. For example, a query that groups sales by `order_date` and `region` does not need to scan every descriptive column in the table.

Parquet files are immutable from the practical perspective of table writes. Updating, deleting, or compacting records usually creates new files and marks old files as no longer active in the table log. This immutability is why the transaction log is essential: it tells readers which Parquet files make up the current table state.

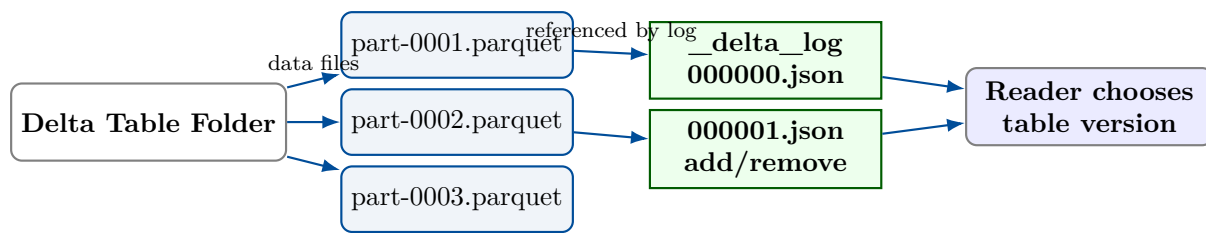
**Tip**

Think of Parquet files as efficient pages and the Delta log as the table of contents plus audit trail. Query engines need both to produce a consistent result.

### 4.2.2 The JSON Transaction Log

Every Delta commit creates a new version. Early versions are commonly represented as JSON log files under `_delta_log`; checkpoint files may later summarize metadata for efficient reads. A commit can add files, remove files, change metadata, update protocol requirements, or record operation metrics.

Because the log is ordered, a reader can reconstruct version 0, version 1, version 2, and so on. This is the foundation for ACID behavior and Time Travel. When one job writes a new Silver table version, concurrent readers can continue using a consistent previous version until they refresh.



*Delta readers combine immutable Parquet files with ordered log actions to produce the correct table snapshot.*

Figure 4.1: Delta table internals: Parquet data files plus a transaction log.

### 4.2.3 Why Internals Affect Optimization

Optimization rewrites files, but it should not change the logical table result. Compaction creates new larger Parquet files and records removal actions for the older smaller files. Z-Ordering rewrites data into a layout that improves skipping for common filters. Time Travel remains possible because previous versions still reference older files until retention cleanup removes them.

#### Common Pitfall

**Deleting files manually.** Removing Parquet files outside Delta operations can corrupt historical versions and break readers. Use Delta commands and retention-aware maintenance instead of manual file deletion.

## 4.3 Tuesday: Optimizing Delta Tables

Optimization starts with the workload. A table read mostly by date, customer, and product needs a different layout than a table read by device, event type, and timestamp. Fabric supports Delta table optimization patterns that reduce file overhead and improve query pruning for common analytics [9, 18].

### 4.3.1 V-Order in Fabric

V-Order is a Fabric optimization for Parquet files that improves compression and read performance for Fabric analytical engines. It is especially relevant when a Delta table supports SQL analytics endpoint queries, Direct Lake semantic models, and repeated reporting scans. V-Order is not a substitute for medallion modeling; it makes well-designed tables more efficient to read.

In Fabric, V-Order can be applied through table write settings or optimization commands depending on the workload and runtime support. The important operational habit is to document whether critical Silver and Gold tables are expected to be V-Order optimized and when the optimization runs.

#### Note

V-Order is a physical layout decision. It should not change business results, table names, schemas, or downstream contracts.

### 4.3.2 OPTIMIZE for Compaction

**OPTIMIZE** rewrites many small files into fewer larger files. This reduces file-listing overhead, Spark task scheduling overhead, and metadata pressure. It is most valuable after frequent appends, streaming micro-batches, or partitioned writes that produce many tiny files.

Compaction has a cost because it reads and rewrites data. Schedule it according to the table's write pattern and consumption needs. A table loaded once nightly may need optimization after each load. A streaming table may need scheduled maintenance during a low-traffic window.

#### Tip

Measure file count before and after compaction. A successful optimization should reduce active file count, improve scan time for representative queries, and preserve row counts.

### 4.3.3 Z-Ordering

Z-Ordering colocates records with similar values across one or more columns so that filters can skip more files. It is useful when queries often filter on high-value columns such as `customer_id`, `device_id`, `order_date`, or `region`. Do not Z-Order every column. Choose columns from real query patterns and avoid changing layout constantly.

| Technique       | Best fit                                                                  | Watch for                                                    |
|-----------------|---------------------------------------------------------------------------|--------------------------------------------------------------|
| V-Order         | Fabric analytical reads over Delta-Parquet tables                         | Requires write or optimize support and clear table policy    |
| <b>OPTIMIZE</b> | Tables with many small files from streaming or frequent appends           | Rewrite cost and maintenance scheduling                      |
| Z-Ordering      | Selective queries that repeatedly filter by known columns                 | Poor column choice can waste compute without improving reads |
| Partitioning    | Large tables commonly filtered by low-cardinality date or business period | Over-partitioning can create the small-file problem          |

Table 4.1: Optimization choices for Delta tables in Fabric.

## 4.4 Wednesday: Managing Delta Tables

Optimization makes reads faster; management keeps tables recoverable and sustainable. The three core management topics are retention cleanup, historical reads, and safe schema changes.

### 4.4.1 VACUUM and Retention

**VACUUM** deletes data files that are no longer referenced by the current table and are older than the retention threshold [19]. Without cleanup, old files can accumulate indefinitely after overwrites, deletes, merges, and compaction. With aggressive cleanup, Time Travel and rollback options can disappear too quickly.

The retention window should reflect business requirements. A development table may keep less history. A regulated Silver table may require longer retention and separate archival controls. In any case, record the policy before automating cleanup.

**Common Pitfall**

**Vacuuming too soon after a mistake.** If an accidental delete is discovered after obsolete files are vacuumed, Time Travel may no longer be able to read the pre-delete version.

#### 4.4.2 Time Travel

Time Travel lets readers query a previous version or timestamp of a Delta table [6]. It is useful for debugging, audit comparison, reproducible experiments, and recovery after a bad write. A data engineer can compare row counts between versions, recover deleted records, or prove when a schema changed.

Time Travel depends on retained log and data files. It should be part of an incident playbook: identify the bad version, query the last known good version, validate it, and restore or rewrite only after approval.

#### 4.4.3 Schema Evolution

Schema enforcement protects tables from unexpected columns and type changes. Schema evolution allows intentional changes, such as adding a nullable column when a source system begins sending `promotion_code`. The difference is governance: unplanned drift should fail or be quarantined; planned evolution should be reviewed, documented, and tested.

**Definition**

**Schema evolution** is the controlled process of changing a Delta table schema, commonly by adding nullable columns, while preserving existing data and downstream compatibility.

### 4.5 Thursday: Hands-on Project

The Week 4 lab uses a large Silver table such as `silver_sales_orders`. You will compact it with `OPTIMIZE` and V-Order, simulate an accidental deletion in a safe training copy, and query the previous version with Time Travel. Use synthetic or instructor-provided data; never run delete experiments on a production table.

#### 4.5.1 Goal and Architecture

Create or reuse a workspace named `fabric-de-week04`. Add a lakehouse named `lh_silver`. Prepare a Delta table named `silver_sales_orders` with enough rows and files to make optimization visible. The table should include columns such as `order_id`, `customer_id`, `order_date`, `region`, `product_id`, and `net_amount`.

**Note**

If your tenant does not support every optimization syntax shown here, keep the learning outcome: compact the Delta table, record file-count and query-time evidence, and demonstrate a versioned read before cleanup.

#### 4.5.2 Step-by-Step Actions

1. Create a training Silver Delta table with realistic columns and many files.
2. Record a baseline row count, active file count if available, and query duration for a representative filter.



3. Run OPTIMIZE with V-Order and optional Z-Ordering for the common query columns.
4. Query Delta history and note the latest version number.
5. In a training copy, simulate an accidental delete for one region or date range.
6. Use VERSION AS OF to read the previous version and validate the missing rows.
7. Restore the affected rows from the previous version or rebuild the training table.
8. Run VACUUM only after confirming the retention policy and recovery needs.

```

1 -- Capture a baseline for representative analytical filters.
2 SELECT region, COUNT(*) AS order_count, SUM(net_amount) AS total_net_amount
3 FROM silver_sales_orders
4 WHERE order_date >= DATE '2026-07-01'
5 GROUP BY region
6 ORDER BY region;
7
8 -- Compact files and apply Fabric V-Order where supported.
9 OPTIMIZE silver_sales_orders VORDER;
10
11 -- Add clustering for common selective filters when supported by the runtime.
12 OPTIMIZE silver_sales_orders
13 ZORDER BY (customer_id, order_date);
14
15 DESCRIBE HISTORY silver_sales_orders;

```

Listing 4.1: Optimizing a Silver Delta table with V-Order and Z-Ordering.

After optimization, rerun the same baseline query. The row count and business totals must match. The expected improvement is fewer active files and faster or more predictable reads, not changed data.

```

1 -- Simulate a mistake only in a training copy.
2 CREATE TABLE silver_sales_orders_training AS
3 SELECT * FROM silver_sales_orders;
4
5 DELETE FROM silver_sales_orders_training
6 WHERE region = 'West'
7 AND order_date >= DATE '2026-07-01';
8
9 DESCRIBE HISTORY silver_sales_orders_training;
10
11 -- Replace 12 with the last known good version from DESCRIBE HISTORY.
12 SELECT COUNT(*) AS west_orders_before_delete
13 FROM silver_sales_orders_training VERSION AS OF 12
14 WHERE region = 'West'
15 AND order_date >= DATE '2026-07-01';

```

Listing 4.2: Using Time Travel after an accidental training-table deletion.

Some recovery workflows are easier in PySpark because you can read the older version into a DataFrame, filter the missing slice, and append it back after validation.

```

1 from pyspark.sql import functions as F
2
3 table_name = "silver_sales_orders_training"
4 last_good_version = 12
5
6 previous_df = (
7     spark.read
8     .format("delta")
9     .option("versionAsOf", last_good_version)
10    .table(table_name)

```

```

11 )
12
13 missing_west_df = (
14     previous_df
15     .where((F.col("region") == "West") & (F.col("order_date") >= F.lit("
2026-07-01"))))
16 )
17
18 missing_west_df.createOrReplaceTempView("v_recovered_west_orders")
19
20 spark.sql("""
21 SELECT COUNT(*) AS rows_to_restore, SUM(net_amount) AS amount_to_restore
22 FROM v_recovered_west_orders
23 """).show(truncate=False)

```

Listing 4.3: Reading a prior Delta version in PySpark for targeted recovery.

```

1 -- Run only after recovery is complete and retention requirements are satisfied
2 VACUUM silver_sales_orders_training RETAIN 168 HOURS;

```

Listing 4.4: Retention-aware cleanup after recovery validation.

### 4.5.3 Validation Checklist

- Baseline and post-optimization row counts match.
- Representative query filters are documented before choosing Z-Order columns.
- DESCRIBE HISTORY shows optimization and delete operations.
- Time Travel reads the last known good version before retention cleanup.
- Recovery logic is tested on a training copy, not a production table.
- VACUUM is delayed until the recovery window and retention policy are understood.

## 4.6 Friday: Use Case and Architecture

A streaming pipeline has appended micro-batches to a Silver events table for six months. Each micro-batch wrote many tiny files into date partitions. Analysts now report that simple filters take minutes, capacity consumption spikes during business hours, and Direct Lake reports refresh unpredictably. The table contains correct data, but the physical layout has degraded.

### 4.6.1 Symptoms

The small-file problem shows up in several ways. Spark jobs spend too much time listing files and scheduling tasks. SQL queries scan many file footers before reading useful rows. Metadata operations slow down. Optimization attempts may compete with streaming writes. Downstream users experience unstable latency even when daily row volume has not changed.

### 4.6.2 Remediation Strategy

The remediation plan has two parts: fix the existing table and prevent recurrence. First, pause or coordinate streaming writes, snapshot table history, and run partition-aware compaction for the worst partitions. Second, adjust streaming trigger intervals, batch sizing, partition strategy, and scheduled OPTIMIZE cadence so new tiny files do not accumulate unchecked.

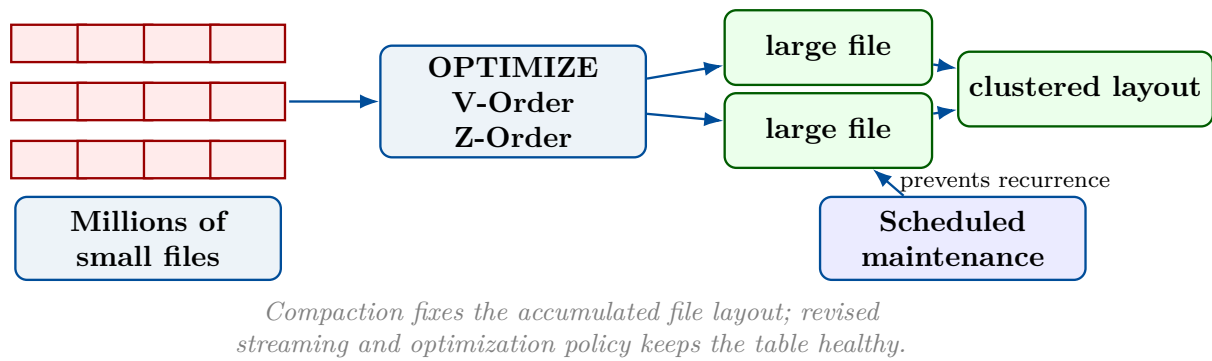


Figure 4.2: Small-file remediation through compaction and optimized layout.

### 4.6.3 Operational Runbook

1. Identify affected tables, partitions, file counts, average file sizes, and top user queries.
2. Confirm retention requirements and capture the current Delta version.
3. Coordinate a maintenance window or use an approach compatible with active streaming writes.
4. Compact the oldest and worst partitions first; validate row counts after each batch.
5. Apply V-Order and targeted Z-Ordering for documented query filters.
6. Tune streaming triggers or batch sizes so future writes produce healthier files.
7. Schedule recurring optimization and record metrics after every maintenance run.
8. Delay `VACUUM` until rollback confidence and retention requirements are satisfied.

| Evidence                           | Interpretation                                       | Action                                                                   |
|------------------------------------|------------------------------------------------------|--------------------------------------------------------------------------|
| Millions of active files           | Metadata and scheduling overhead dominate reads      | Run partition-aware <code>OPTIMIZE</code> and prevent tiny future writes |
| Queries filter by date and device  | Layout should support data skipping on those columns | Consider date partitioning plus Z-Order on <code>device_id</code>        |
| Streaming writes every few seconds | Micro-batches may create too many files              | Increase trigger interval or use available output coalescing patterns    |
| History needed for audit           | Cleanup cannot be aggressive                         | Align <code>VACUUM</code> retention with audit and restore policy        |

Table 4.2: Diagnosing and remediating a degraded streaming Delta table.

## 4.7 Operational Guidance for Week 4

Delta operations should be automated only after they are understood manually. The first maintenance run should capture baseline query time, row counts, active file counts, operation history, and table version. The automated version should emit the same evidence to a run log.

### 4.7.1 Performance and Reliability

Avoid optimizing blindly. Optimize high-value tables first: large Silver tables, Gold dimensions and facts used by Direct Lake, and tables with frequent appends. Keep Z-Order columns stable

unless query patterns materially change. Coordinate compaction with streaming writers and avoid running expensive maintenance during peak reporting windows.

### 4.7.2 Security and Governance Checklist

- Restrict who can run destructive commands such as `DELETE`, `TRUNCATE`, and `VACUUM`.
- Record table owners, retention policies, optimization cadence, and recovery contacts.
- Do not shorten retention without understanding legal, audit, and incident-response needs.
- Validate schema evolution in development before allowing new columns into shared Silver or Gold tables.
- Keep optimization notebooks parameterized and free of personal workspace paths.

## 4.8 Interview Q&A

1. **Q: What are the two major physical components of a Delta table?**  
A: Parquet data files store the columnar data, and the Delta transaction log records table versions, add and remove actions, schema metadata, and operation history.
2. **Q: Why does `OPTIMIZE` help a table with many small files?**  
A: It compacts many small files into fewer larger files, reducing file-listing overhead, task scheduling overhead, and metadata pressure during reads.
3. **Q: When would you use Z-Ordering?**  
A: Use it when important queries repeatedly filter on specific columns and colocating similar values will improve data skipping.
4. **Q: How is V-Order different from Z-Ordering?**  
A: V-Order is a Fabric-oriented Parquet layout optimization for analytical reads, while Z-Ordering clusters rows by selected columns to improve selective filtering.
5. **Q: What risk does `VACUUM` introduce?**  
A: It can permanently remove old unreferenced files, which may prevent Time Travel or recovery to versions that still need those files.
6. **Q: How should schema evolution be governed?**  
A: Treat it as an intentional contract change: review the new field, test downstream impact, document the schema, and avoid silently accepting unplanned drift.

## 4.9 Further Reading

Review Microsoft documentation for Fabric Lakehouses [20], OneLake [22], Fabric Spark [7], V-Order [9], Delta table maintenance and optimization [18, 19], and Fabric notebooks [12]. For format internals, read the Delta Lake documentation [6], the Delta Lake paper [3], and Apache Parquet documentation [1].

## Key Takeaways

- Delta tables combine Parquet data files with an ordered transaction log that defines consistent table versions.
- Fabric optimization is a physical layout practice: V-Order, `OPTIMIZE`, and Z-Ordering should improve reads without changing results.

- The small-file problem is common after frequent appends or streaming micro-batches and should be fixed with compaction plus better write policy.
- Time Travel is a recovery and debugging feature, but it depends on retained log and data files.
- `VACUUM` controls storage growth by deleting obsolete files, but it must respect recovery and audit requirements.
- Schema evolution should be planned and governed because Silver and Gold tables are contracts for downstream users.
- Part I closes with an integrated lakehouse that uses OneLake shortcuts, Spark transformations, Delta tables, V-Order, and a Gold star schema.

## 4.10 Exercises

1. **(Conceptual)** Explain why a Delta table needs both Parquet files and a transaction log.
2. **(Conceptual)** Compare compaction, V-Order, Z-Ordering, and partitioning. Which problem does each address?
3. **(Hands-on)** Create a training Delta table, run `DESCRIBE HISTORY`, and identify the operation that created the latest version.
4. **(Hands-on)** Run an optimization command on a non-production table and compare row counts before and after.
5. **(Recovery)** Simulate an accidental delete in a training copy and query the previous version with `VERSION AS OF`.
6. **(Operations)** Draft a retention policy that balances 30 days of recovery with storage cleanup.
7. **(Design)** Choose Z-Order columns for a Silver IoT table with filters by event date, device, region, and alert type. Explain your choice.
8. **(Interview)** In five sentences, explain the small-file problem and how you would prevent it in a streaming pipeline.

## 4.11 Milestone 1 Capstone: Spark-Driven Medallion Lakehouse

### Capstone Project

**Mission.** Close Part I by building a Spark-driven medallion lakehouse in Microsoft Fabric. The solution must create Bronze, Silver, and Gold lakehouses, ingest sample data through OneLake shortcuts, clean and conform data with parameterized Spark notebooks, publish a Gold star schema, and optimize Delta tables with V-Order.

**Estimated time.** 6–8 hours, depending on tenant permissions, shortcut source availability, and sample-data size.

**What you'll practice.**

- Creating workspaces and lakehouses aligned to Bronze, Silver, and Gold responsibilities.
- Using OneLake shortcuts to avoid unnecessary source-data copying.
- Writing parameterized PySpark notebooks for repeatable Silver transformations.

- Building Gold facts and dimensions from curated Silver Delta tables.
- Running `OPTIMIZE` with V-Order and validating table history.

## Scenario

Contoso Retail wants a Part I reference implementation for Fabric data engineering. Source files include customers, products, stores, orders, and order lines. Some data may remain in external object storage or instructor-provided locations; Fabric should expose it through OneLake shortcuts. The engineering team must land raw evidence in Bronze, clean and standardize records in Silver, and publish Gold star-schema tables for Power BI and SQL consumers.

## Requirements

Build a working design with these minimum elements:

- A workspace named `fabric-de-milestone1` or an instructor-approved equivalent.
- Lakehouses named `lh_bronze`, `lh_silver`, and `lh_gold`, or clearly separated layer paths if the tenant limits lakehouse attachments.
- OneLake shortcuts in Bronze for at least two sample sources, such as orders and products.
- A parameterized Silver notebook that accepts `processing_date` and `source_system`.
- Silver Delta tables with typed dates, deduplicated business keys, and required null checks.
- Gold star-schema tables such as `dim_customer`, `dim_product`, `dim_date`, `dim_store`, and `fact_sales`.
- V-Order optimization for the major Silver and Gold Delta tables.
- A short operations note covering retry unit, retention, optimization cadence, and data-quality metrics.

## Reference Architecture

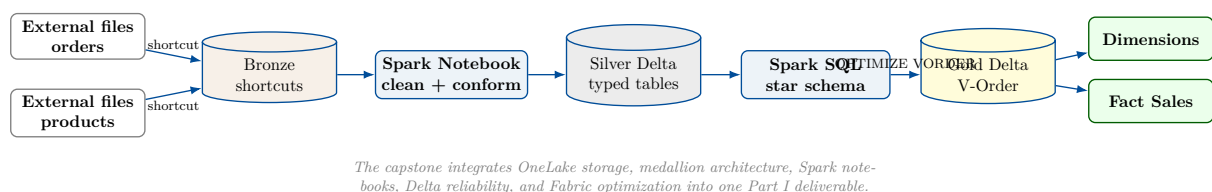


Figure 4.3: Milestone 1 reference architecture from Bronze shortcuts to V-Order optimized Gold tables.

## Implementation Steps

1. **Create the workspace.** Create `fabric-de-milestone1` and assign the correct Fabric capacity.
2. **Create lakehouses.** Add `lh_bronze`, `lh_silver`, and `lh_gold`. Document any tenant-specific deviations.
3. **Create shortcuts.** In Bronze, create OneLake shortcuts to sample orders, order lines, customers, products, and stores where available.

4. **Build Silver.** Use PySpark to read raw files or shortcut paths, apply explicit schemas, drop invalid rows, deduplicate keys, and write Delta tables.
5. **Build Gold.** Use Spark SQL or DataFrame logic to create dimensions and `fact_sales` from Silver.
6. **Optimize.** Run OPTIMIZE with V-Order on important Silver and Gold tables.
7. **Validate.** Compare Bronze counts, Silver accepted and rejected counts, and Gold aggregate totals.
8. **Document operations.** Record retry unit, retention policy, optimization cadence, and ownership.

```

1 from pyspark.sql import functions as F
2 from pyspark.sql.types import (
3     StructType, StructField, StringType, IntegerType, DoubleType
4 )
5
6 processing_date = "2026-07-31"
7 source_system = "retail_sample"
8
9 orders_schema = StructType([
10     StructField("order_id", StringType(), True),
11     StructField("customer_id", StringType(), True),
12     StructField("store_id", StringType(), True),
13     StructField("order_timestamp", StringType(), True),
14     StructField("order_status", StringType(), True),
15 ])
16
17 order_lines_schema = StructType([
18     StructField("order_id", StringType(), True),
19     StructField("product_id", StringType(), True),
20     StructField("quantity", IntegerType(), True),
21     StructField("unit_price", DoubleType(), True),
22 ])
23
24 orders_path = f"Files/bronze/shortcuts/orders/order_date={processing_date}/*.csv"
25 lines_path = f"Files/bronze/shortcuts/order_lines/order_date={processing_date}/*.csv"
26
27 orders_df = (
28     spark.read.option("header", "true").schema(orders_schema).csv(orders_path)
29     .withColumn("source_system", F.lit(source_system))
30     .withColumn("ingested_at", F.current_timestamp())
31     .dropDuplicates(["order_id"])
32     .dropna(subset=["order_id", "customer_id", "order_timestamp"])
33     .withColumn("order_timestamp", F.to_timestamp("order_timestamp"))
34     .withColumn("order_date", F.to_date("order_timestamp"))
35     .where(F.col("order_timestamp").isNotNull())
36 )
37
38 lines_df = (
39     spark.read.option("header", "true").schema(order_lines_schema).csv(
40         lines_path)
41     .dropna(subset=["order_id", "product_id"])
42     .withColumn("quantity", F.col("quantity").cast("int"))
43     .withColumn("unit_price", F.col("unit_price").cast("double"))
44     .where((F.col("quantity") > 0) & (F.col("unit_price") >= 0))
45 )
46 orders_df.write.format("delta").mode("overwrite").saveAsTable("silver_orders")

```

```

47 lines_df.write.format("delta").mode("overwrite").saveAsTable("
    silver_order_lines")

```

Listing 4.5: Parameterized Silver transformation for the Milestone 1 capstone.

```

1 CREATE OR REPLACE TABLE dim_customer AS
2 SELECT DISTINCT
3     customer_id,
4     source_system
5 FROM silver_orders
6 WHERE customer_id IS NOT NULL;
7
8 CREATE OR REPLACE TABLE dim_date AS
9 SELECT DISTINCT
10    order_date,
11    YEAR(order_date) AS calendar_year,
12    MONTH(order_date) AS calendar_month,
13    DAY(order_date) AS calendar_day
14 FROM silver_orders
15 WHERE order_date IS NOT NULL;
16
17 CREATE OR REPLACE TABLE fact_sales AS
18 SELECT
19     o.order_id,
20     o.customer_id,
21     o.store_id,
22     l.product_id,
23     o.order_date,
24     l.quantity,
25     l.unit_price,
26     l.quantity * l.unit_price AS sales_amount
27 FROM silver_orders o
28 JOIN silver_order_lines l
29     ON o.order_id = l.order_id;
30
31 OPTIMIZE silver_orders VORDER;
32 OPTIMIZE silver_order_lines VORDER;
33 OPTIMIZE dim_customer VORDER;
34 OPTIMIZE dim_date VORDER;
35 OPTIMIZE fact_sales VORDER;

```

Listing 4.6: Gold star schema and V-Order optimization.

## Deliverables

Submit evidence that the Part I milestone is complete:

- Workspace and lakehouse names, including Bronze, Silver, and Gold boundaries.
- Screenshot or exported evidence of OneLake shortcuts in Bronze.
- Notebook parameters and successful Silver transformation output.
- Schemas and row counts for Silver Delta tables.
- Gold star-schema table list with at least one fact and three dimensions.
- DESCRIBE HISTORY evidence showing optimization on key Delta tables.
- A short operations note for retry, retention, optimization cadence, and data-quality checks.

## Acceptance Criteria

Use this rubric before declaring Milestone 1 complete:



- ☐ Bronze, Silver, and Gold storage boundaries are visible and documented.
- ☐ At least two Bronze data sources are exposed through OneLake shortcuts or clearly documented substitutes.
- ☐ Silver notebooks use parameters rather than hardcoded personal paths.
- ☐ Silver tables enforce required keys, types, and duplicate rules.
- ☐ Gold contains a star schema with dimensions and **fact\_sales**.
- ☐ Major Silver and Gold tables are Delta tables optimized with V-Order where supported.
- ☐ Row-count reconciliation proves that rejected and accepted records are understood.
- ☐ The design states retention, Time Travel, and **VACUUM** expectations.
- ☐ No secrets, real customer identifiers, or personal workspace paths appear in submitted evidence.
- ☐ The project demonstrates Weeks 1–4 as one coherent lakehouse workflow.

### Stretch Goals

If you finish early, extend the milestone without changing its core contract:

- Add a quarantine table with rejection reasons for invalid Bronze records.
- Add a product and store dimension with slowly changing attribute notes.
- Add Time Travel validation that compares the latest Gold totals to a previous version.
- Add a data-quality metrics table for each processing date.
- Create a Direct Lake semantic model over the Gold star schema.
- Document how the development workspace would promote to test and production.

## Appendix A

# Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the six platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, and Snowflake. The names differ, but the underlying data-engineering concepts—durable object storage, tiered lifecycle economics, immutability, managed relational engines, elastic compute, and disciplined data organization—recur everywhere.

### Note

This appendix grows with each chapter. Every new chapter contributes the equivalence tables introduced in its cross-cloud callout, so by the end of the book you will hold a single consolidated Rosetta Stone for moving fluently between clouds. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

## A.1 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

| Capability          | AWS                        | Azure                   | GCP                          |
|---------------------|----------------------------|-------------------------|------------------------------|
| Object storage      | Amazon S3                  | ADLS Gen2 / Blob        | Cloud Storage                |
| Hot tier            | S3 Standard                | Hot                     | Standard                     |
| Infrequent access   | S3 Standard-IA             | Cool                    | Nearline                     |
| Archive (instant)   | Glacier Instant Re-trieval | Cold                    | Coldline                     |
| Deep archive        | Glacier Deep Archive       | Archive                 | Archive                      |
| Lifecycle rules     | Lifecycle policies         | Lifecycle management    | Object Lifecycle Mgmt        |
| Versioning          | S3 Versioning              | Blob versioning         | Object Versioning            |
| Immutability (WORM) | Object Lock                | Immutable blob stor-age | Bucket Lock                  |
| Cross-region copy   | CRR / SRR                  | Object replication      | Dual/multi-region + Transfer |
| Encryption at rest  | SSE-S3 / SSE-KMS           | SSE + CMK (Key Vault)   | Google-managed / CMEK        |

The three remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in cloud object storage as immutable, columnar *micro-partitions* that the service manages on your

behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers.

## A.2 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

| Capability          | AWS                     | Azure                        | GCP                    |
|---------------------|-------------------------|------------------------------|------------------------|
| Managed relational  | Amazon RDS              | Azure SQL Database           | Cloud SQL              |
| Cloud-native engine | Amazon Aurora           | SQL Hyperscale               | AlloyDB                |
| Managed PostgreSQL  | RDS / Aurora PostgreSQL | Azure DB for PostgreSQL      | Cloud SQL / AlloyDB    |
| High availability   | Multi-AZ                | Zone-redundant HA            | Regional HA            |
| Read scaling        | Read Replicas           | Read replicas / geo-replicas | Read Replicas          |
| Global / geo        | Aurora Database Global  | Active geo-replication       | Cross-region replicas  |
| Serverless compute  | Aurora Serverless v2    | Azure SQL Serverless         | AlloyDB (elastic)      |
| Migration service   | DMS + SCT               | Azure Database Migration Svc | Database Migration Svc |

**Snowflake**, **Fabric**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling.

## A.3 Elastic Compute and Analytical Warehouses

Snowflake’s Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

| Concept             | Snowflake               | AWS                       | Azure / GCP                    |
|---------------------|-------------------------|---------------------------|--------------------------------|
| Compute unit        | Virtual Warehouse       | Redshift RA3 / Serverless | Synapse pool / Big-Query slots |
| Elastic concurrency | Multi-cluster warehouse | Concurrency scaling       | Autoscale / on-demand          |
| Pause when idle     | Auto-suspend / resume   | Pause / resume            | Auto-pause / serverless        |
| Billing unit        | Credits                 | RPU / node-hours          | DWU / slot-hours               |
| Result reuse        | Result cache            | Result cache              | Result / BI cache              |

**Microsoft Fabric** meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage.

## A.4 Data Organization and the Medallion Pattern

Fabric’s Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

| Layer              | Fabric           | Data<br>(AWS/Azure/GCP)        | lake   | Snowflake           |
|--------------------|------------------|--------------------------------|--------|---------------------|
| Bronze (raw)       | Bronze Lakehouse | raw/<br>(S3/ADLS/GCS)          | prefix | RAW schema / stage  |
| Silver (validated) | Silver Lakehouse | validated/ prefix              |        | STAGING schema      |
| Gold (curated)     | Gold Lakehouse   | curated/ prefix                |        | ANALYTICS<br>schema |
| Storage format     | Delta-Parquet    | Parquet / Delta / Ice-<br>berg |        | Micro-partitions    |

**MongoDB** realizes the same separation with distinct collections or databases (for example **raw\_events**, **validated**, **curated**) governed by schema-design patterns rather than a lake layout.

## A.5 Document Data Modeling

MongoDB’s Chapter 2 schema-design patterns generalize to every document and wide-column store.

| Concept              | MongoDB               | Amazon<br>nomoDB    | Dy- | Azure Cosmos DB               |
|----------------------|-----------------------|---------------------|-----|-------------------------------|
| Primary access unit  | Document / collection | Item / table        |     | Item / container              |
| Partitioning         | Shard key             | Partition key       |     | Partition key                 |
| One-table modeling   | Embedding + patterns  | Single-table design |     | Embedding + parti-<br>tioning |
| Time-series grouping | Bucket Pattern        | Composite sort keys |     | Bucketing by key              |
| Optional attributes  | Attribute Pattern     | Sparse attributes   |     | Sparse properties             |

Google Cloud’s closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in **VARIANT** (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections.

## A.6 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

| Concept           | DynamoDB          | Cosmos DB           | Cloud Spanner                   | MongoDB                          |
|-------------------|-------------------|---------------------|---------------------------------|----------------------------------|
| Model             | Key-value / doc   | Multi-model         | Distributed SQL                 | Document                         |
| Partitioning      | Partition key     | Partition key       | Split by key range              | Shard key                        |
| Throughput unit   | RCU / WCU         | Request Units (RU)  | Nodes / processing<br>units     | Cluster tier                     |
| Secondary indexes | GSI / LSI         | Automatic indexing  | Secondary indexes               | Indexes                          |
| Change stream     | DynamoDB Streams  | Change Feed         | Change streams                  | Change streams                   |
| Expiry            | TTL               | TTL                 | (manual)                        | TTL index                        |
| Multi-region      | Global Tables     | Multi-region writes | Multi-region (syn-<br>chronous) | Zones / global<br>clusters       |
| Consistency       | Eventual / strong | 5 tunable levels    | External (True-<br>Time)        | Tunable<br>read/write<br>concern |

**Snowflake** and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in **VARIANT**/JSON columns. Google’s dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column).

## A.7 Managed Apache Spark

Microsoft Fabric’s Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

| Capability        | Fabric                 | AWS                         | Azure / GCP                             |
|-------------------|------------------------|-----------------------------|-----------------------------------------|
| Managed Spark     | Starter / Custom pools | Glue / EMR / EMR Serverless | Synapse Spark / Databricks; Dataproc    |
| Notebook UX       | Fabric Notebooks       | Glue Studio / EMR Studio    | Synapse / Databricks; Vertex AI         |
| Serverless option | Autoscaling pools      | Glue / EMR Serverless       | Synapse serverless; Dataproc Serverless |
| Lake format       | Delta on OneLake       | Delta / Iceberg / Hudi      | Delta; Delta / Iceberg                  |

**Snowflake** offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector.

## A.8 Query Acceleration and Materialized Views

Snowflake’s Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

| Feature             | Snowflake               | AWS Redshift          | Azure Synapse / GCP BigQuery |
|---------------------|-------------------------|-----------------------|------------------------------|
| Materialized views  | Materialized Views      | Materialized views    | Materialized views           |
| Point-lookup accel. | Search Optimization Svc | Sort keys / zone maps | (indexing) / search indexes  |
| Result reuse        | Result cache            | Result cache          | Result-set cache / BI Engine |
| Ephemeral tables    | Transient / temporary   | Temp tables           | Temp tables                  |

**Fabric** accelerates reads through Direct Lake and its Warehouse engine, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set.

## A.9 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

| Capability          | AWS                               | Azure                    | GCP / others                  |
|---------------------|-----------------------------------|--------------------------|-------------------------------|
| Managed cache       | ElastiCache (Redis/Memcached)     | Cache for Redis          | Memorystore (Redis/Memcached) |
| Real-time API layer | AppSync / API Gateway             | API Management / SignalR | API Gateway / Apigee          |
| Hybrid ingestion    | DataSync / Snow family / IoT Core | Data Box / IoT Hub / Arc | Transfer Appliance / IoT Core |
| Streaming intake    | Kinesis / MSK                     | Event Hubs               | Pub/Sub                       |

**Snowflake** and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache.

## A.10 Wide-Column and Key-Value NoSQL

GCP’s Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

| Characteristic   | GCP Bigtable              | AWS                      | Azure / open source            |
|------------------|---------------------------|--------------------------|--------------------------------|
| Data model       | Wide-column               | DynamoDB (key-value/doc) | Cosmos DB (multi-model)        |
| HBase-compatible | HBase API                 | Keyspaces (Cassandra)    | Managed Instance for Cassandra |
| Scaling unit     | Nodes / clusters          | On-demand / provisioned  | RU/s (Cosmos)                  |
| Best fit         | Time-series, IoT, ad-tech | Serverless key-value     | Global multi-model             |

**Snowflake** and **Fabric** address these workloads analytically rather than as operational key-value stores; **MongoDB** covers the document end of the same NoSQL spectrum.

## A.11 Data Protection: Cloning, Time Travel, and Migration

Snowflake’s Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB’s relational-to-document migration map to data-protection and migration tooling across the clouds.

| Capability             | Snowflake          | AWS / Azure                | GCP / Fabric                           |
|------------------------|--------------------|----------------------------|----------------------------------------|
| Instant clone          | Zero-copy clone    | Aurora clone / SQL DB copy | BigQuery table clone; OneLake shortcut |
| Point-in-time recovery | Time Travel        | PITR (RDS/SQL DB backups)  | PITR; Delta time travel                |
| Retention safety net   | Fail-safe (7 days) | Automated backups          | Backups; Delta VACUUM window           |
| Schema migration       | —                  | DMS / SCT                  | Database Migration Service             |

**MongoDB** migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, and **Delta Lake** exposes historical versions through `VERSION AS OF` time-travel queries.

### Tip

Use this appendix as a study aid for the book’s lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other five would express the same idea. That habit turns six separate vocabularies into one mental model.

# Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

**Access tier** A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

**AlloyDB** Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

**Apache Spark** A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

**Aurora** Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

**Availability SLA** The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

**Bigtable** Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

**Bucket (storage)** The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

**Bucket Pattern** A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

**Capped collection** A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

**Change Feed / Streams** An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

**Checkpoint** In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

**Clustering key** A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

**CMEK / CMK** Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

**Cloud Spanner** Google Cloud’s horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

**Consistency level** A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).

**Cosmos DB** Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).

**Cross-Region Replication (CRR)** Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure’s equivalent user-configured feature is *object replication*.

**DynamoDB** Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.

**Durability** The probability that stored data is not lost, commonly quoted as “eleven nines” (99.999999999%) for cloud object storage.

**ElastiCache** Amazon’s managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure’s analog is Cache for Redis and GCP’s is Memorystore.

**Embedding vs referencing** The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).

**Fail-safe** Snowflake’s non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.

**Global Secondary Index (GSI)** A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table’s partition key.

**Hierarchical namespace (HNS)** The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.

**Journal (WAL)** The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB’s WiredTiger engine.

**JSON Schema validation** MongoDB’s rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.

**Interleaved table** A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.

**Lakehouse** An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.

**Lifecycle policy** A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.

**Medallion architecture** A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.

**Materialized view** A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).

**Memorystore** Google Cloud’s managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.



- Micro-partition** Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.
- Multi-AZ** An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.
- Multi-cluster warehouse** A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.
- Object Lock** The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.
- OneLake** Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.
- Read replica** A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.
- Relational Migrator** MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.
- Request Unit (RU)** Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.
- Search Optimization Service (SOS)** A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.
- Shortcut (Fabric)** A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.
- Storage class** The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).
- SSE-KMS** Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.
- Time Travel** Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF / TIMESTAMP AS OF` capability.
- Transient table** A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.
- TrueTime** Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.
- TTL (Time to Live)** An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).
- Versioning** Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

**Virtual warehouse** Snowflake's unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

**Wide-column store** A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

**WiredTiger** MongoDB's default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

**WORM** Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

**Z-ordering** A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

**Zero-copy clone** Snowflake's instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

# Bibliography

- [1] Apache Software Foundation. Apache parquet documentation. <https://parquet.apache.org/docs/>, 2026. Open file format documentation.
- [2] Apache Software Foundation. Apache spark overview. <https://spark.apache.org/docs/latest/>, 2026. Apache Spark documentation.
- [3] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Luszczak, et al. Delta lake: High-performance acid table storage over cloud object stores. In *Proceedings of the VLDB Endowment*, volume 13, pages 3411–3424, 2020.
- [4] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2021.
- [5] Databricks. What is the medallion lakehouse architecture? <https://www.databricks.com/glossary/medallion-architecture>, 2025. Architecture documentation.
- [6] Linux Foundation Delta Lake Project. Delta lake documentation. <https://docs.delta.io/>, 2026. Open table format documentation.
- [7] Microsoft. Apache spark compute in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/spark-compute>, 2026. Microsoft Learn documentation.
- [8] Microsoft. Create, configure, and use an environment in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/create-and-use-environment>, 2026. Microsoft Learn documentation.
- [9] Microsoft. Delta lake table optimization and v-order in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/delta-optimization-and-v-order>, 2026. Microsoft Learn documentation.
- [10] Microsoft. Direct lake in power bi and microsoft fabric. <https://learn.microsoft.com/fabric/get-started/direct-lake-overview>, 2026. Microsoft Learn documentation.
- [11] Microsoft. Domains in fabric. <https://learn.microsoft.com/fabric/governance/domains>, 2026. Microsoft Learn documentation.
- [12] Microsoft. How to use microsoft fabric notebooks. <https://learn.microsoft.com/fabric/data-engineering/how-to-use-notebook>, 2026. Microsoft Learn documentation.
- [13] Microsoft. Microsoft fabric capacity metrics app. <https://learn.microsoft.com/fabric/enterprise/metrics-app>, 2026. Microsoft Learn documentation.
- [14] Microsoft. Microsoft fabric concepts and licenses. <https://learn.microsoft.com/fabric/enterprise/licenses>, 2026. Microsoft Learn documentation.

- [15] Microsoft. Microsoft fabric tenant settings. <https://learn.microsoft.com/fabric/admin/fabric-switch>, 2026. Microsoft Learn documentation.
- [16] Microsoft. Microsoft fabric trial capacity. <https://learn.microsoft.com/fabric/get-started/fabric-trial>, 2026. Microsoft Learn documentation.
- [17] Microsoft. Onelake shortcuts. <https://learn.microsoft.com/fabric/onelake/onelake-shortcuts>, 2026. Microsoft Learn documentation.
- [18] Microsoft. Optimize delta lake tables in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/lakehouse-table-maintenance>, 2026. Microsoft Learn documentation.
- [19] Microsoft. Vacuum delta tables in microsoft fabric. <https://learn.microsoft.com/fabric/data-engineering/lakehouse-table-maintenance>, 2026. Microsoft Learn documentation.
- [20] Microsoft. What is a lakehouse in microsoft fabric? <https://learn.microsoft.com/fabric/data-engineering/lakehouse-overview>, 2026. Microsoft Learn documentation.
- [21] Microsoft. What is microsoft fabric? <https://learn.microsoft.com/fabric/get-started/microsoft-fabric-overview>, 2026. Microsoft Learn documentation.
- [22] Microsoft. What is onelake? <https://learn.microsoft.com/fabric/onelake/onelake-overview>, 2026. Microsoft Learn documentation.
- [23] Microsoft. Workspaces in microsoft fabric. <https://learn.microsoft.com/fabric/fundamentals/workspaces>, 2026. Microsoft Learn documentation.

# Index

- 100 GB daily processing, [57](#)
- ACID, [19](#)
- ADLS Gen2, [20](#)
- Admin portal, [5](#)
- Admin role, [6](#)
- Amazon S3, [20](#), [24](#)
- Apache Spark, [50](#)
  - equivalence, [84](#)
- Bigtable
  - equivalence, [85](#)
- Bronze layer, [33](#), [45](#)
  - CSV, [55](#)
  - JSON, [61](#)
  - raw, [35](#)
- bursting, [8](#)
- cached
  - equivalence, [84](#)
- capacity, [17](#)
  - trial, [1](#), [4](#)
- capacity admin, [5](#)
- capacity metrics, [8](#)
- capacity unit, [17](#)
- capacity units, [4](#), [14](#)
- capstone project, [27](#), [45](#), [61](#)
  - Part I, [76](#)
- cloning
  - equivalence, [85](#)
- Contributor role, [6](#)
- cost management, [8](#)
- cross-cloud equivalence, [81](#)
- CSV ingestion, [39](#)
- curated reports, [41](#)
- Custom pools, [50](#), [52](#)
- data curation, [33](#)
- data skipping, [69](#)
- data virtualization, [14](#), [20](#)
- Data Warehouse, [33](#)
  - Fabric, [36](#)
- Dataverse, [20](#), [24](#)
- Delta Lake, [14](#), [19](#), [33](#), [66](#)
  - compaction, [73](#)
  - history, [70](#)
  - internals, [68](#)
  - optimization, [66](#), [69](#)
  - partitioning, [61](#)
  - remediation, [73](#)
  - retention, [70](#)
  - Spark, [50](#)
  - transaction log, [66](#), [68](#)
- Delta-Parquet, [18](#)
- Direct Lake, [36](#)
- distributed databases
  - equivalence, [83](#)
- document modeling
  - equivalence, [83](#)
- domain, [18](#), [38](#)
- elastic compute
  - equivalence, [82](#)
- environment promotion, [38](#)
- F SKU, [4](#), [17](#)
- Fabric capacity, [14](#)
- Fabric Notebooks, [50](#), [53](#)
- Fabric portal, [7](#)
- Fabric trial, [1](#)
  - starting, [4](#)
- financial institution, [33](#), [41](#)
- financial medallion, [45](#)
- glossary, [86](#)
- Gold layer, [33](#), [45](#)
  - curated, [35](#)
  - star schema, [76](#)
- hands-on lab, [21](#), [39](#), [55](#), [71](#)
- in-memory cache
  - equivalence, [84](#)
- JSON logs, [57](#)
- JSON transaction log, [68](#)
- Lakehouse, [14](#), [33](#)
  - versus warehouse, [36](#)

- lakehouse
  - creation, [1](#), [21](#)
  - organization, [38](#)
- landing zone, [27](#)
- materialized views
  - equivalence, [84](#)
- medallion architecture, [14](#), [19](#), [33](#)
  - capstone, [76](#)
  - equivalence, [82](#)
  - principles, [35](#)
- Member role, [6](#)
- Microsoft 365 Developer Program, [3](#)
- Microsoft Entra ID, [3](#)
- Microsoft Fabric, [14](#)
  - Apache Spark, [50](#)
  - Delta Lake, [66](#)
  - getting started, [1](#)
  - overview, [16](#)
- Milestone 1 capstone, [76](#)
- multi-cloud, [81](#)
- multi-cloud architecture, [24](#)
- multi-cloud retail, [27](#)
- NoSQL
  - equivalence, [83](#)
- notebook
  - verification, [9](#)
- notebooks, [7](#)
- null handling, [55](#)
- object storage
  - equivalence, [81](#)
- OneDrive for Data, [18](#)
- OneLake, [14](#)
  - concepts, [18](#)
  - Files, [53](#)
  - medallion architecture, [33](#)
  - paths, [39](#)
  - shortcuts, [20](#), [27](#)
  - Spark access, [50](#)
- OneLake File Explorer, [7](#)
- OneLake shortcuts
  - capstone, [76](#)
- OPTIMIZE, [66](#), [69](#)
  - V-Order, [71](#)
- organizational account, [1](#)
- Parquet, [19](#), [66](#), [68](#)
- point-in-time recovery
  - equivalence, [85](#)
- Power BI, [1](#), [14](#)
- Power BI Desktop, [7](#)
- Power BI free license, [3](#)
- PySpark, [7](#), [21](#), [50](#)
  - basics, [53](#)
  - deduplication, [55](#)
  - Delta writes, [39](#)
  - Spark SQL integration, [54](#)
- query acceleration
  - equivalence, [84](#)
- raw logs, [41](#)
- regulatory reporting, [41](#)
- relational databases
  - equivalence, [82](#)
- REST API, [7](#)
- retail analytics, [24](#)
- sample file upload, [9](#)
- Schema Evolution, [66](#), [70](#)
- security, [45](#)
- service mapping, [81](#)
- setup verification, [9](#)
- shortcut, [14](#)
- Silver Delta table, [27](#)
- Silver layer, [33](#)
  - Delta, [55](#), [61](#)
  - optimization, [71](#)
  - validated, [35](#)
- small-file problem, [66](#), [73](#)
- software as a service, [16](#)
- Spark environments, [50](#), [52](#)
- Spark Notebooks
  - capstone, [61](#)
  - ingestion, [57](#)
- Spark pools, [50](#)
  - Fabric, [52](#)
- Spark SQL, [50](#), [54](#)
- SQL analytics endpoint, [36](#)
- Star Schema, [76](#)
- Starter pools, [50](#), [52](#)
- storage classes
  - equivalence, [81](#)
- streaming ingestion, [73](#)
- telemetry ingestion, [57](#)
  - capstone, [61](#)
- temporary views, [54](#)
- tenant, [3](#)
- tenant setting, [1](#)
- tenant settings, [5](#)
- throttling, [8](#)
- Time Travel, [66](#), [70](#)
  - accidental deletion, [71](#)

- time travel
  - equivalence, [85](#)
- trial expiry, [8](#)
- Users can create Fabric items, [5](#)
- V-Order, [66](#), [69](#)
  - capstone, [76](#)
- VACUUM, [66](#), [70](#)
- Viewer role, [6](#)
- virtual warehouse
  - equivalence, [82](#)
- wide-column
  - equivalence, [85](#)
- work or school account, [1](#)
- workloads, [16](#)
- workspace, [18](#)
  - capacity assignment, [6](#)
  - creation, [1](#), [21](#)
  - medallion organization, [38](#)
  - roles, [6](#)
- workspace isolation, [33](#), [41](#), [45](#)
- Z-Ordering, [66](#), [69](#)